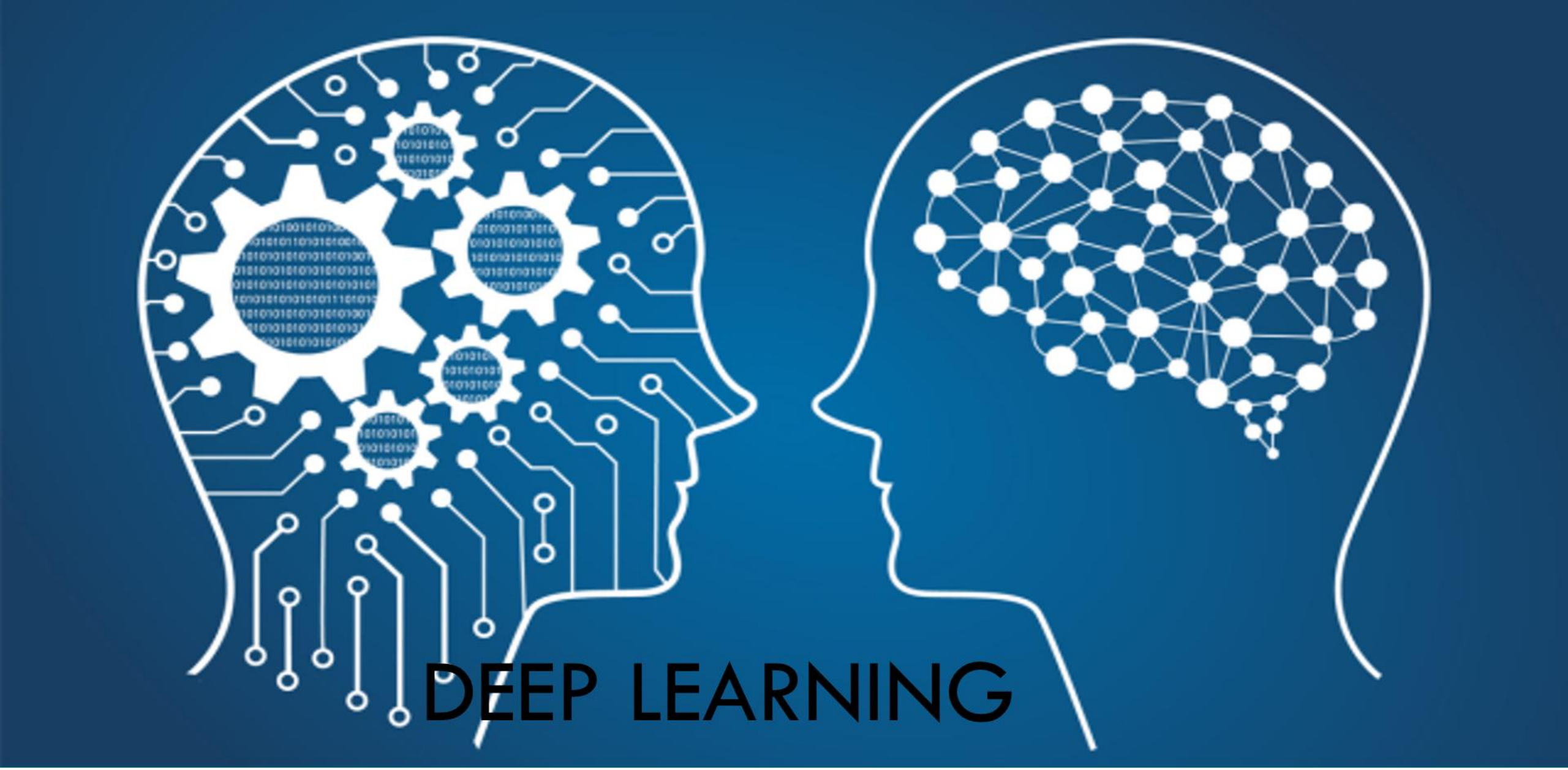
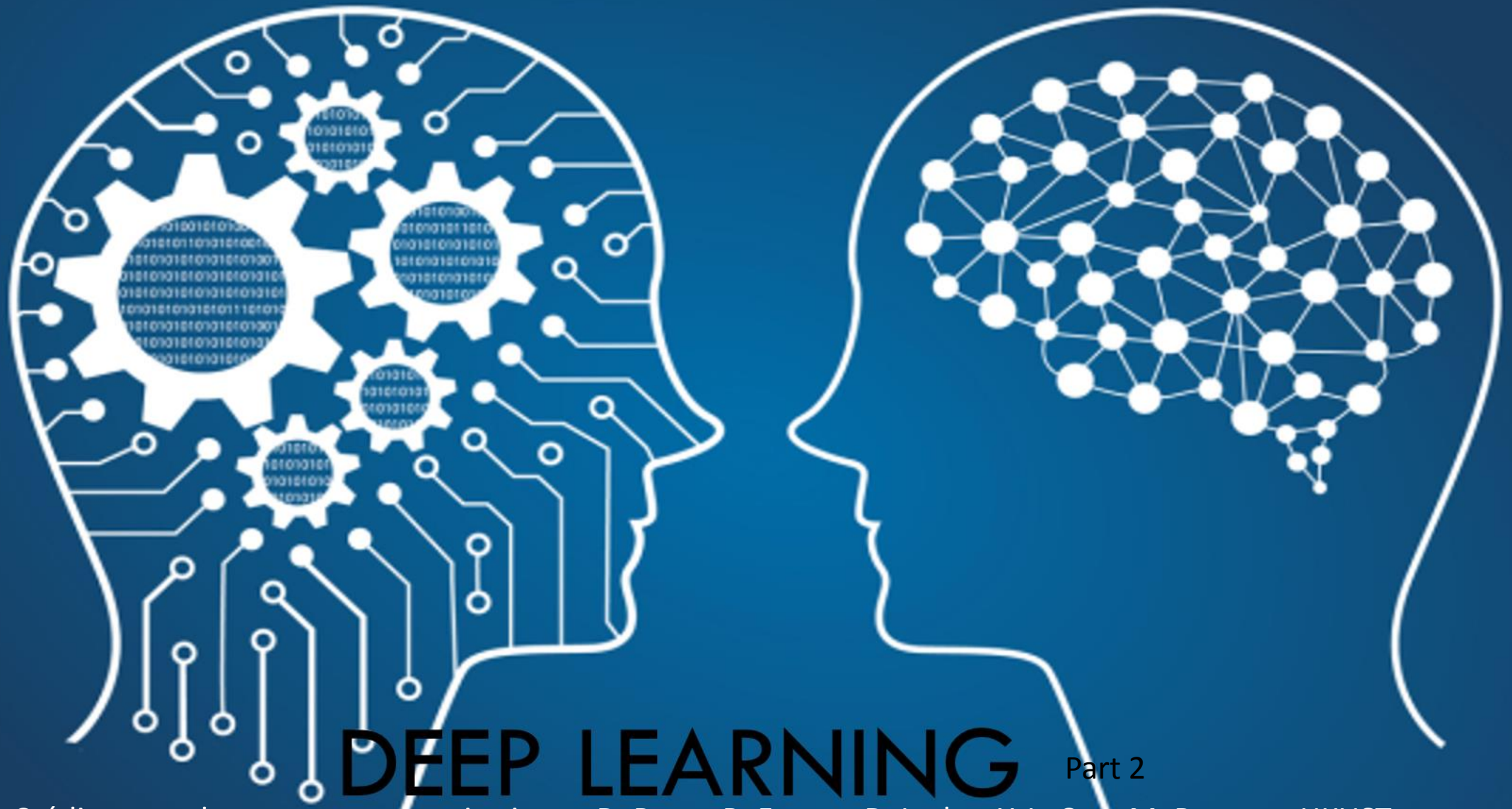


DEEP LEARNING

Pr. BEN LAHMAR EL Habib



DEEP LEARNING

Part 2

Crédits pour des transparents qui suivent: D. Batra, R. Fergus, D. Larlus, Y. LeCun, M. Renzato, HKUST

Pr. BEN LAHMAR EL Habib

Deep learning

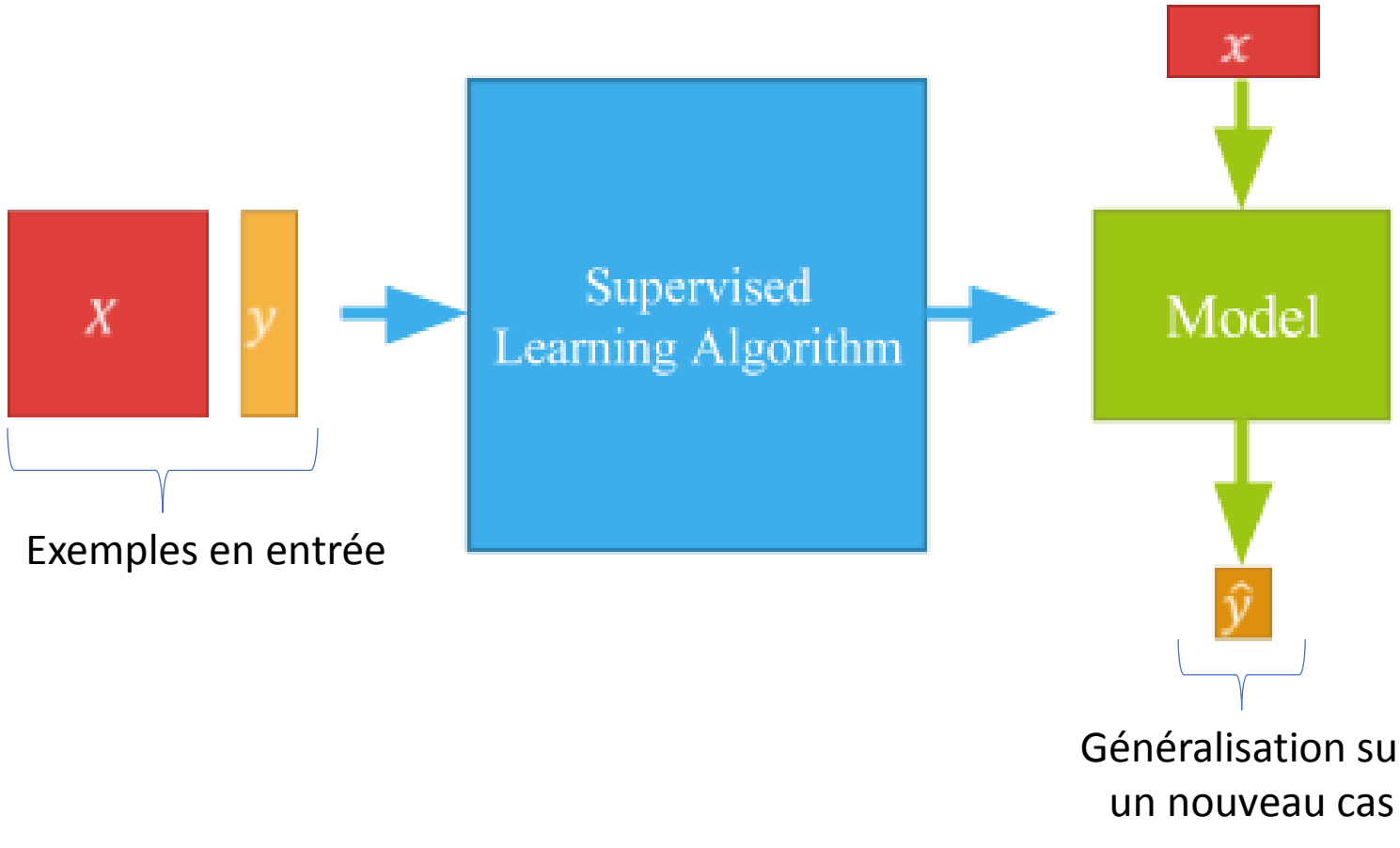


Des réseaux neuronaux artificiels

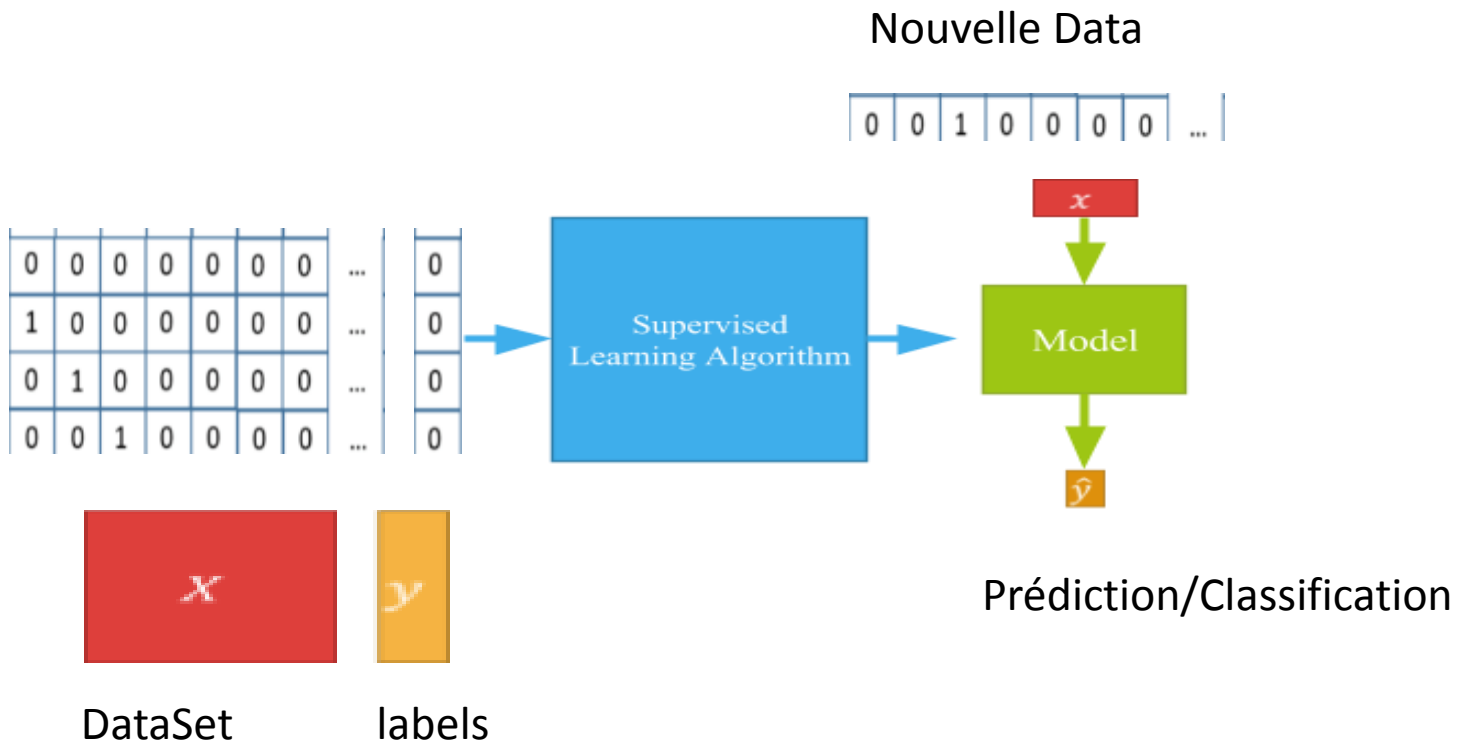
L'une des principales technologies de Machine Learning et d'intelligence artificielle.

Apprentissage supervisé

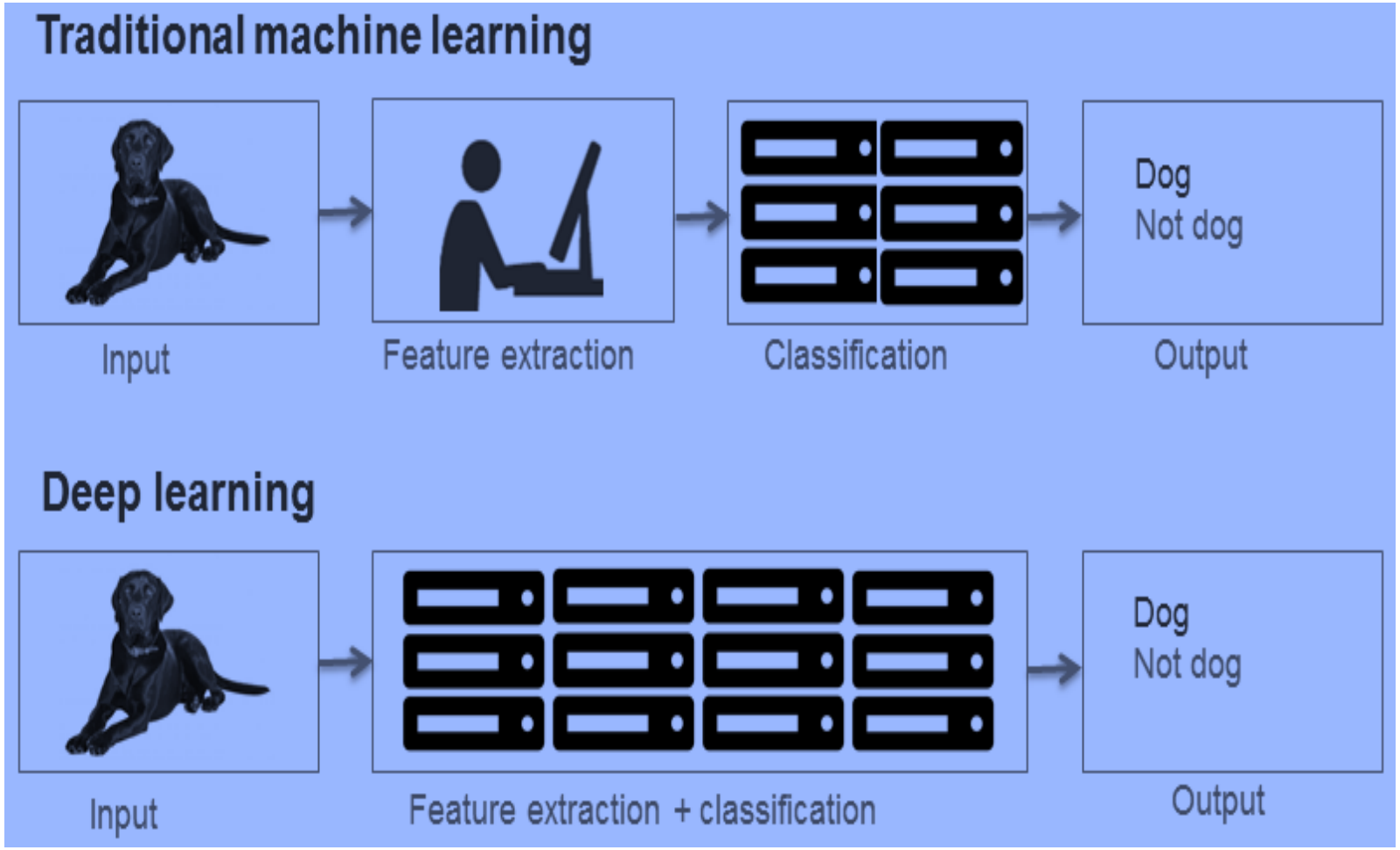
Observer des exemples et produire des généralisation



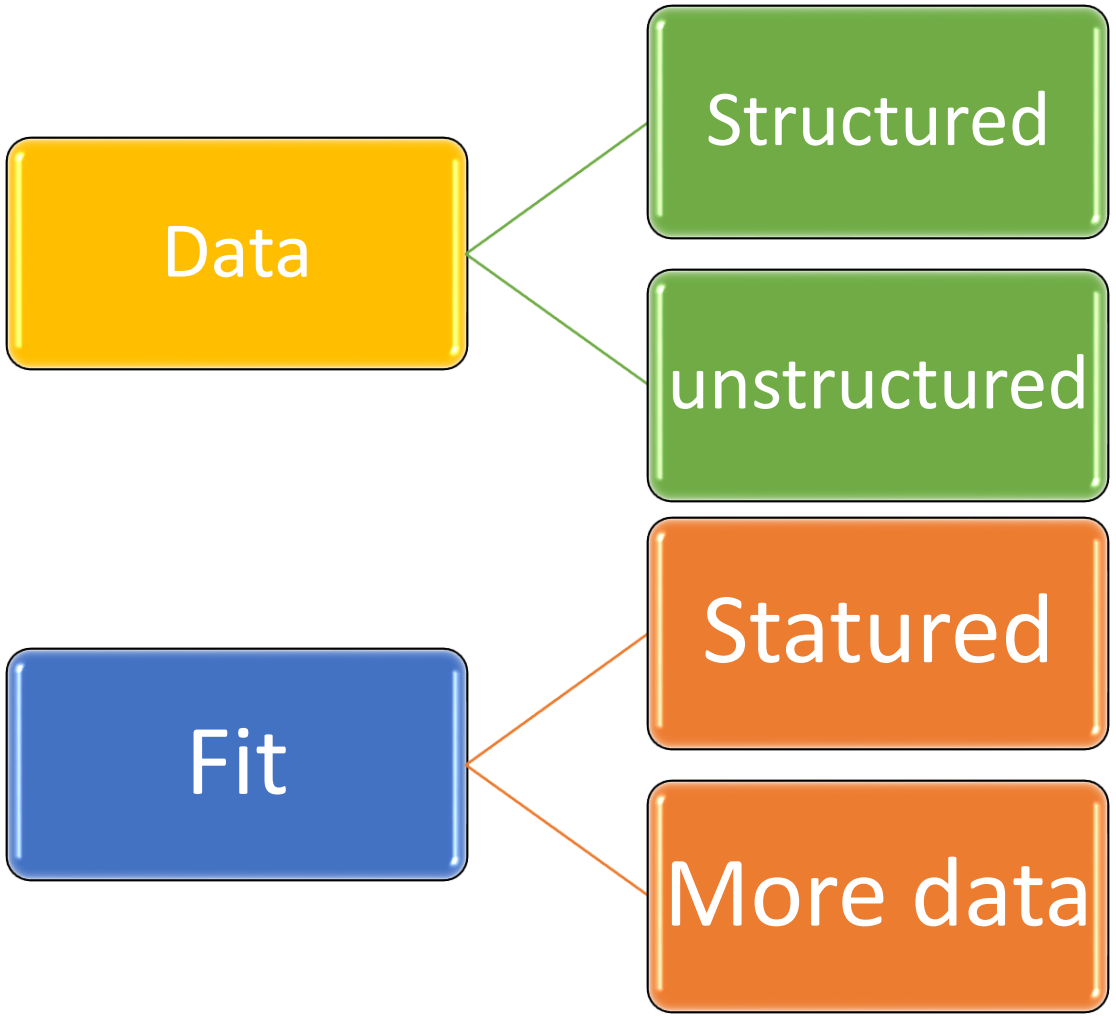
Apprentissage supervisé



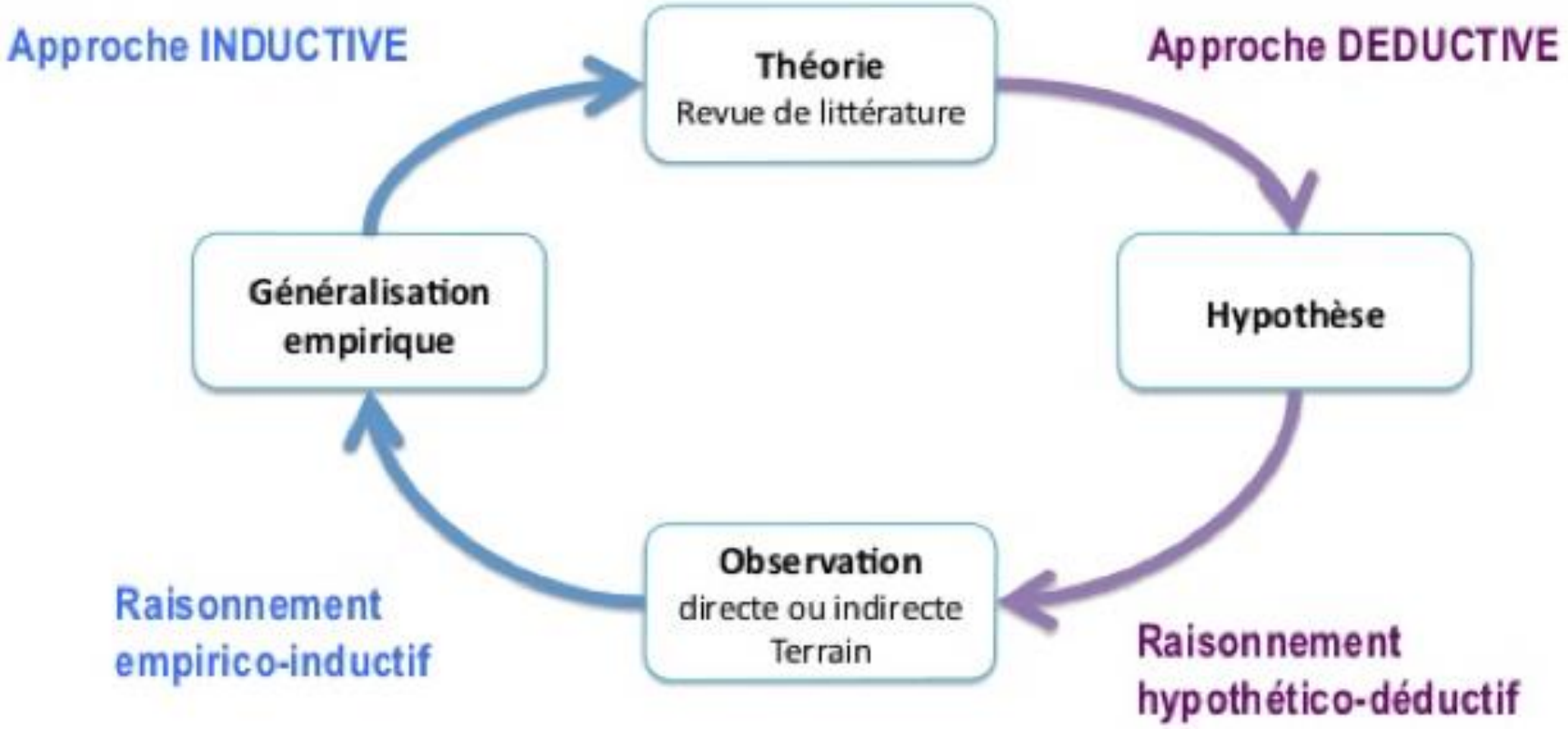
Deep learning Vs Machine learning



Deep learning Vs Machine learning

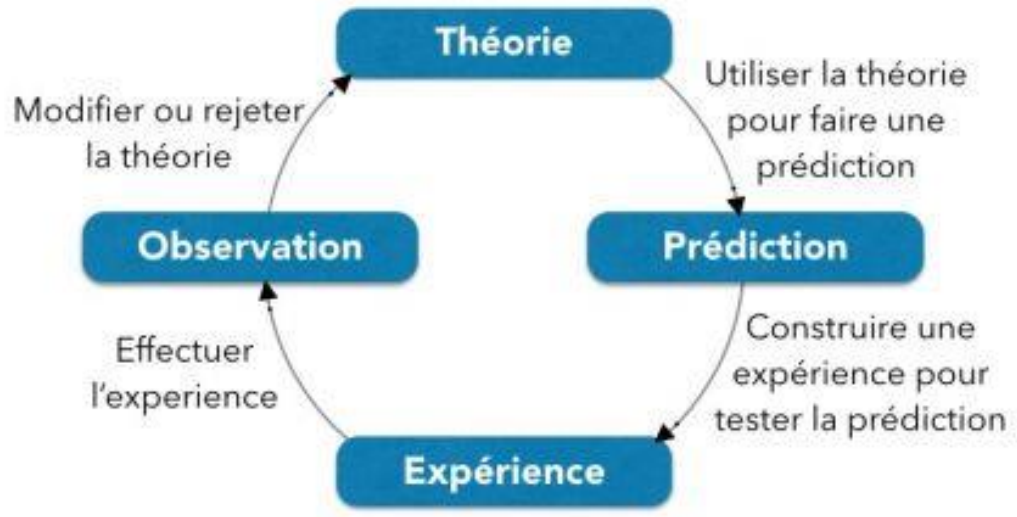


Approches scientifiques



Deep Learning: Une rupture *scientifique ?!*

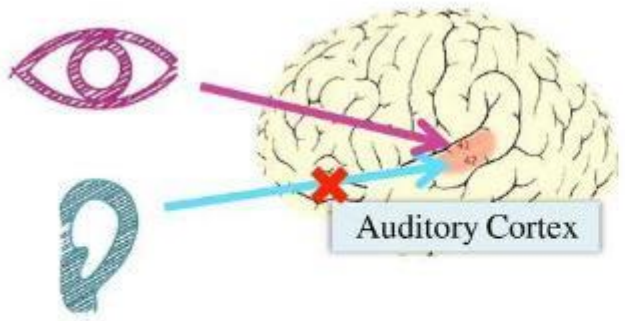
Principes de base de la méthode scientifique : les modèles (ou théories) sont classiquement construits par les chercheurs à partir des observations.



Le deep learning change la donne en assistant l'expertise scientifique dans la construction des modèles.

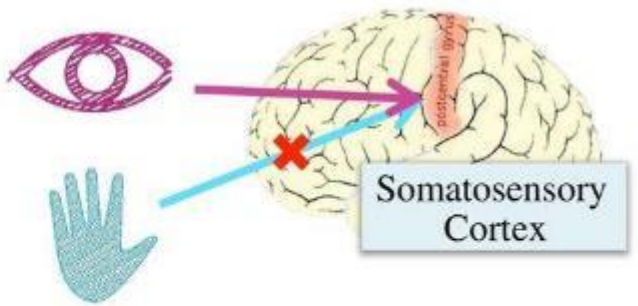
Expériences de recâblage neuronal

neural rewiring experiments



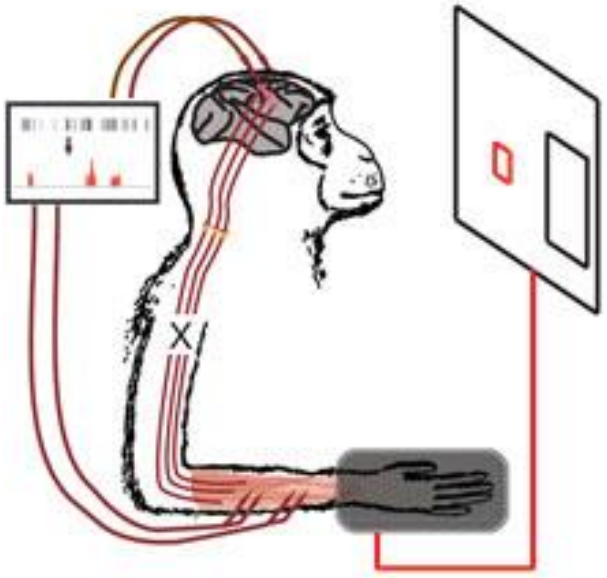
Auditory cortex learns to see

[Roe et al., 1992]



Somatosensory cortex learns to see

[Metin & Frost, 1989]

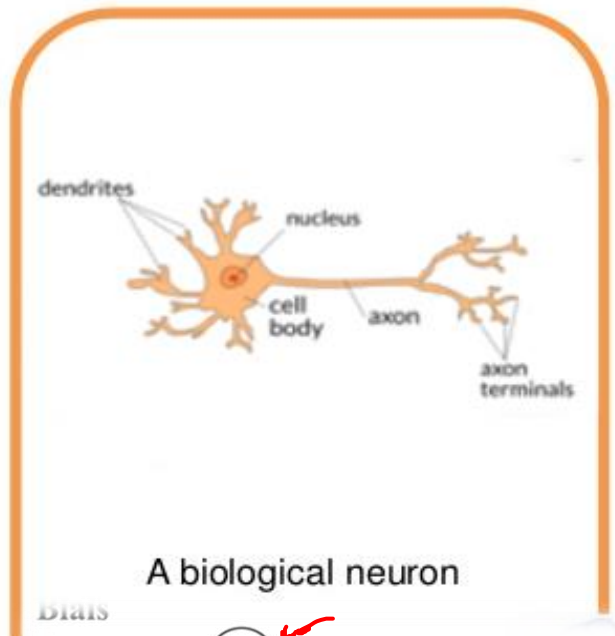


- recâblé l'entrée rétinienne d'un furet dans le cortex auditif .

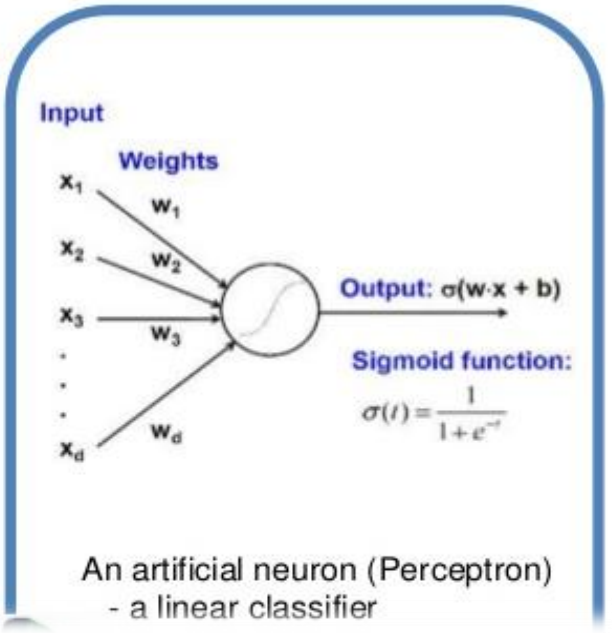
- recâblé l'entrée rétinienne d'un hamster dans le cortex somatosensoriel

le cortex auditif a appris à traiter les informations visuelles provenant des yeux du furet et a appris à voir.

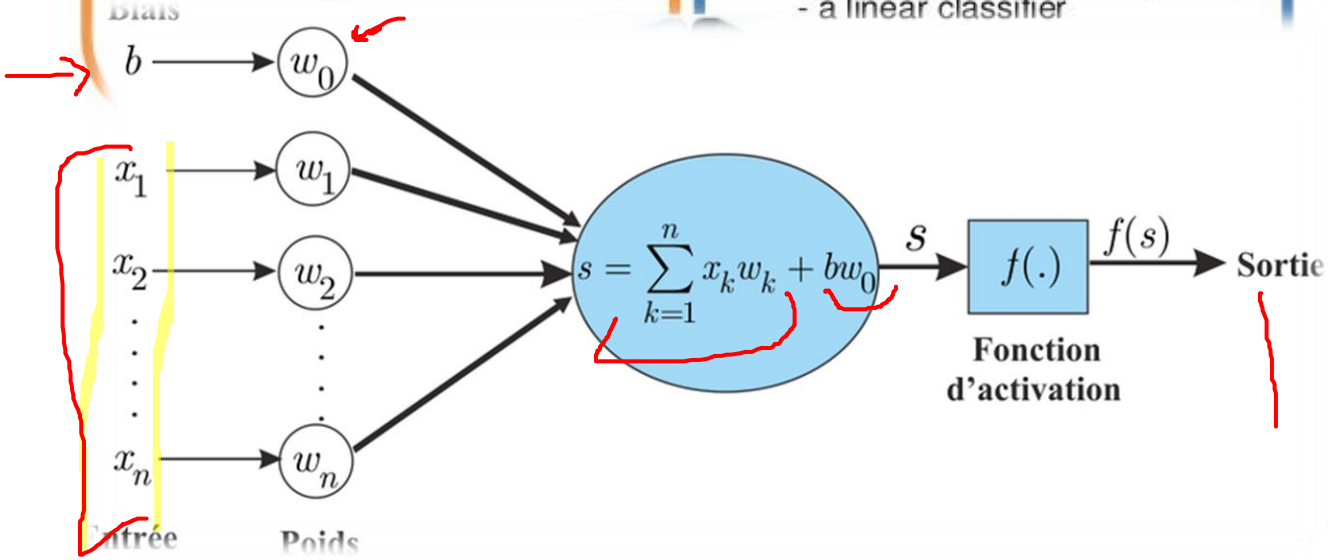
Neurone - Perceptron



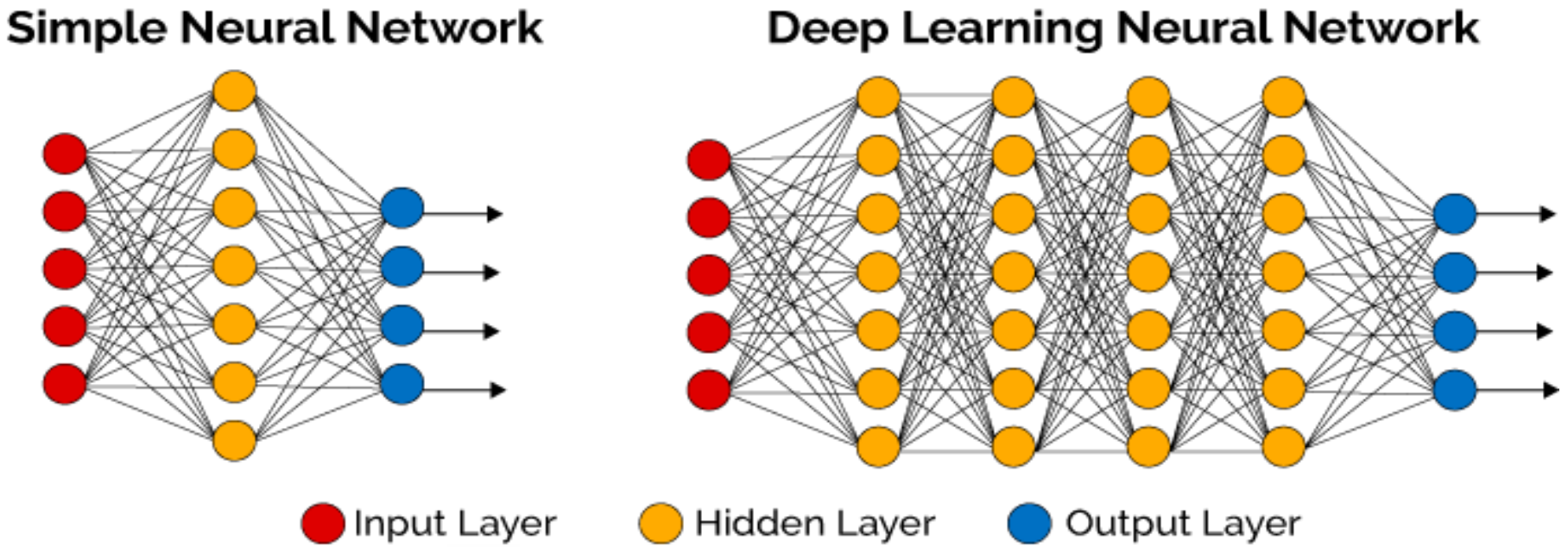
A biological neuron

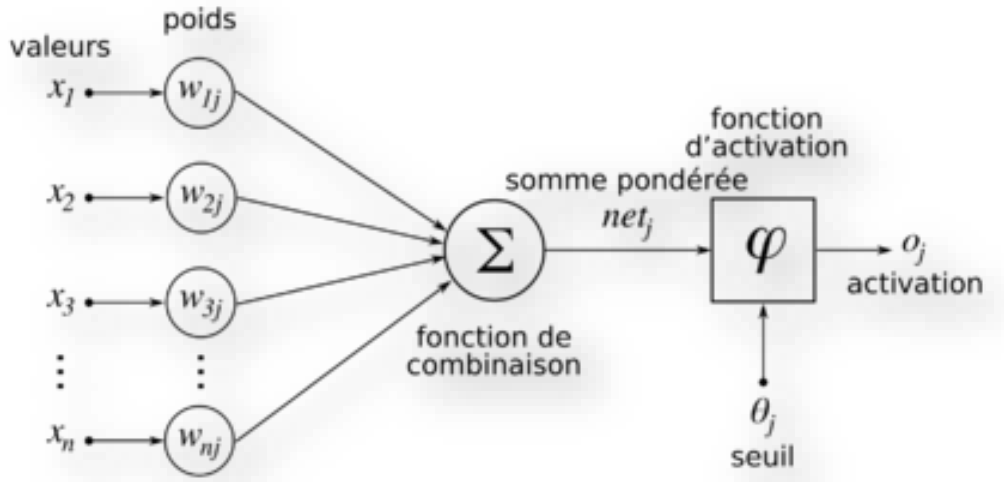


An artificial neuron (Perceptron)
- a linear classifier



ANN

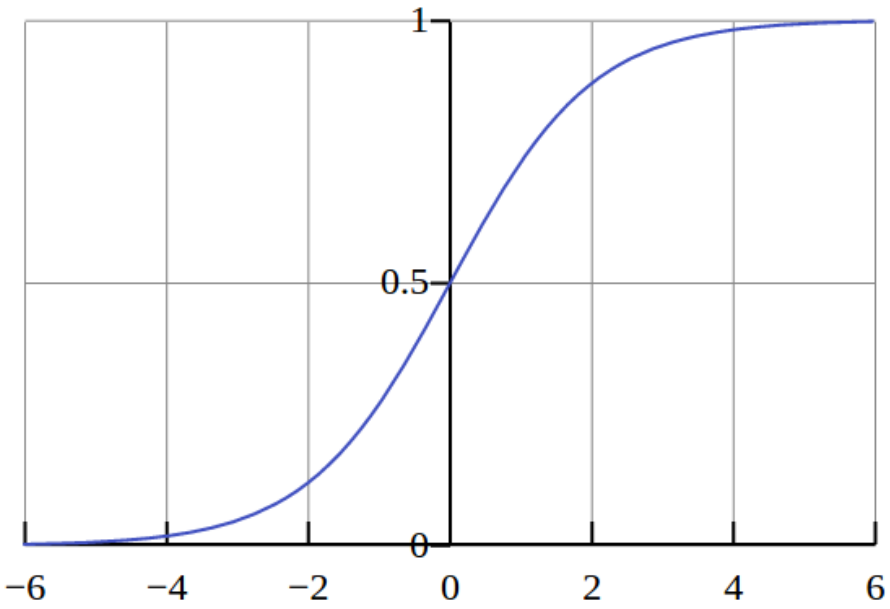

Simple Neural Network
Deep Learning Neural Network
Input Layer
Hidden Layer
Output Layer


valeurs
poids
x_1
x_2
x_3
x_n
w_{1j}
w_{2j}
w_{3j}
w_{nj}
Σ
somme pondérée
net_j
fonction de combinaison
fonction d'activation
φ
o_j
activation
θ_j
seuil

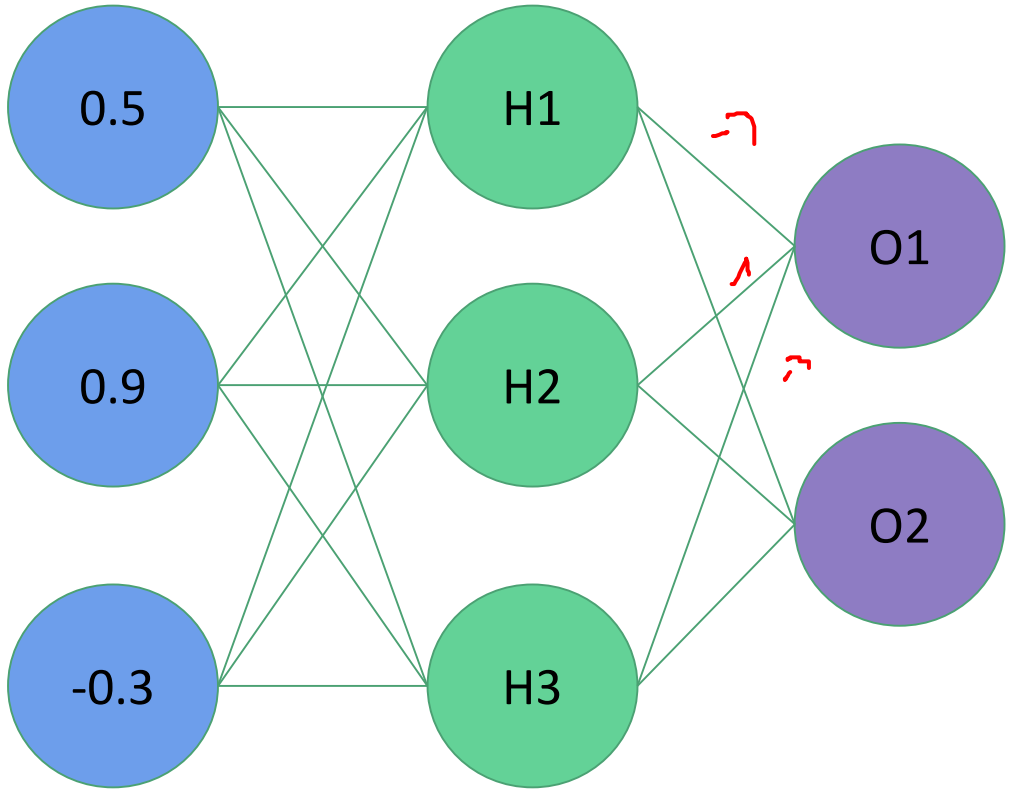
Activation Functions

- Activation Functions are applied to the inputs at each neuron
  - A common activation function is the Sigmoid

$$S(t) = \frac{1}{1 + e^{-t}}$$



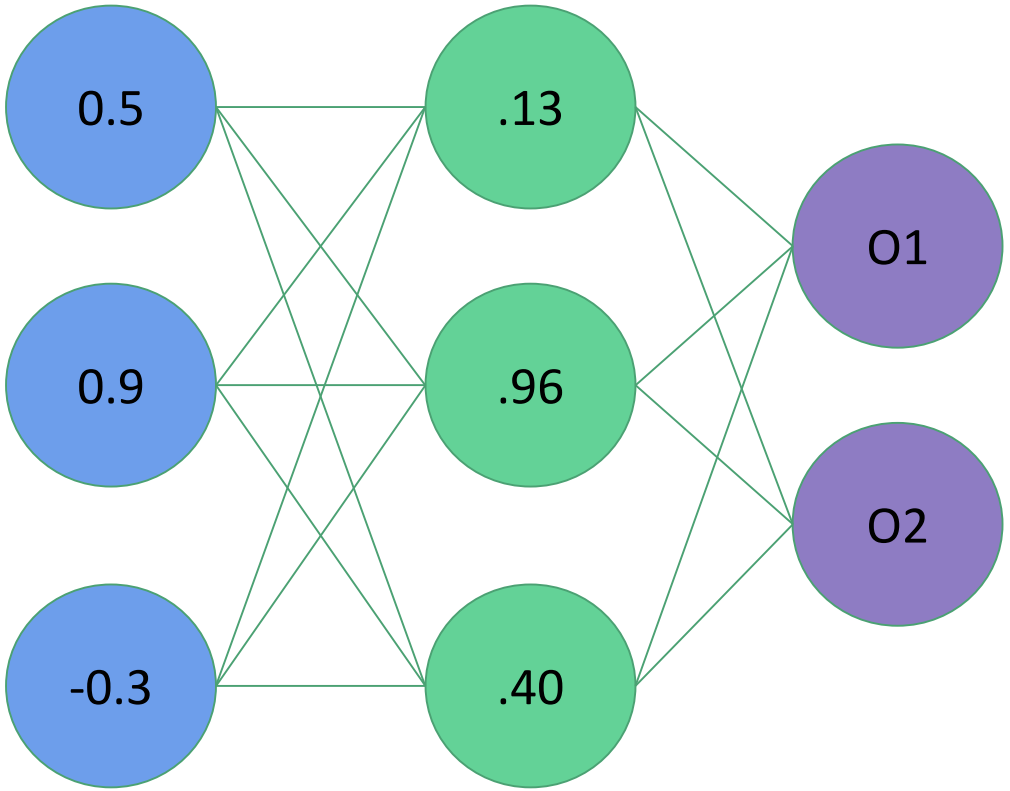
Inference



H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

Inference

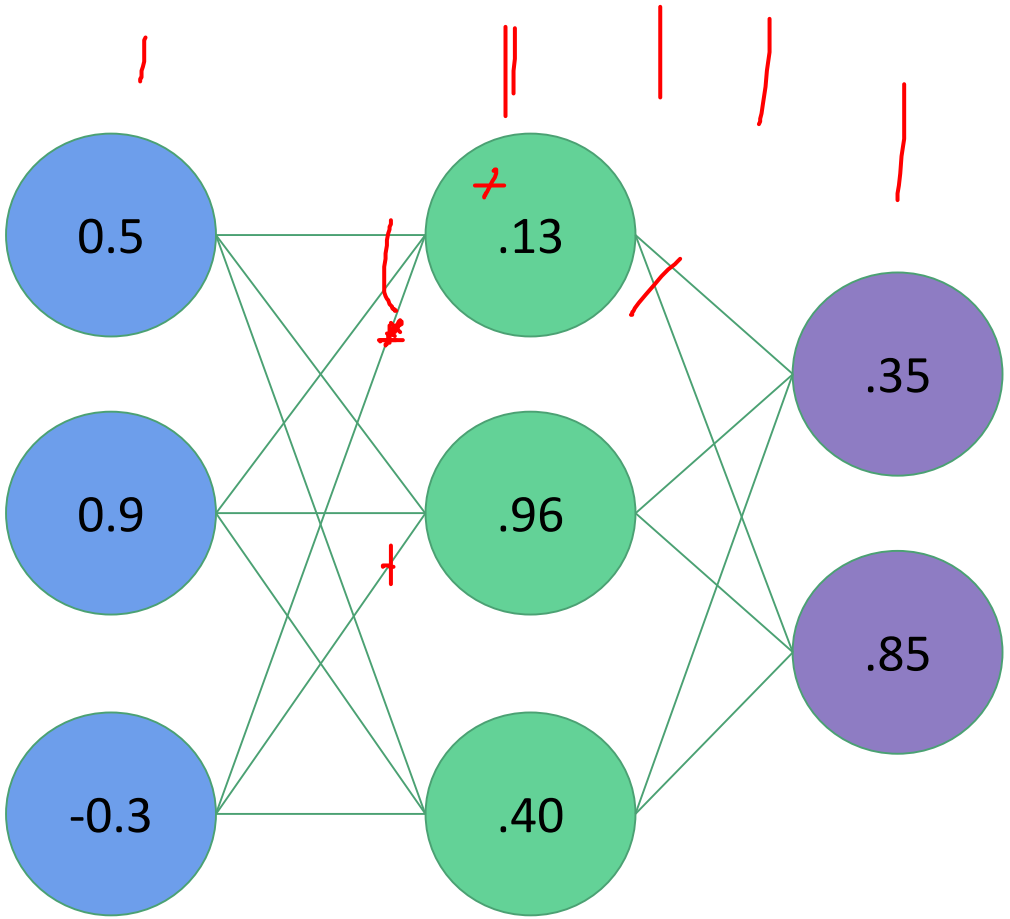


H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

H1 = S(0.5 * 1.0 + 0.9 * -2.0 + -0.3 * 2.0) = S(-1.9) = .13
H2 = S(0.5 * 2.0 + 0.9 * 1.0 + -0.3 * -4.0) = S(3.1) = .96
H3 = S(0.5 * 1.0 + 0.9 * -1.0 + -0.3 * 0.0) = S(-0.4) = .40

Inference



H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

O1 = S(.13 * -3.0 + .96 * 1.0 + .40 * -3.0) = S(-.63) = .35

O1 = S(.13 * 0.0 + .96 * 1.0 + .40 * 2.0) = S(1.76) = .85

Matrix Formulation

H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

Hidden Layer Weights

Inputs

$$S\left(\begin{bmatrix} 1.0 & -2.0 & 2.0 \\ 2.0 & 1.0 & -4.0 \\ 1.0 & -1.0 & 0.0 \end{bmatrix} * \begin{bmatrix} 0.5 \\ 0.9 \\ -0.3 \end{bmatrix}\right) = S\left(\begin{bmatrix} -1.9 & 3.1 & -0.4 \end{bmatrix}\right) =$$

Hidden Layer Outputs

$$\begin{bmatrix} .13 & .96 & 0.4 \end{bmatrix}$$

Training Neural Networks

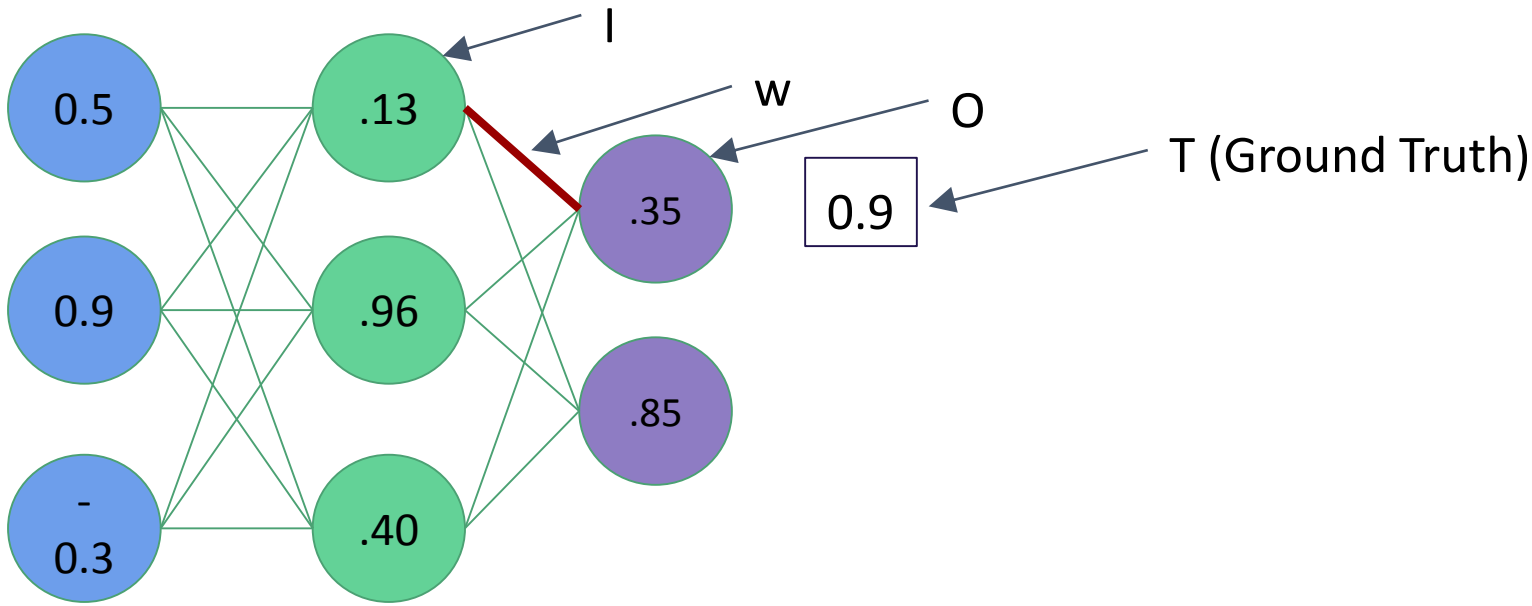
- Procédure d’apprentissage des réseaux de neurones
  - Effectuer une inférence sur l'ensemble d'apprentissage
  - Calculer l'erreur entre les prédictions et les étiquettes réelles de l'ensemble d'apprentissage
  - Déterminez la contribution de chaque neurone à l'erreur
  - Modifiez les poids du réseau neuronal pour minimiser l'erreur
- Les contributions aux erreurs sont calculées à l'aide de la rétropropagation
- La minimisation des erreurs est obtenue avec la descente de gradient

Backpropagation

Problem: Which weights should be updated and by how much?

- Insight: Use the derivative of the error with respect to weight to assign “blame”

Backpropagation Example



$$\frac{\partial E}{\partial w} = I \cdot (O - T) \cdot O \cdot (1 - O)$$

$$\frac{\partial E}{\partial w} = .13 \cdot (.35 - .9) \cdot .35 \cdot (1 - .35)$$

Gradient Descent

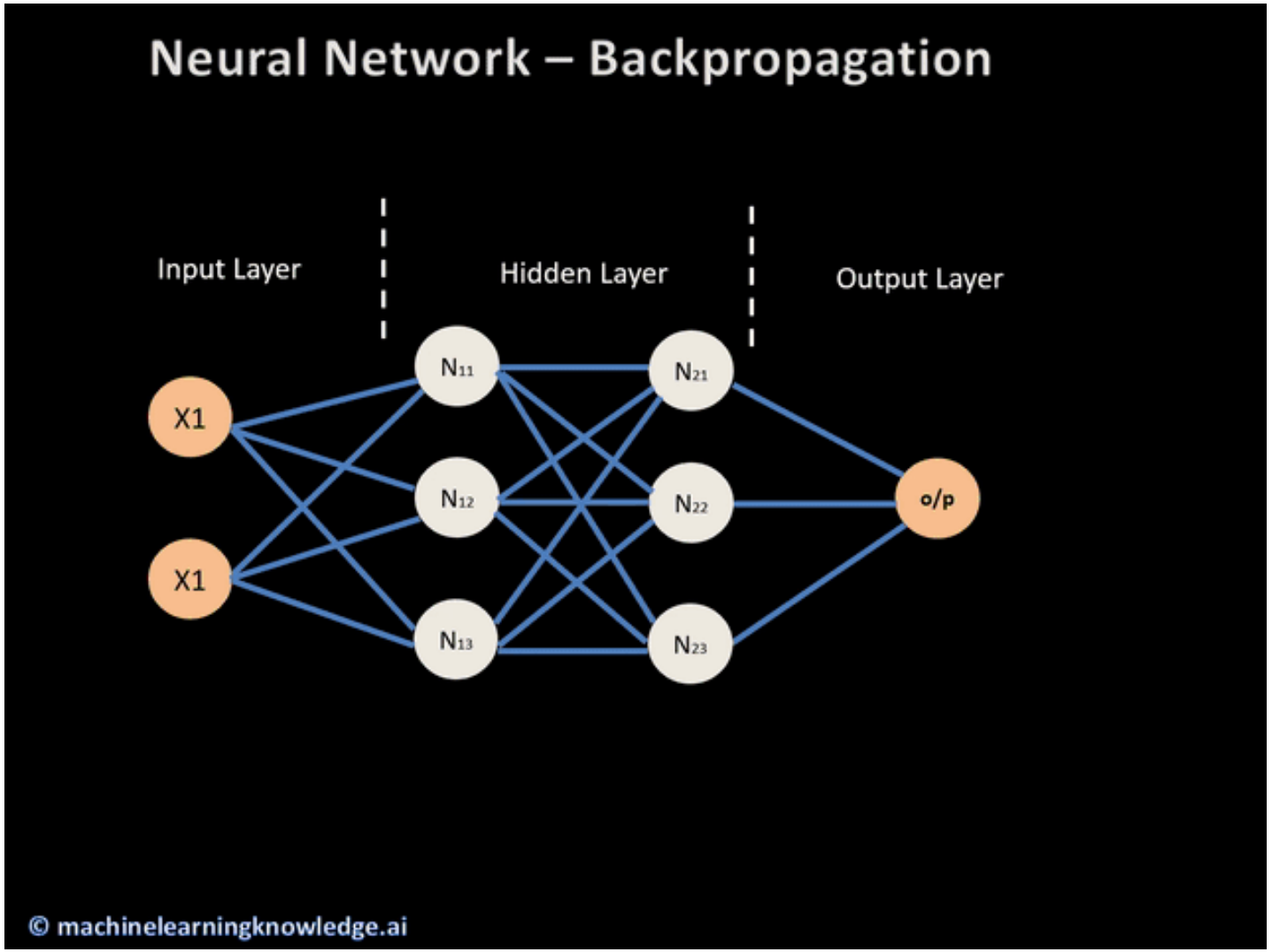
Gradient Descent minimizes the neural network's error

- At each time step the error of the network is calculated on the training data
- Then the weights are modified to reduce the error

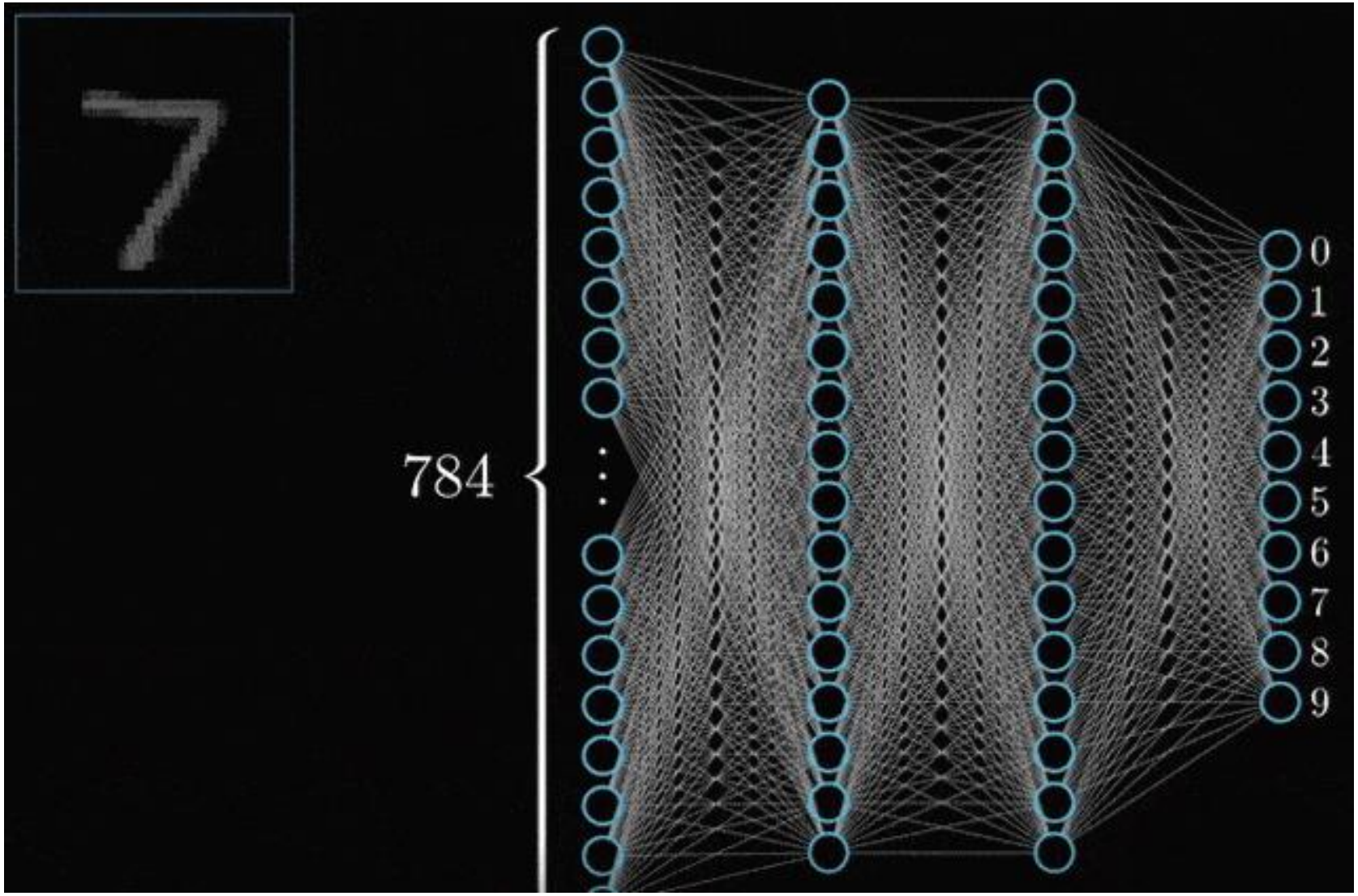
Gradient Descent terminates when

- The error is sufficiently small
- The max number of time steps has been exceeded

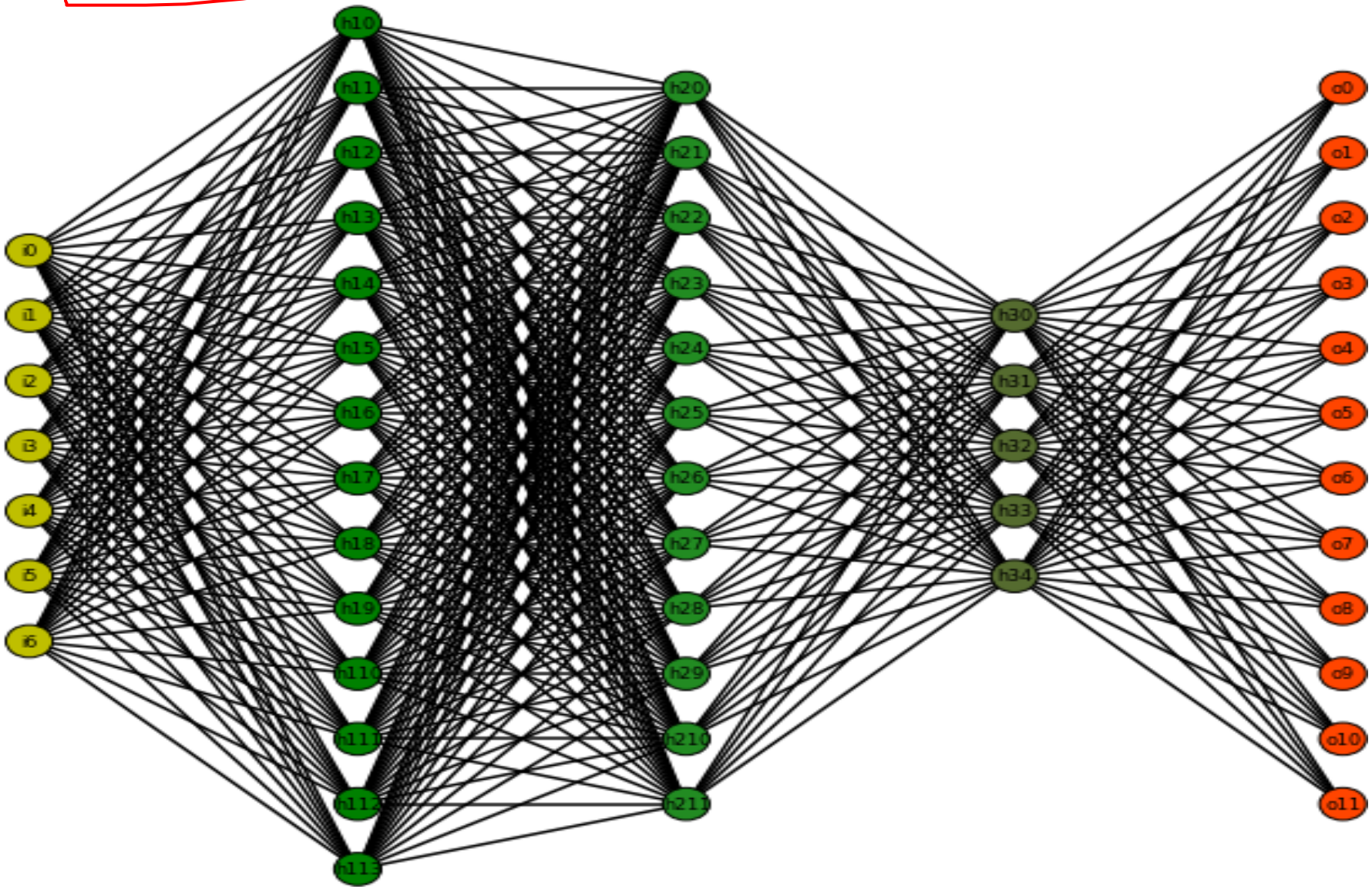
Forward and Back-Propagation



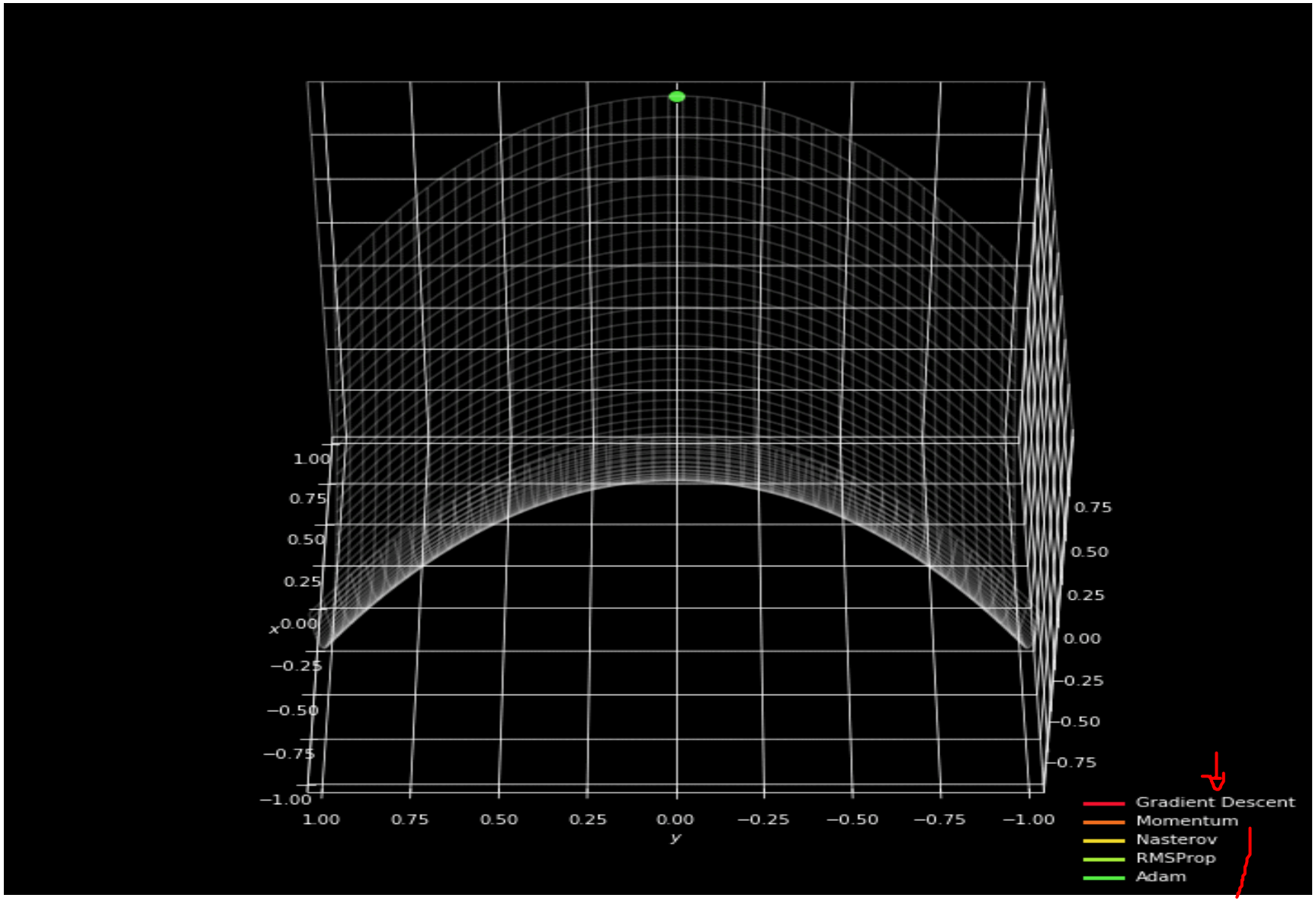
Prédiction



$+10^9$ Paramètres



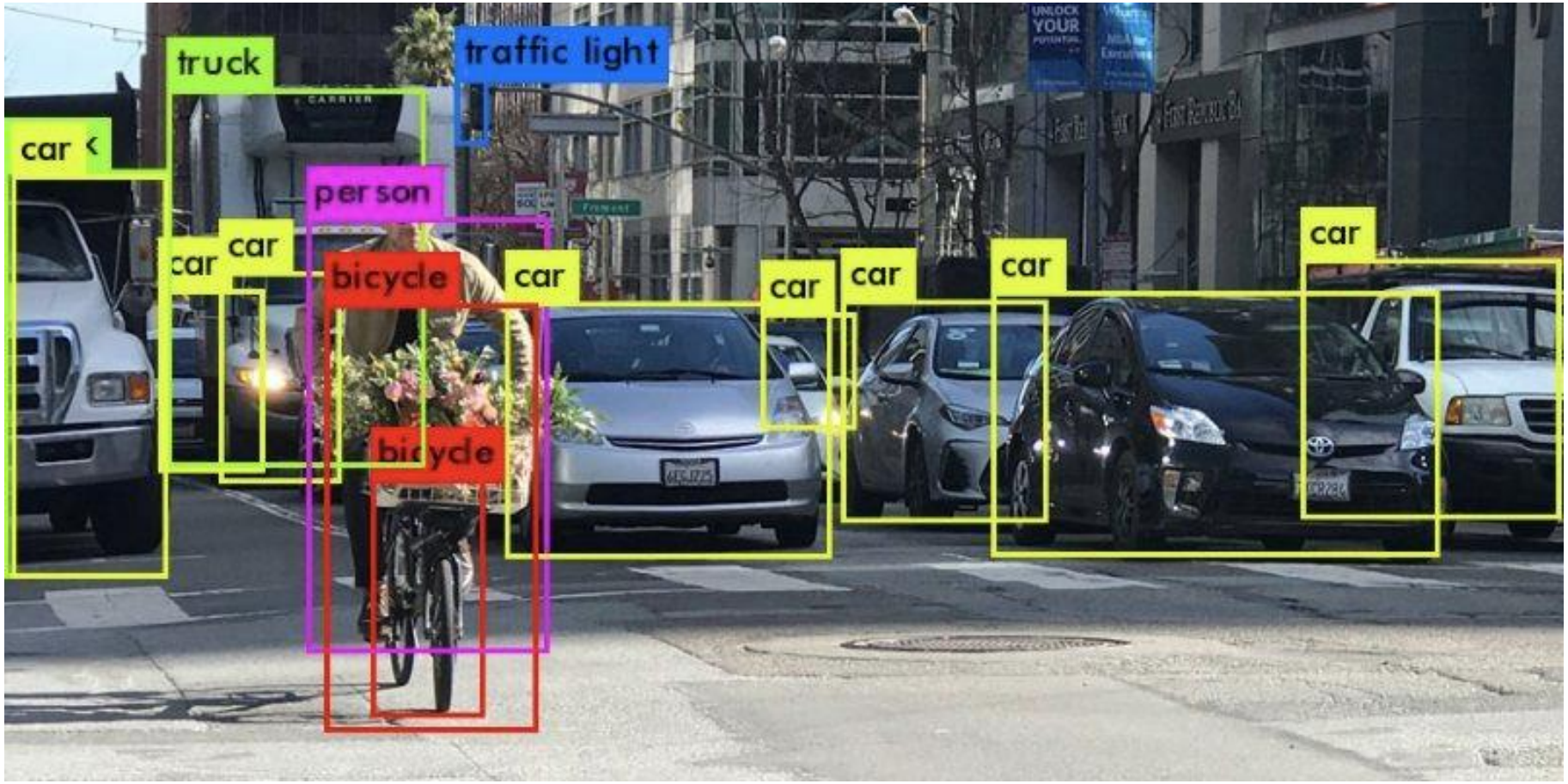
Optimisation



Deep Learning

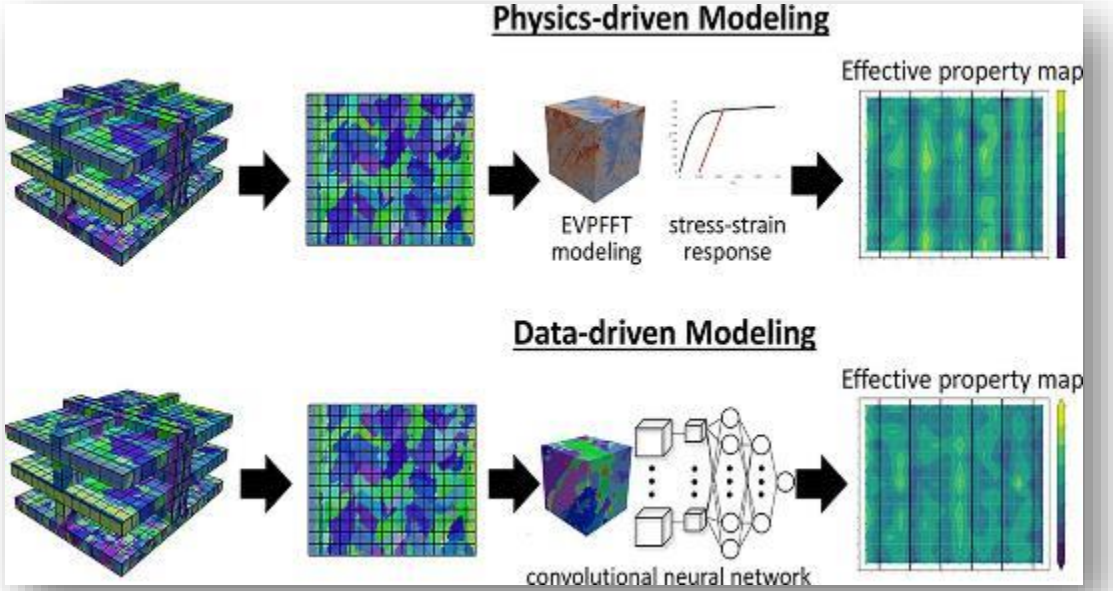


Domaine app

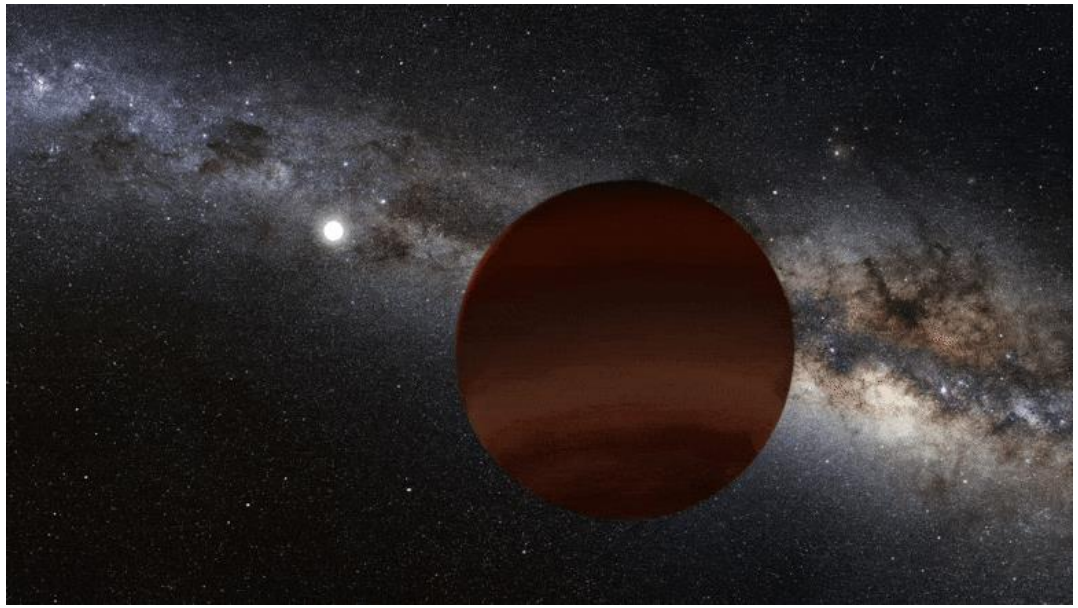

truck
traffic light
car
person
car
car
bicycle
car
car
car
car
car
bicycle

Domaine app

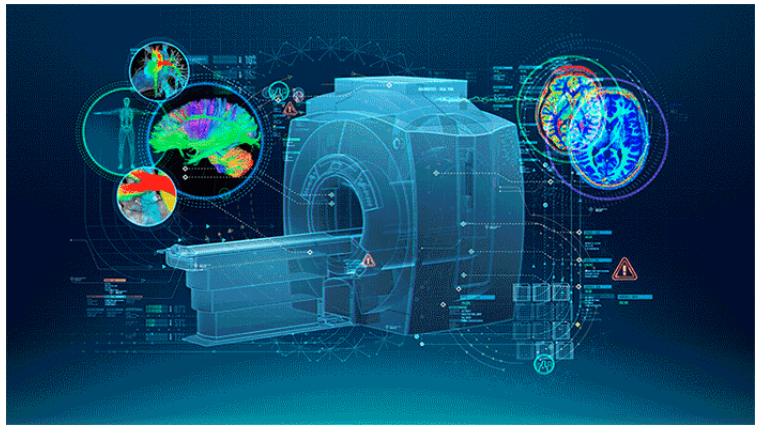
Predicting microstructure-dependent mechanical properties in additively manufactured metals with machine- and deep-learning methods

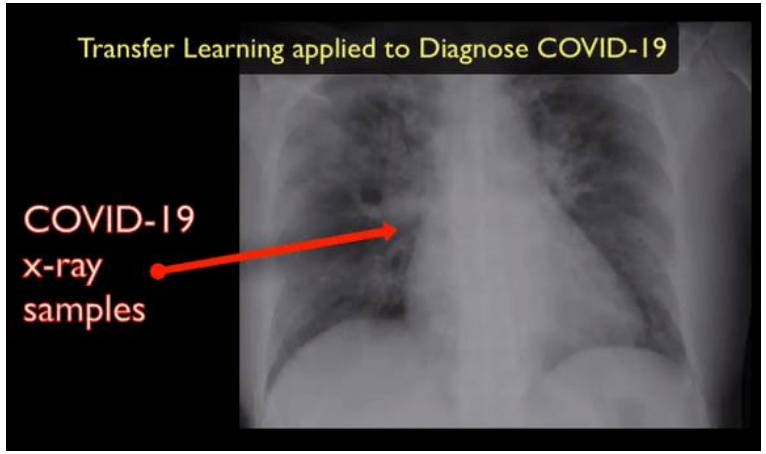






Domaine app

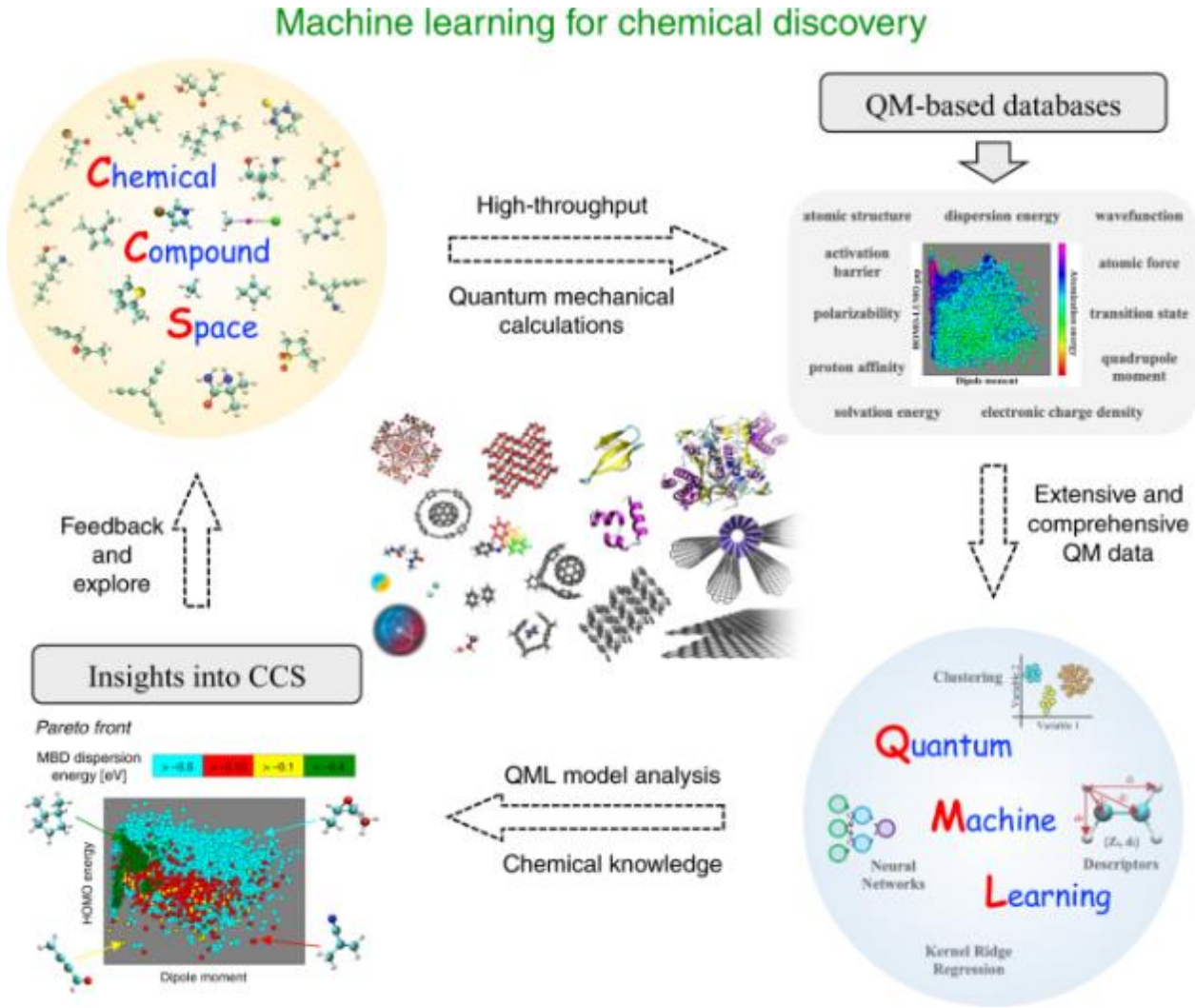



Transfer Learning applied to Diagnose COVID-19
COVID-19
x-ray
samples



Domaine app

La découverte et la formulation de nouveaux médicaments, antiviraux, antibiotiques, catalyseurs, matériaux pour batteries, et en général des produits chimiques aux propriétés adaptées, nécessitent *un changement de paradigme* pour rechercher dans des pans inexplorés du vaste espace chimique



Réf: https://www.nature.com/articles/s41467-020-17844-8

Domaine app

Springer Link

Open Access | Published: 08 September 2020

Machine learning, artificial neural networks and social research

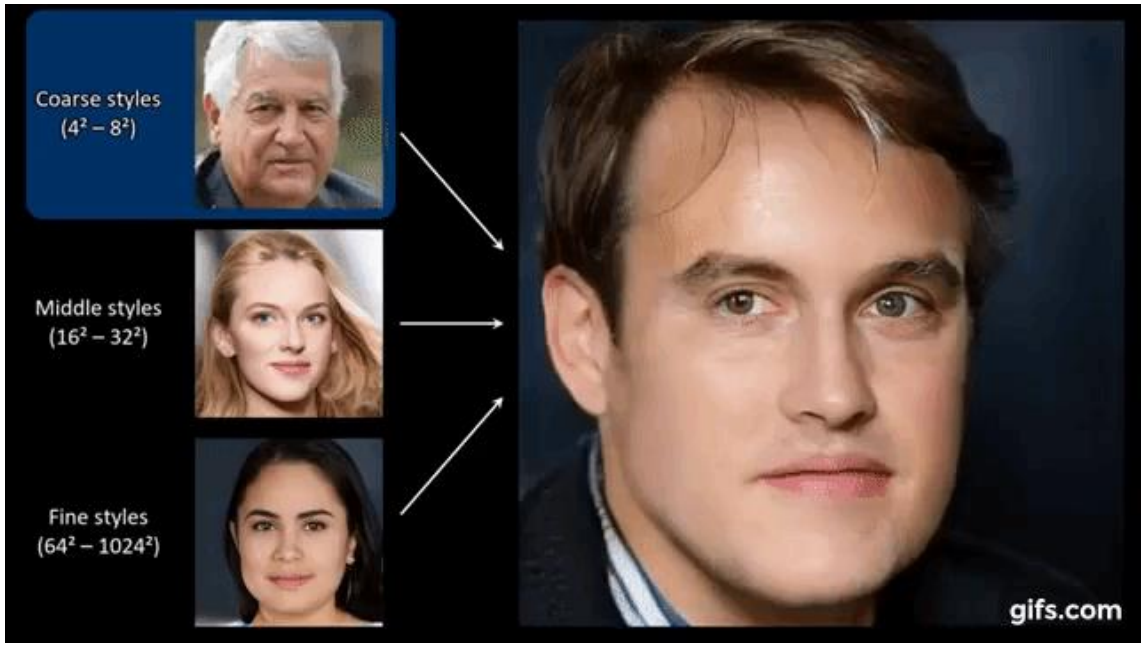
Giovanni Di Franco & Michele Santurro

Quality & Quantity (2020) | Cite this article

1125 Accesses | **2** Altmetric | Metrics

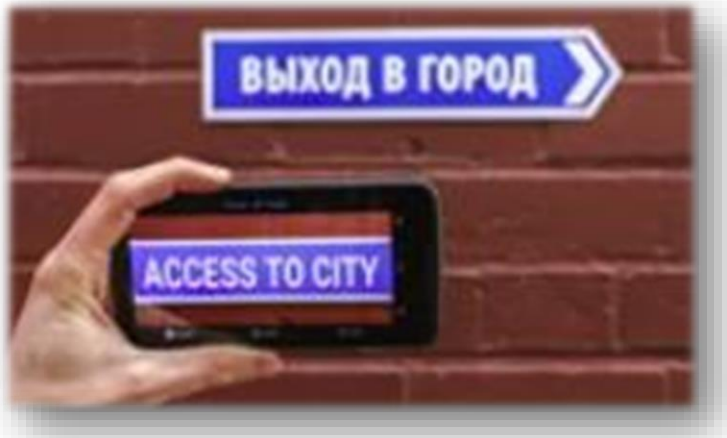
Domaine app



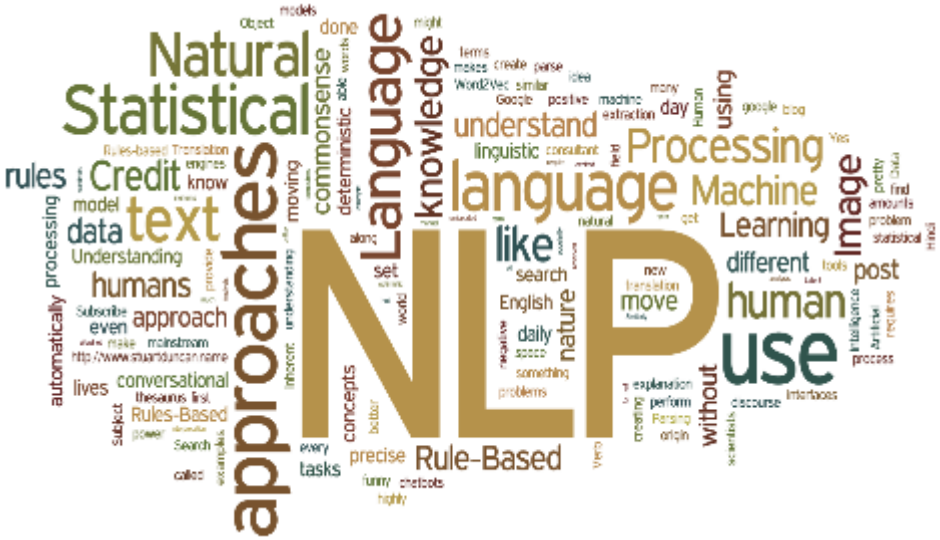

Coarse styles
(4² – 8²)
Middle styles
(16² – 32²)
Fine styles
(64² – 1024²)
gifs.com



NLP


ВЫХОД В ГОРОД
ACCESS TO CITY



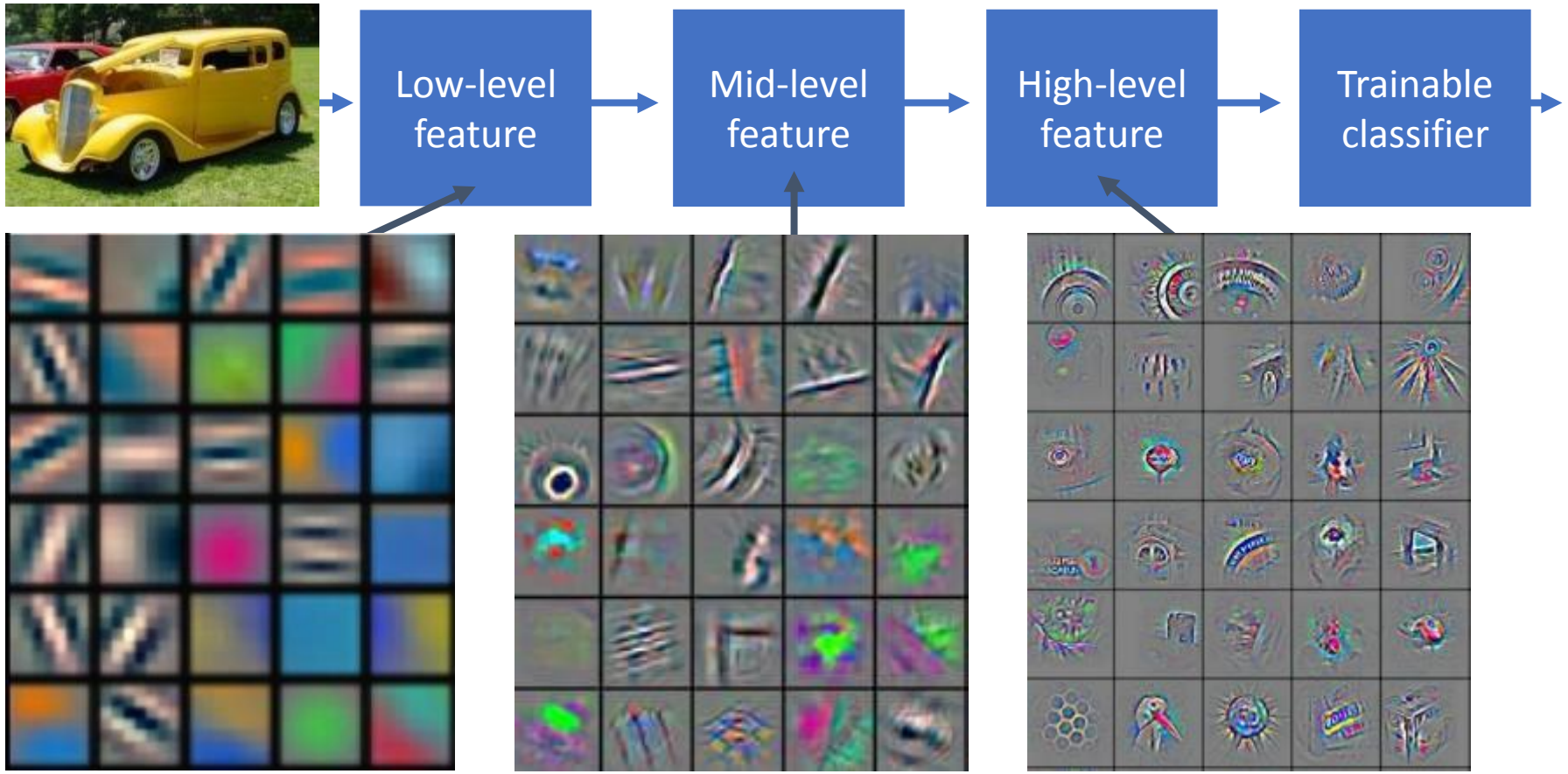

NLP
Natural
Statistical
approaches
Language
knowledge
understand
Processing
language
Machine
Learning
Credit
text
data
rules
like
human
use
Image
humans
approach
conversational
Rules-Based
Rule-Based
tasks
precise
concepts
commonsense
deterministic
moving
automatically processing
Understanding
lives
even
different
post
nature
daily
English
search
set
move
without
linguistic
Google
positive
machine
extraction
day
using
Word2Vec
similar
make
create
parse
terms
idea
done
Object
models
might
rules-based Translation
engines
know
model
pretty
find
amounts
problem
statistical
tools
Yes
got
natural
along
new
translation
explanation
perform
Parsing
origin
creating
discourse
Interfaces
process
intelligence
requires
Artificial
Subscribe
mainstream
http://www.mainduncan.name
subject
power
Search
called
every
better
Very
funny
chatbots
highly


THINK
24,000
$77,147
$21,60
Who is Bram Stoker?
$ 17,973
$1,000
$600

Deep learning = Learning representations/features

Deep learning = learning hierarchical representations

It's deep if it has more than one stage of non-linear feature transformation



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

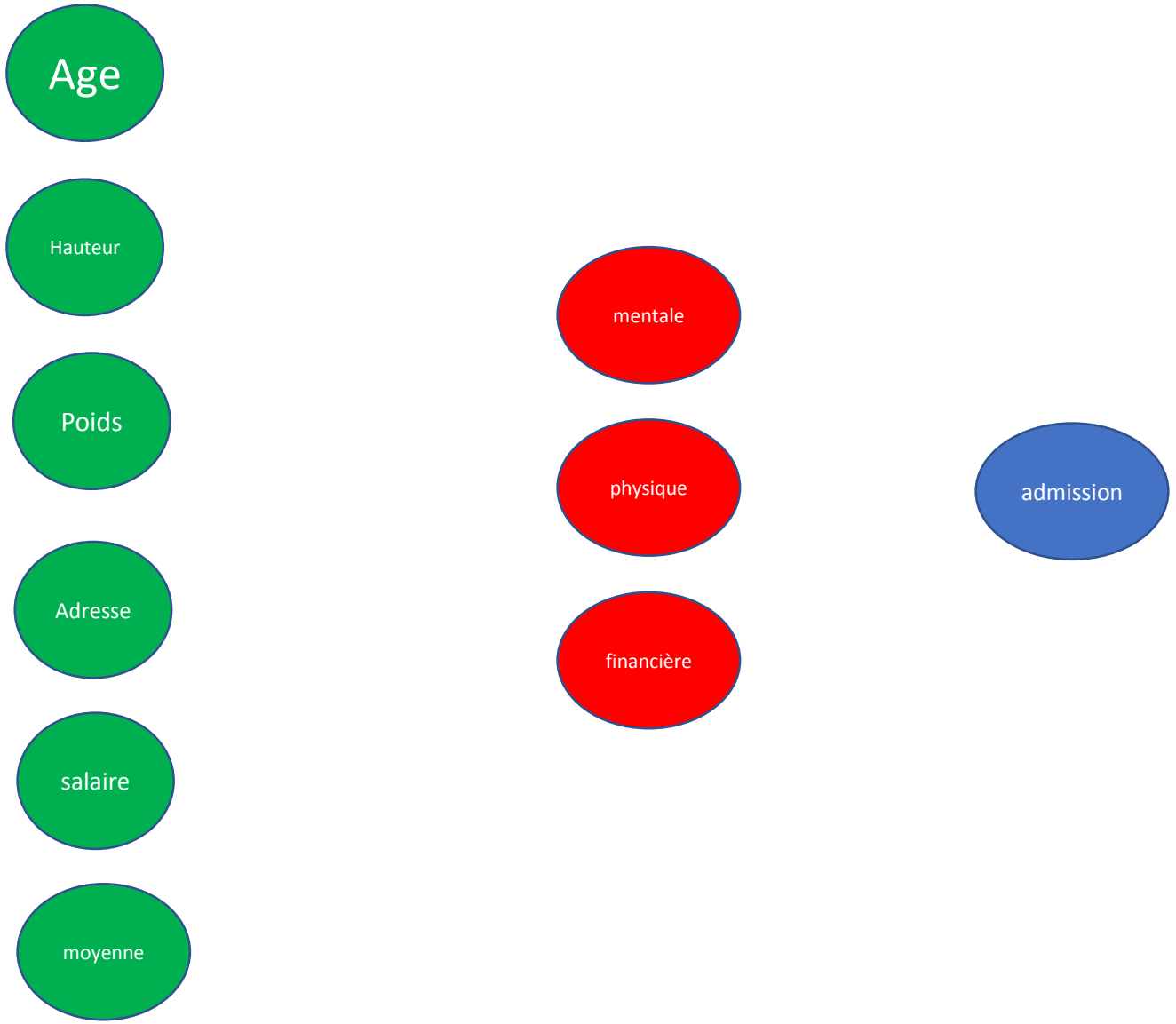
Exemple

L’admission d’un étudiant dans une école supérieure se fait selon les Compétences suivants:

- Capacité mentale
- Condition physique
- Situation financière

– Supposons qu’on a les informations suivantes:
    - L'Age, la hauteur, le poids, l’adresse, salaire des partants, moyenne du Bac.

Exemple


Age
Hauteur
Poids
Adresse
salaire
moyenne
mentale
physique
financière
admission

Exemple

Prédiction des prix des appartements

– Supposons qu'on a les caractéristiques (features) suivantes:

Surface

Nombre de chambre

Adresse

Nombre de personnes

proximité de commodités

Qualité des écoles

Prix

Deep learning =
Learning representations/features

– The traditional model of pattern recognition (since the late 50's)
  – Fixed/engineered features (or fixed kernel) + trainable classifier

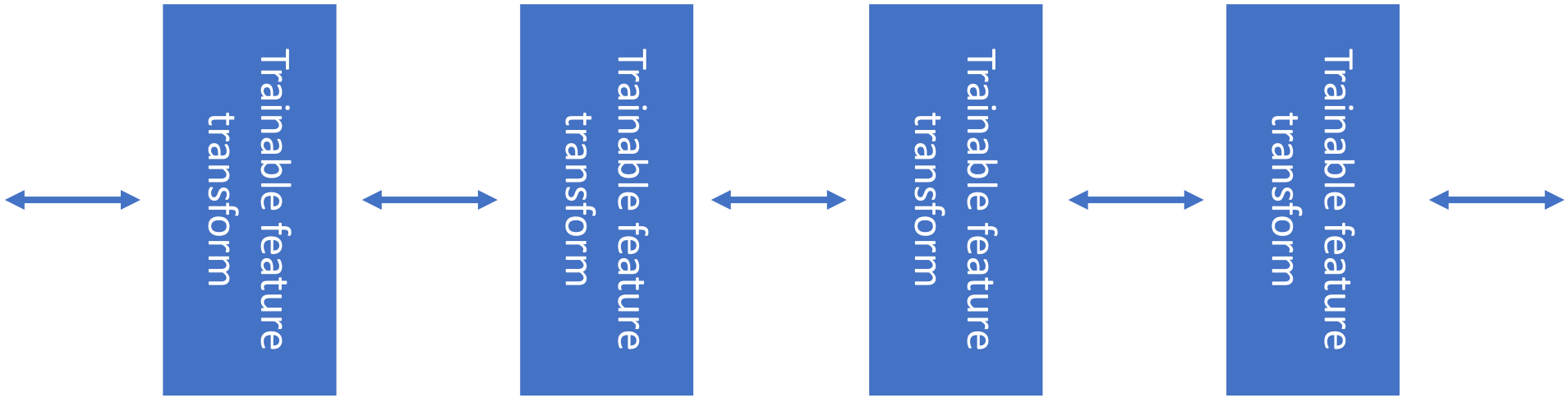


– End-to-end learning / Feature learning / Deep learning
  – Trainable features (or kernel) + trainable classifier



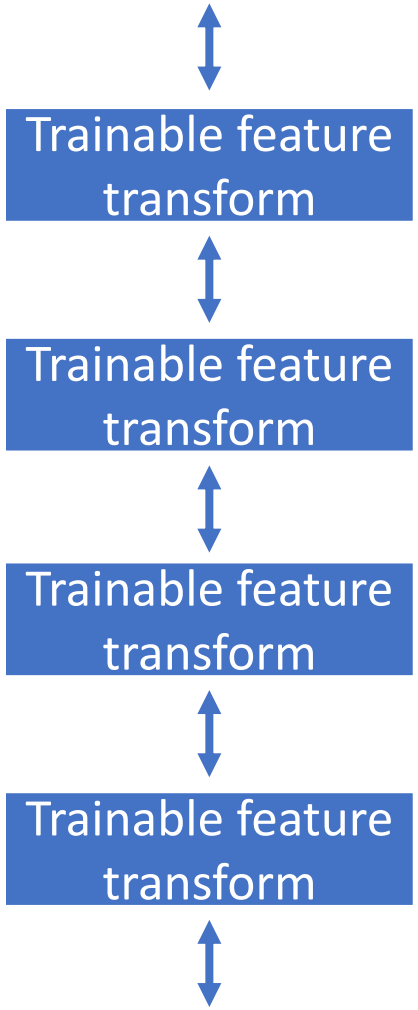
Trainable feature hierarchy

- Hiérarchie des représentations avec un niveau d'abstraction croissant. Chaque étape est une sorte de transformation de fonctionnalité entraînable
  - Reconnaissance d'image
    - Pixel → bord → texton → motif → partie → objet
  - Texte
    - Caractère → mot → groupe de mots → clause → phrase → histoire
  - Discours/Speech
    - Échantillon → bande spectrale → son →… → téléphone → phonème → mot



Learning representations: a challenge for ML, CV, AI, neuroscience, cognitive science...

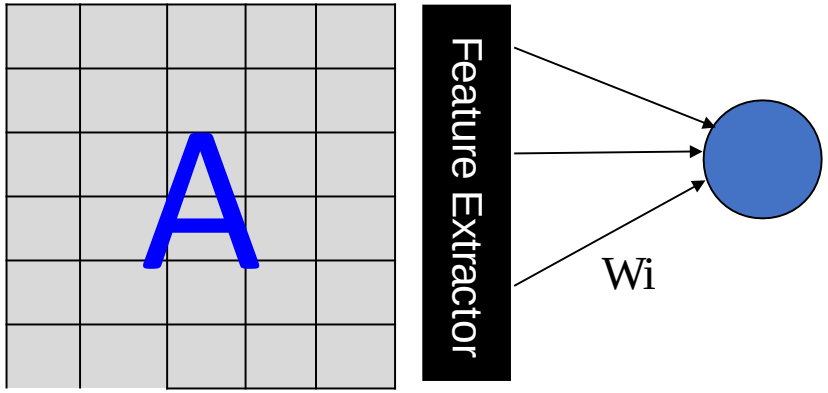
- Comment apprenons-nous les représentations du monde perceptif?
  - Comment un système perceptif peut-il se construire en regardant le monde?
  - Combien de structure préalable est nécessaire
- ML / AI: comment apprendre les fonctionnalités ou les hiérarchies de fonctionnalités?
  - Quel est le principe fondamental? Quel est l'algorithme d'apprentissage? Quelle est l'architecture?
- Neuroscience: comment le cortex apprend-il la perception?
- Le cortex «exécute-t-il» un seul algorithme d'apprentissage général? (ou un petit nombre d'entre eux)
- CogSci: comment l'esprit apprend-il des concepts abstraits en plus de concepts moins abstraits?
- Le Deep Learning aborde le problème de l'apprentissage des représentations hiérarchiques avec un seul algorithme
  - Ou peut-être avec quelques algorithmes



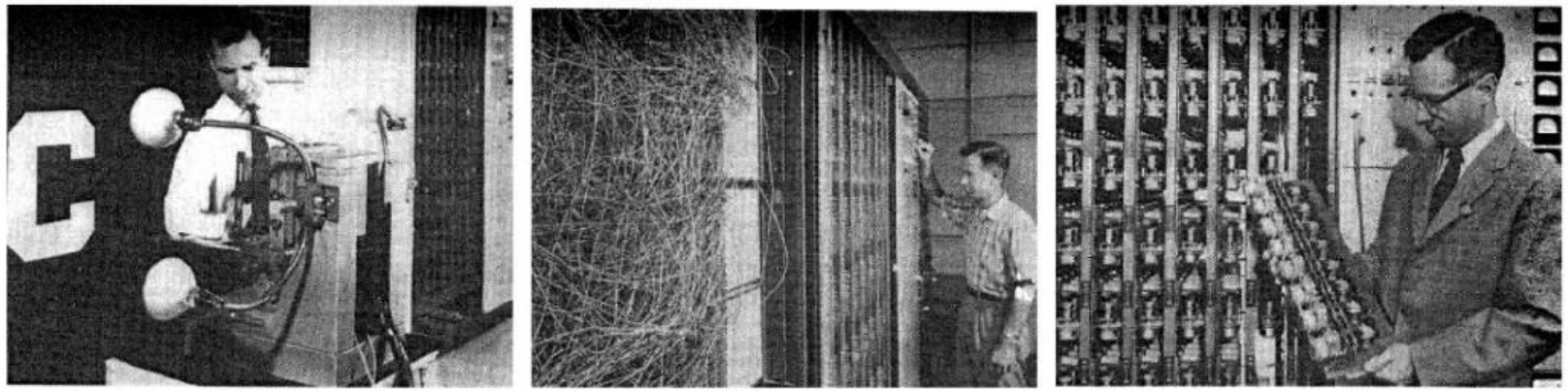
This basic model has not evolved much since the 50's

Old Paradigm

- The first learning machine: the Perceptron
  - Built at Cornell in 1960
- The Perceptron was a linear classifier on top of a simple feature extractor
- The vast majority of practical applications of ML today use glorified linear classifiers or glorified template matching.
- Designing a feature extractor requires considerable efforts by experts.



$$y = sign\left(\sum_{i=1}^{N} W_i F_i(X) + b\right)$$



Deep Learning

Machines linéaires et leurs limites

Classifieurs linéaires

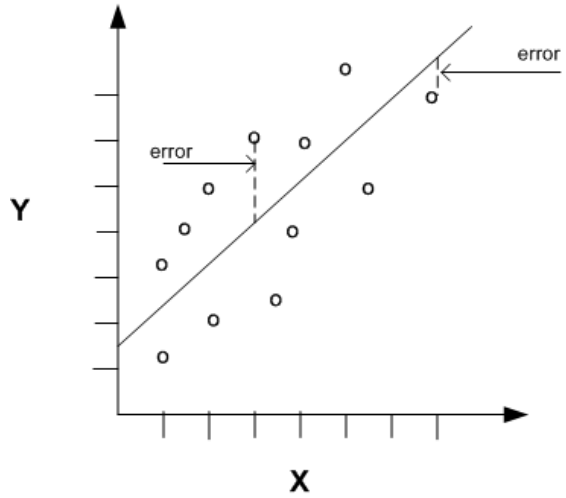
- En apprentissage automatique, les **classifieurs linéaires** sont une famille d'algorithmes de classement statistique. Le rôle d'un classifieur est de classer dans des groupes (des classes) les échantillons qui ont des propriétés similaires, mesurées sur des observations.
- Un classifieur linéaire est un type particulier de classifieur, qui calcule la décision par combinaison linéaire des échantillons.

Pour un vecteur d'observation x, à valeur dans $\mathbb{R}^N$, la sortie du classifieur est donnée par :

$$g(x) = f(w^T x + w_0) = f\left(\sum_{i=1}^{N} w_j x_j + w_0\right),$$

Linear machines: regression with mean square

Linear regression, mean square loss:



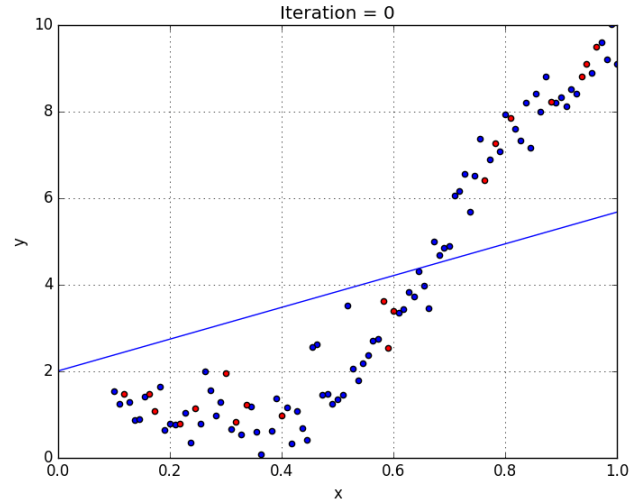
– Decision rule: $y = W'X$

– Loss function: $L(W, y^i, X^i) = \frac{1}{2}(y^i - W'X^i)^2$

– Gradient of loss: $\frac{\partial L(W,y^i,X^i)}{\partial W}' = -(y^i - W(t)'X^i)X^i$

– Update rule: $W(t+1) = W(t) + \eta(t)(y^i - W(t)'X^i)X^i$

– Direct solution: solve linear system $[\sum_{i=1}^{P} X^i X^{i\prime}]W = \sum_{i=1}^{P} y^i X^i$



Linear machines

Perception

– Decision rule: (F is threshold function) $y = F(W'X)$

– Loss function: $L(W, y^i, X^i) = (F(W'X^i) - y^i)W'X^i$

– Gradient of loss: $\frac{\partial L(W,y^i,X^i)}{\partial W}' = -(y^i - F(W(t)'X^i))X^i$

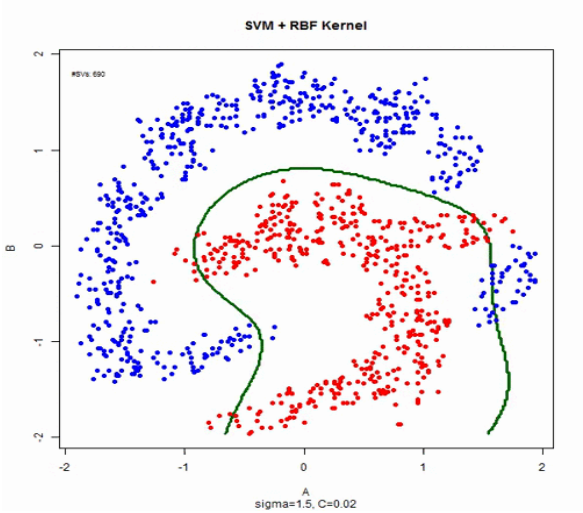
– Update rule: $W(t+1) = W(t) + \eta(t)(y^i - F(W(t)'X^i))X^i$

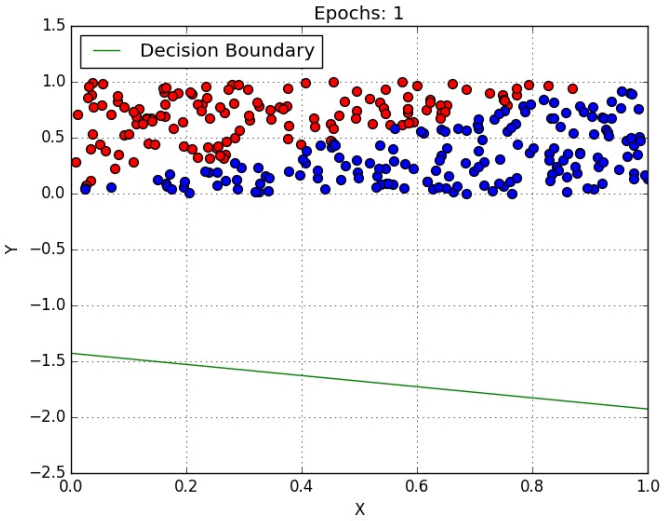
– Direct solution: fine W such that $-y^i F(W'X^i) < 0 \quad \forall i$

Linear machines: logistic regression

Logistic regression, negative log-likelihood loss function:

- Decision rule: $y = F(W'X)$, with $F(a) = \tanh(a) = \frac{1-\exp(a)}{1+\exp(a)}$ (sigmoid function).
- Loss function: $L(W, y^i, X^i) = 2\log(1 + \exp(-y^i W' X^i))$
- Gradient of loss: $\frac{\partial L(W,y^i,X^i)}{\partial W}' = -\left(Y^i - F(W'X)\right)X^i$
- Update rule: $W(t+1) = W(t) + \eta(t)(y^i - F(W(t)'X^i))X^i$





General gradient-based supervised learning machine

Neural nets, and many other models:

- Decision rule: $y = F(W,X)$, where F is some function, and W some parameter vector.
- Loss function: $L(W, y^i, X^i) = D(y^i, F(W, X^i))$, where $D(y, f)$ measures the "discrepancy" between A and B.
- Gradient loss:

$$\frac{\partial L(W,y^i,X^i)}{\partial W}' = \frac{\partial D(y^i,f}{\partial f}\frac{\partial F(W,X^i)}{\partial W}$$

- Update rule:

$$W(t+1) = W(t) - \eta(t)\frac{\partial D(y^i,f}{\partial f}\frac{\partial F(W,X^i)}{\partial W}$$

Three questions:

- What architecture $F(W,X)$.
- What loss function $L(W, y^i, X^i)$.
- What optimization method.

Learning Feature Hierarchy

- Learn hierarchy
- All the way from pixels → classifier
- One layer extracts features from output of previous layer



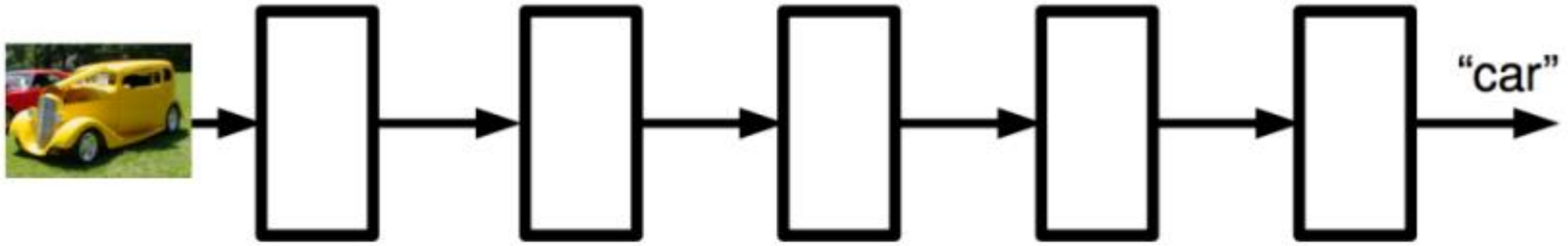
- Train all layers jointly

what is Deep Learning?

A few different ideas:

- (Hierarchical) Compositionality
  - Cascade of non-linear transformations
  - Multiple layers of representations
- End-to-End Learning
  - Learning (goal-driven) representations
  - Learning to feature extraction
- Distributed Representations
  - No single neuron “encodes” everything
  - Groups of neurons work together

Deep Learning

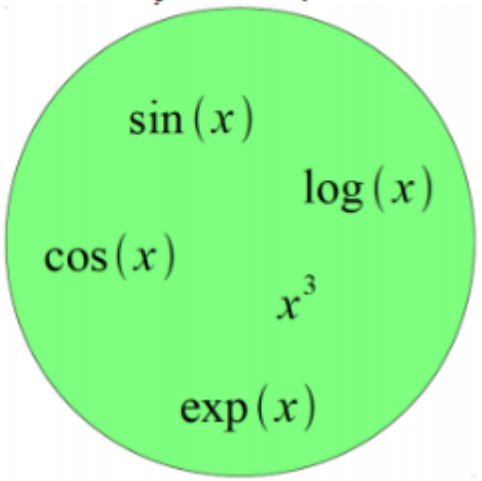


What is Deep Learning

- Cascade of non-linear transformations
- End to end learning
- General framework (any hierarchical model is deep)

Building a Complicated Function

Given a library of simple functions

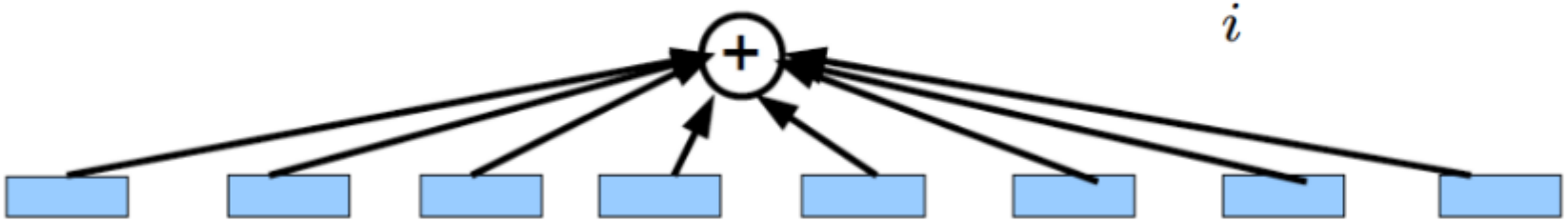


Compose into a complicated function

Idea 1: Linear Combinations

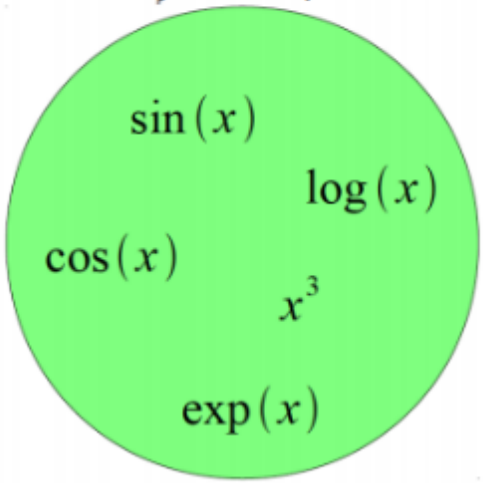
- Boosting
- Kernels
- …

$$f(x) = \sum_i \alpha_i g_i(x)$$



Building a Complicated Function

Given a library of simple functions

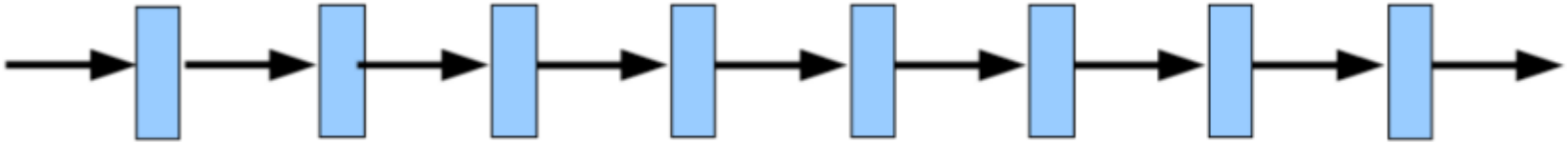


Compose into a complicated function

Idea 2: Compositions

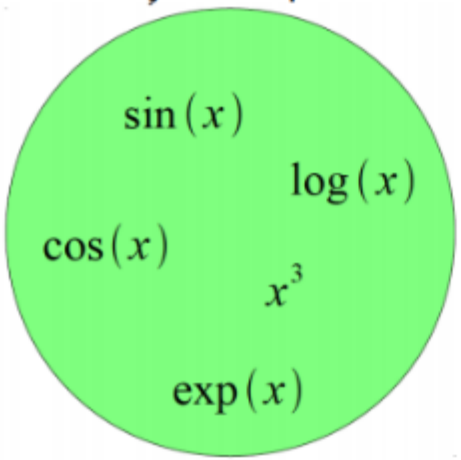
- Deep Learning
- Grammar models
- Scattering transforms…

$$f(x) = g_1(g_2(\ldots(g_n(x)\ldots))$$



Building a Complicated Function

Given a library of simple functions

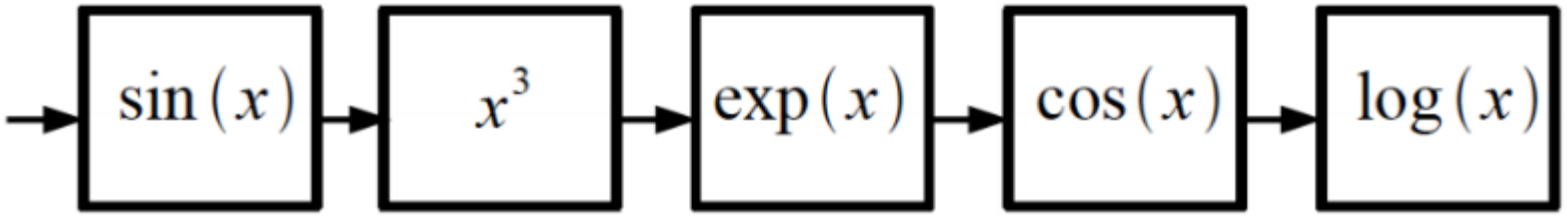


Compose into a complicated function

Idea 2: Compositions

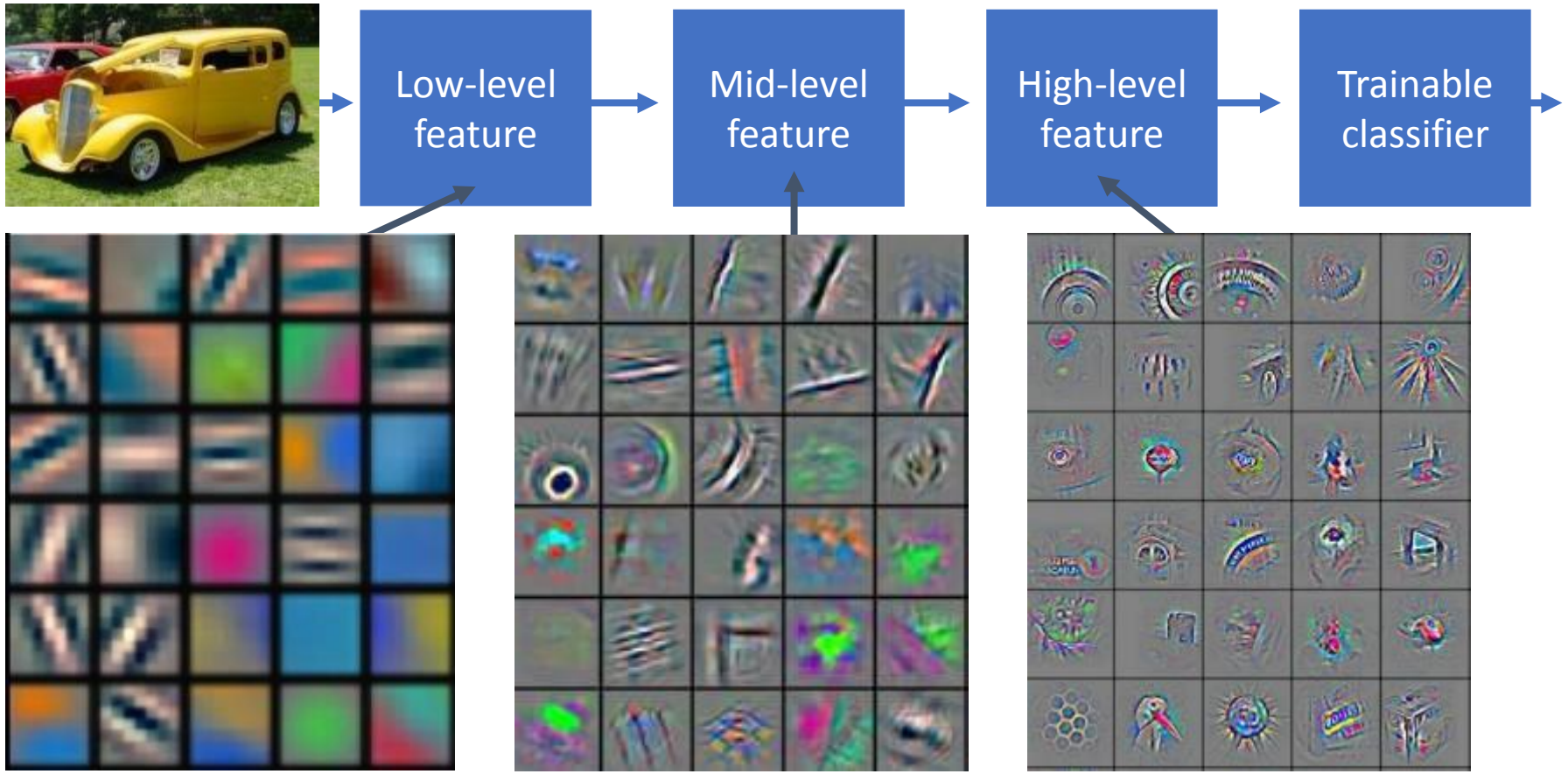
- Deep Learning
- Grammar models
- Scattering transforms…

$$f(x) = \log(\cos(\exp(\sin^3(x))))$$



Deep learning = learning hierarchical representations

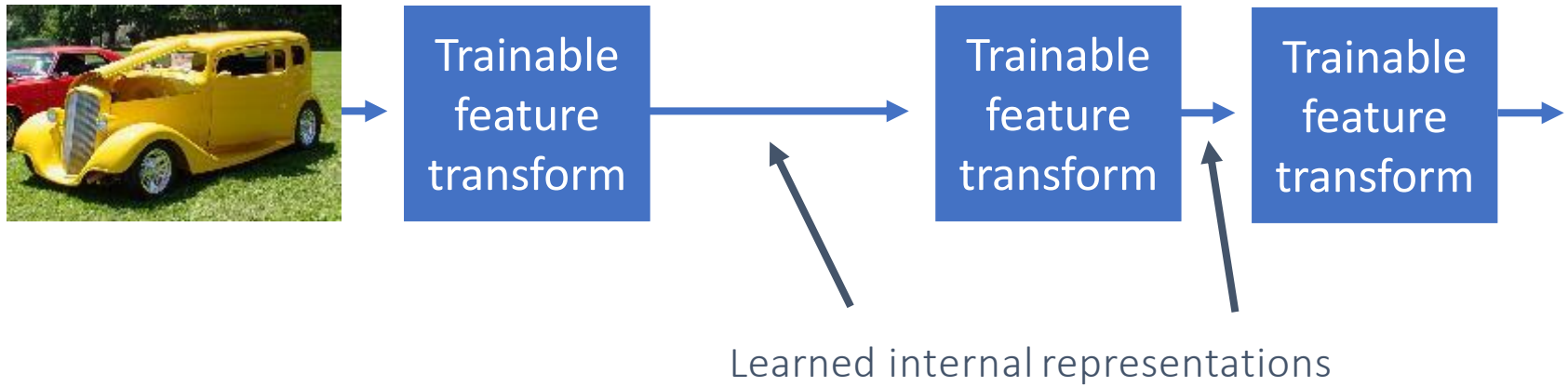
It's deep if it has more than one stage of non-linear feature transformation



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Trainable feature hierarchies: end-to-end learning

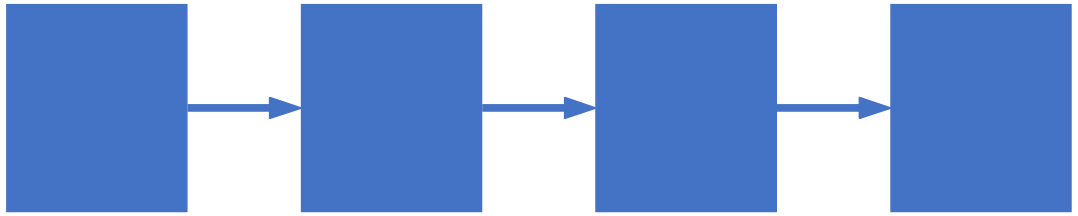
– A hierarchy of trainable feature transforms
  – Each module transforms its input representation into a higher-level one.
  – High-level features are more global and more invariant
  – Low-level features are shared among categories



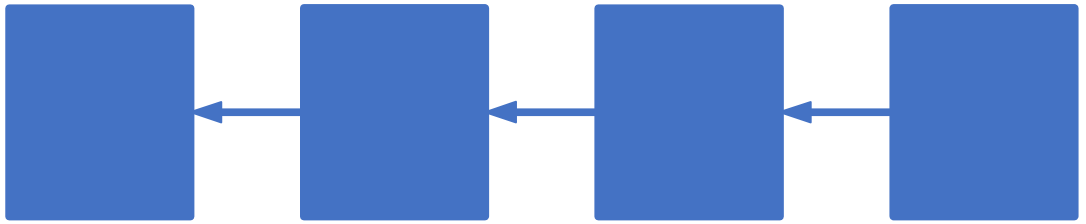
– How can we make all the modules trainable and get them to learn appropriate representations?

Three types of deep architectures

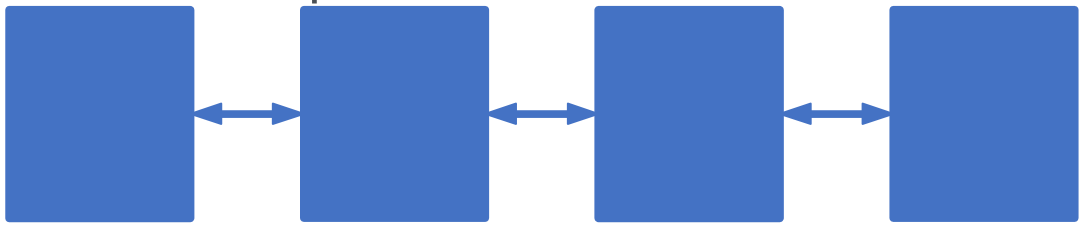
– Feed-forward: multilayer neural nets, convolutional nets



– Feed-back: stacked sparse coding, deconvolutional nets



– Bi-directional: Deep Boltzmann Machines, stacked auto-encoders



Do we really need deep architectures?

- Theoretician's dilemma: “We can approximate any function as close as we want with shallow architecture. Why would we need deep ones?”

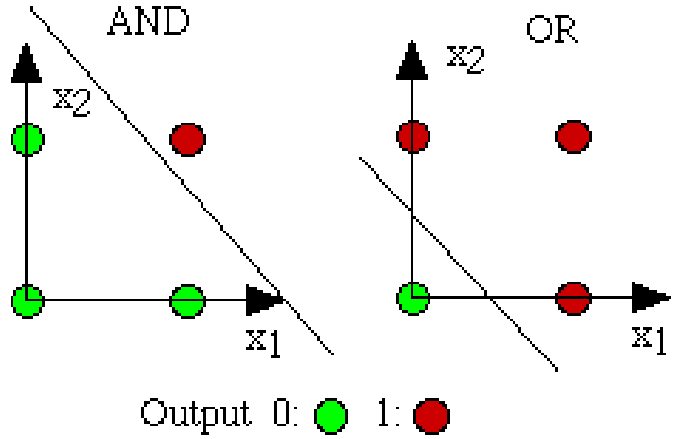
$$y = \sum_{i=1}^{P} \alpha_i K(X, X^i) \qquad y = F(W^1.F(W^0.X))$$

  - kernel machines (and 2-layer neural nets) are “universal”.
- Deep learning machines

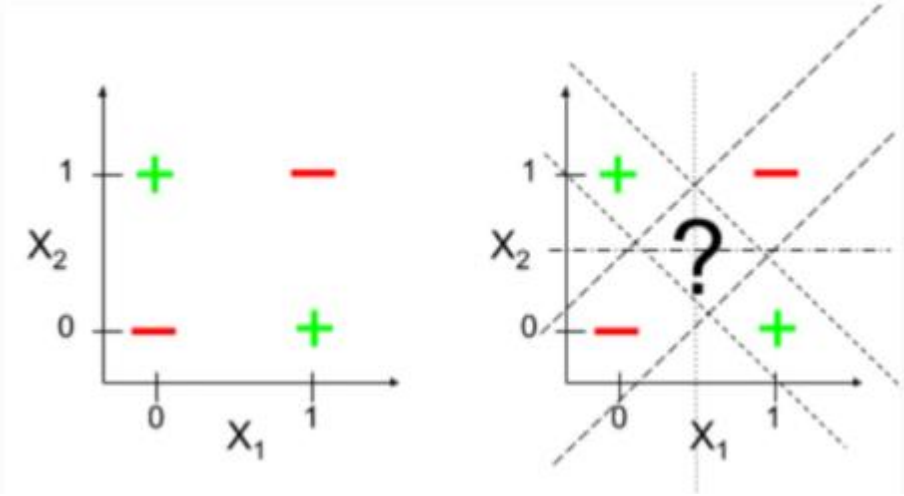
$$y = F(W^K.F(W^{K-1}.F(....F(W^0.X)...)))$$

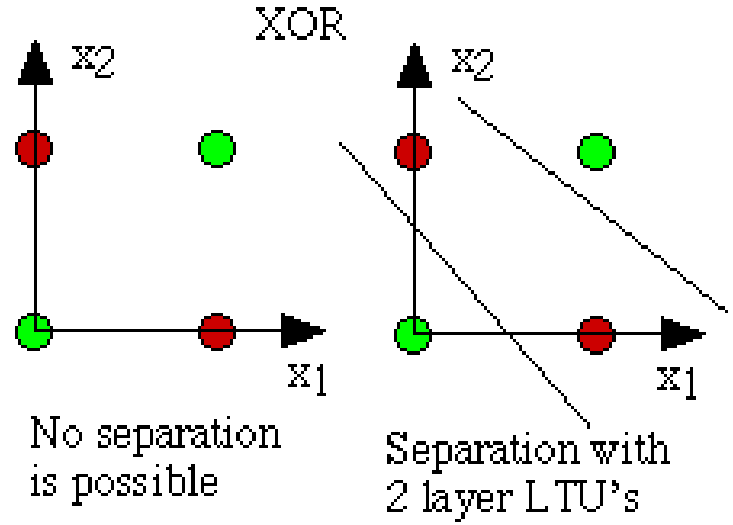
- Deep machines are more efficient for representing certain classes of functions, particularly those involved in visual recognition
  - They can represent more complex functions with less “hardware”
- We need an efficient parameterization of the class of functions that are useful for “AI” tasks (vision, audition, NLP...)

Example XOR



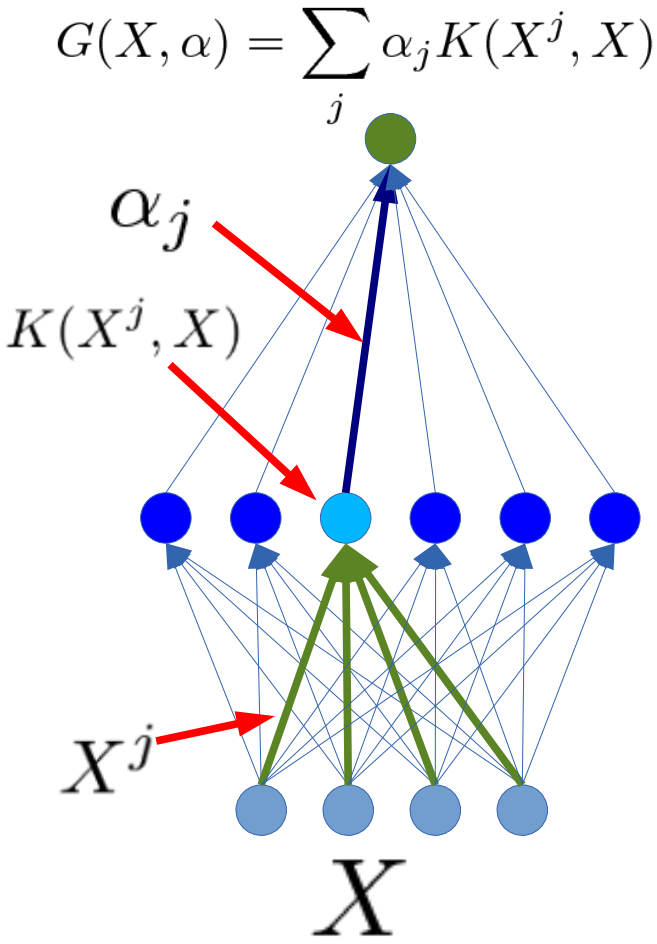
a	b	a XOR b
1	1	0
0	1	1
1	0	1
0	0	0





Which models are deep?

- 2-layer models are not deep (even if you train the first layer)
  - Because there is no feature hierarchy
- Neural nets with 1 hidden layer are not deep
- SVMs and Kernel methods are not deep
  - Layer1: kernels; layer2: linear
  - The first layer is “trained” in with the simplest unsupervised method ever devised: using the samples as templates for the kernel functions.
- Classification trees are not deep
  - No hierarchy of features. All decisions are made in the input space

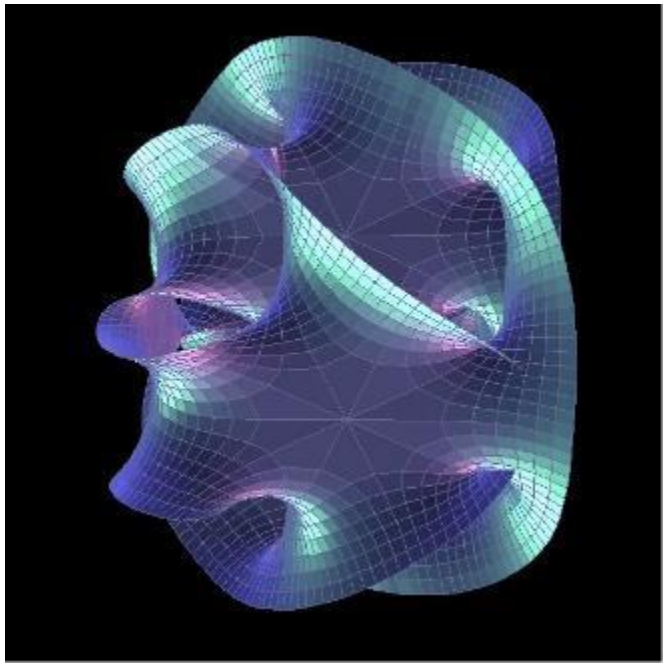


What are good features?

Discovering the hidden structure in high-dimensional data the manifold hypothesis

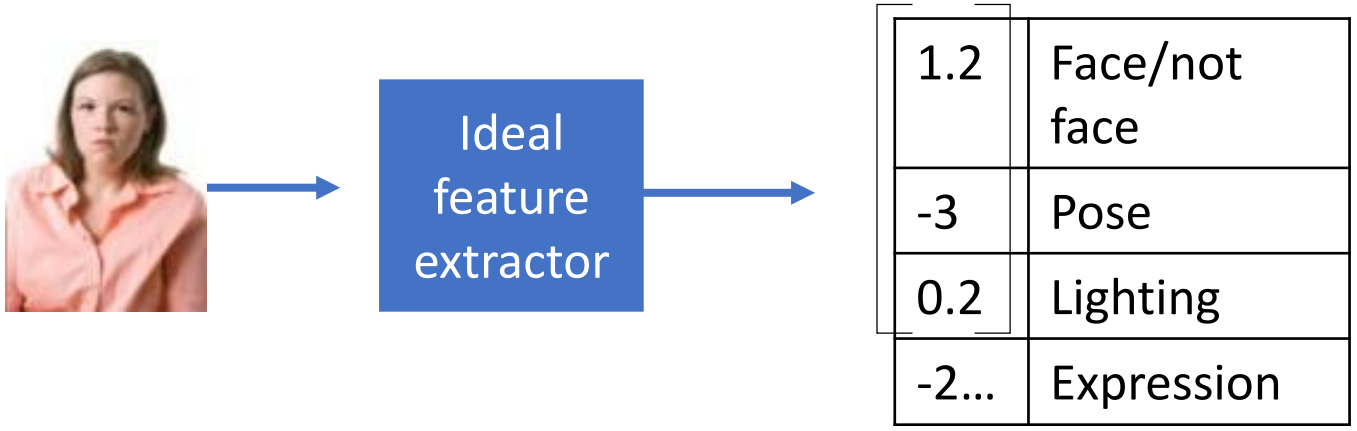
- Learning representations of data:
  - Discovering & disentangling the independent explanatory factors
- The manifold hypothesis:
  - Natural data lives in a low-dimensional (non-linear) manifold
  - Because variables in natural data are mutually dependent





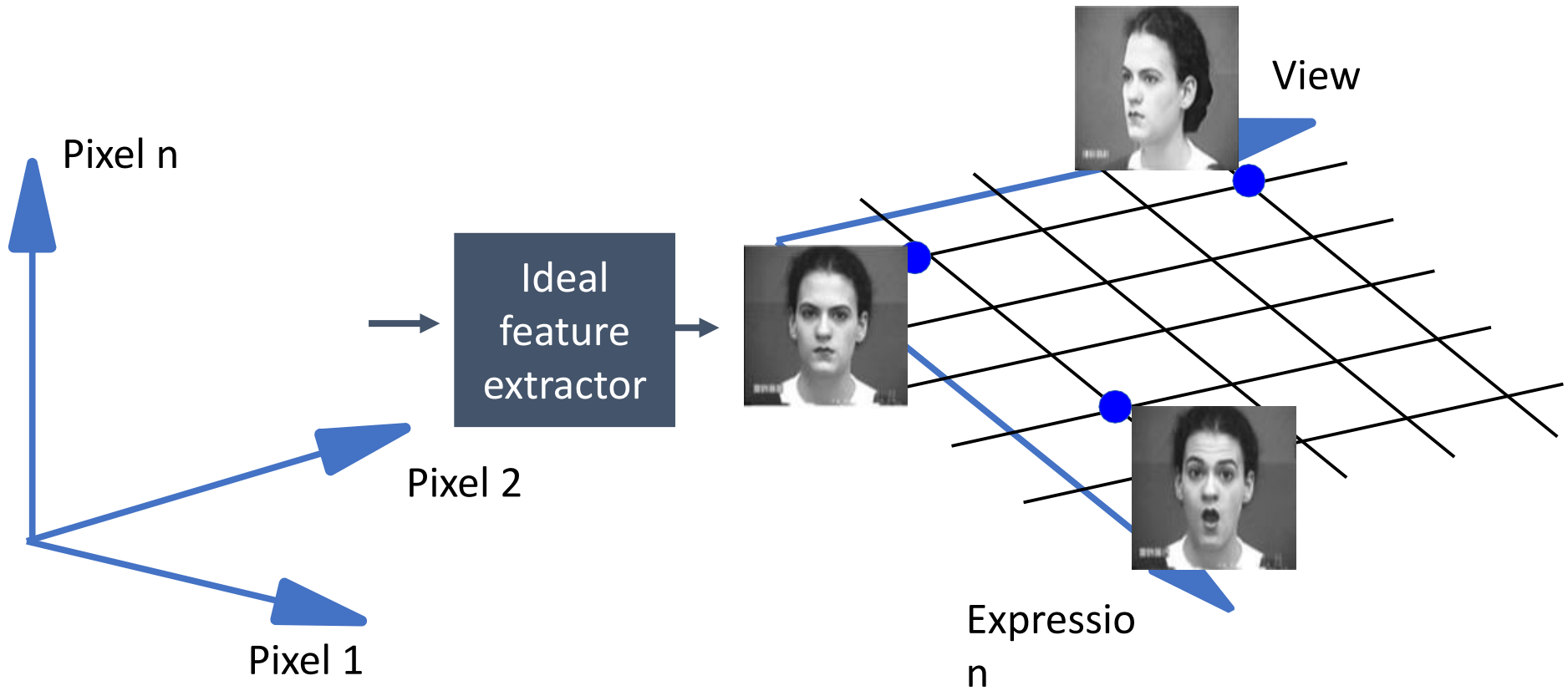
Discovering the hidden structure in high-dimensional data

- Example: all face images of a person
  - 1000x1000 pixels = 1,000,000 dimensions
  - But the face has 3 Cartesian coordinates and 3 Euler angles and humans have less than about 50 muscles in the face
  - Hence the manifold of face images for a person has <56 dimensions
- The perfect representations of a face image:
  - Its coordinates on the face manifold
  - Its coordinates away from the manifold
- We do not have good and general methods to learn functions that turns an image into this kind of representation



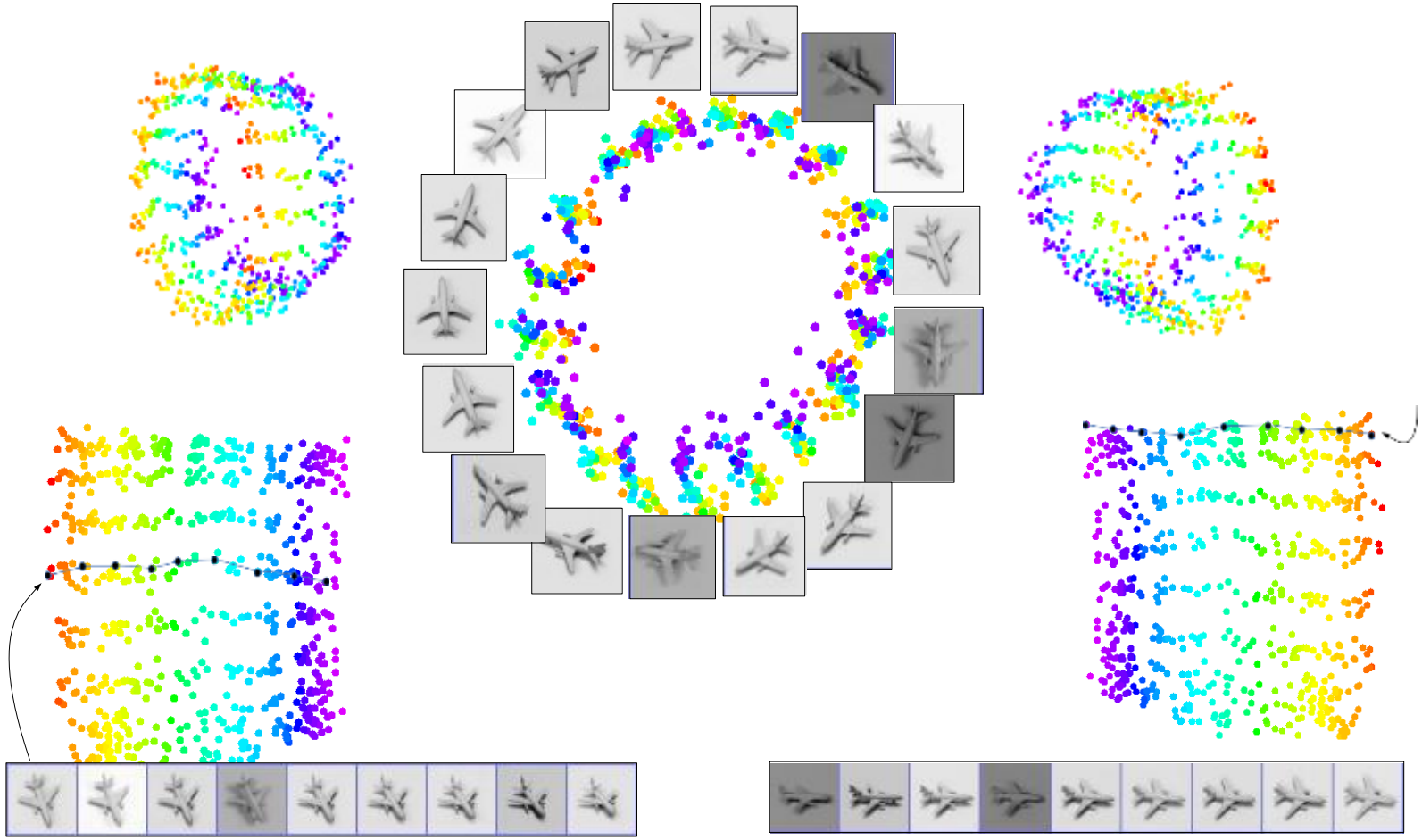
Disentangling factors of variation

The ideal disentangling feature extractor



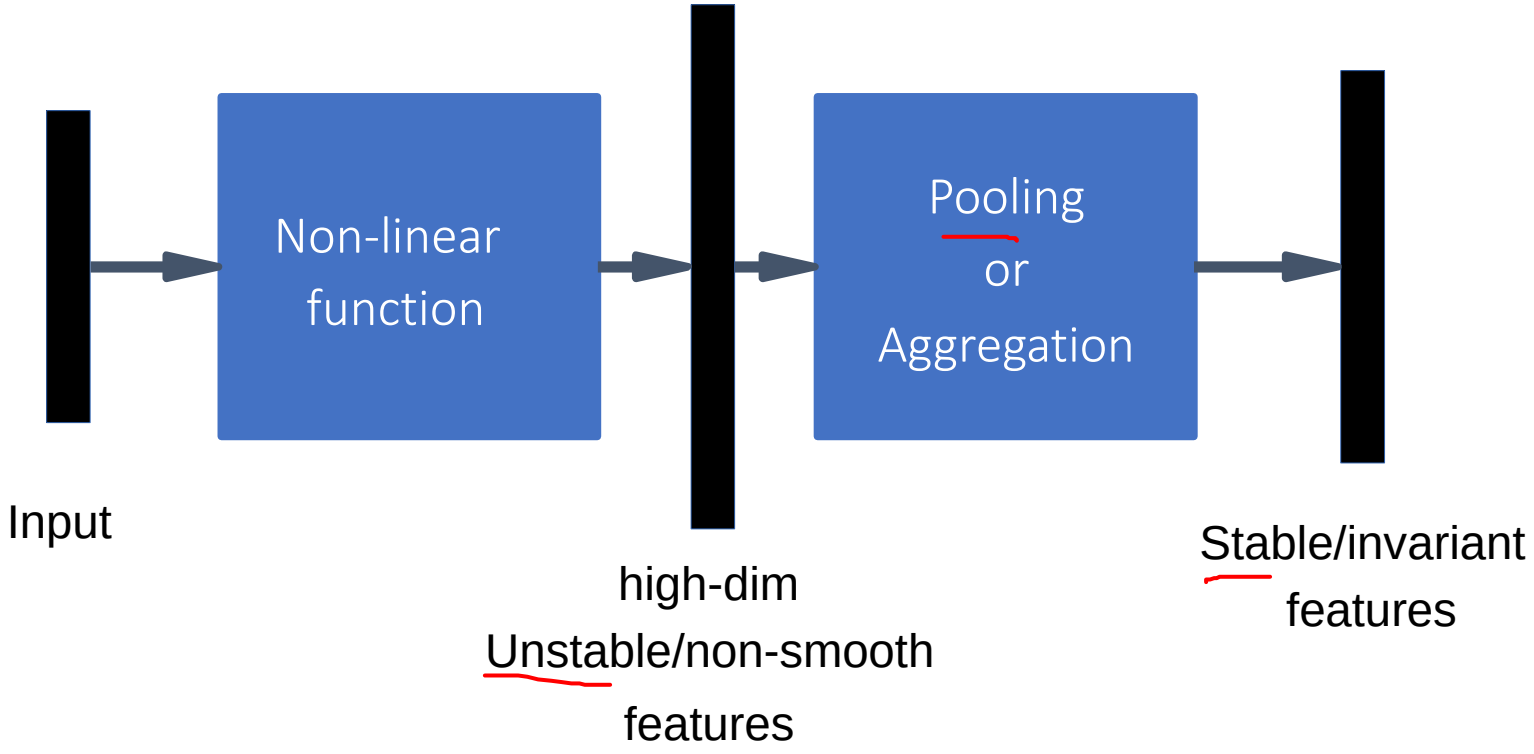
Data manifold & invariance: Some variations must be eliminated

– Azimuth-Elevation manifold. Ignores lighting. [Hadsell et al. CVPR 2006]



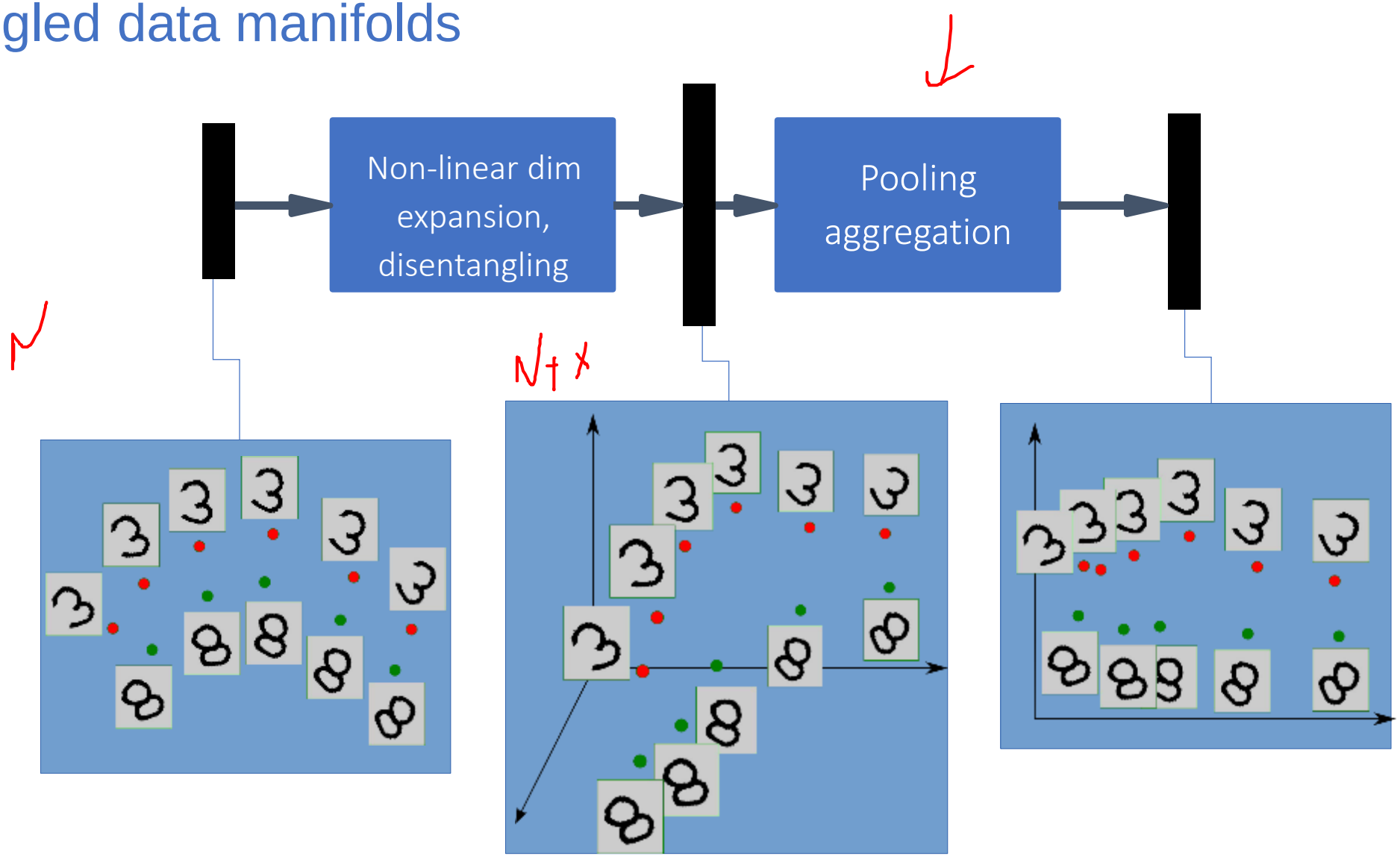
Basic idea for invariant feature learning

- Embed the input non-linearly into a high(er) dimensional space
  - In the new space, things that were non separable may become separable
- Pool regions of the new space together
  - Bringing together things that are semantically similar. Like pooling.



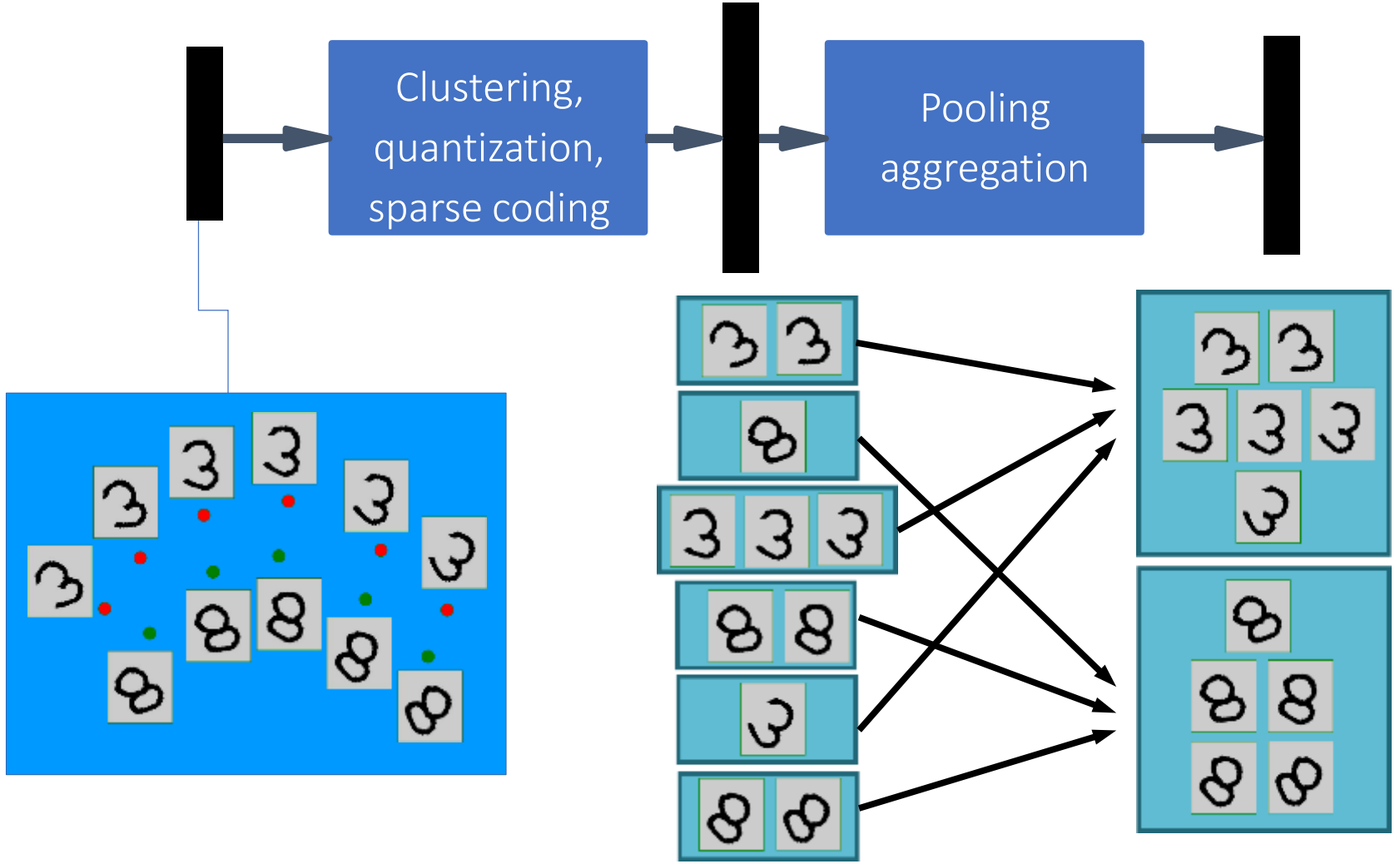
Non-linear expansion → pooling

Entangled data manifolds



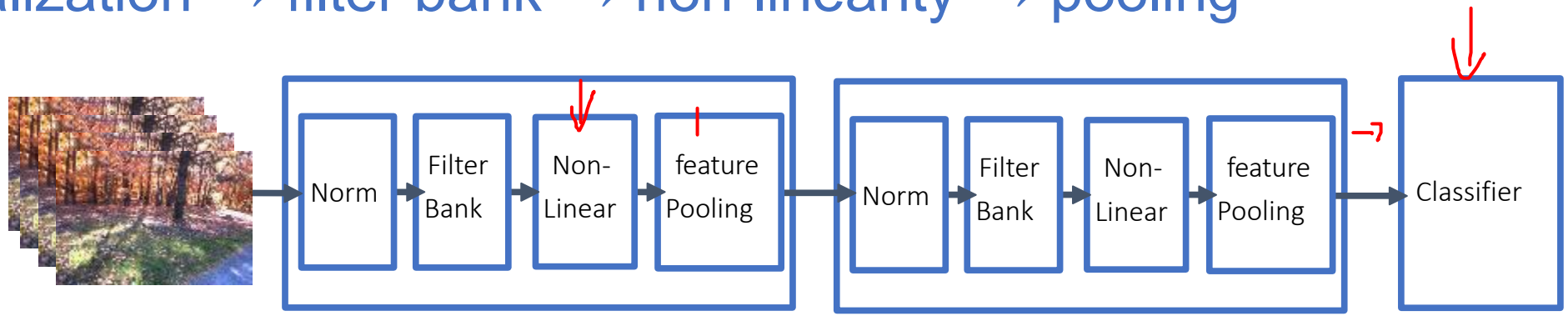
Sparse non-linear expansion → pooling

Use clustering to break things apart, pool together similar things



Overall architecture:

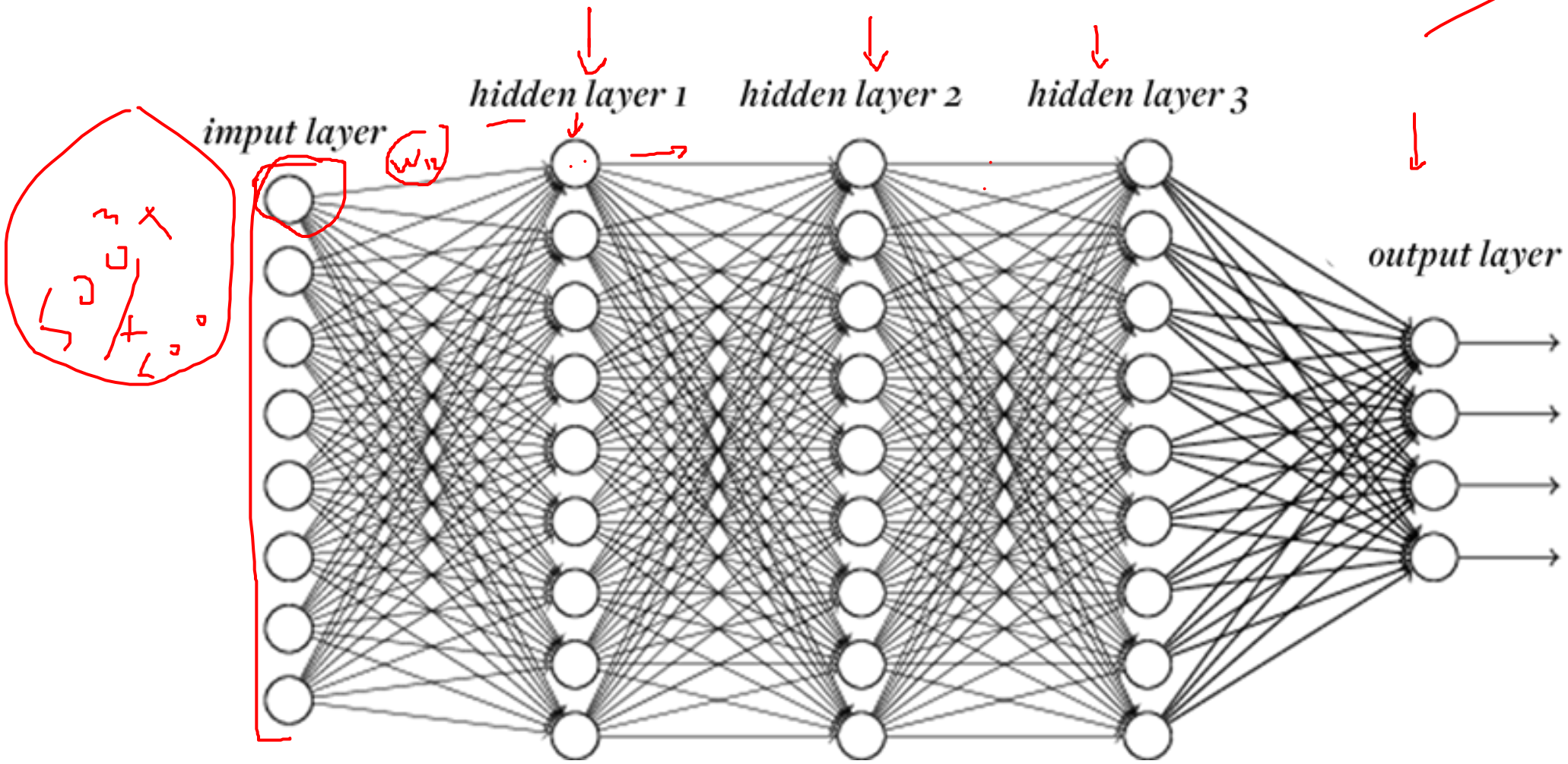
Normalization → filter bank → non-linearity → pooling



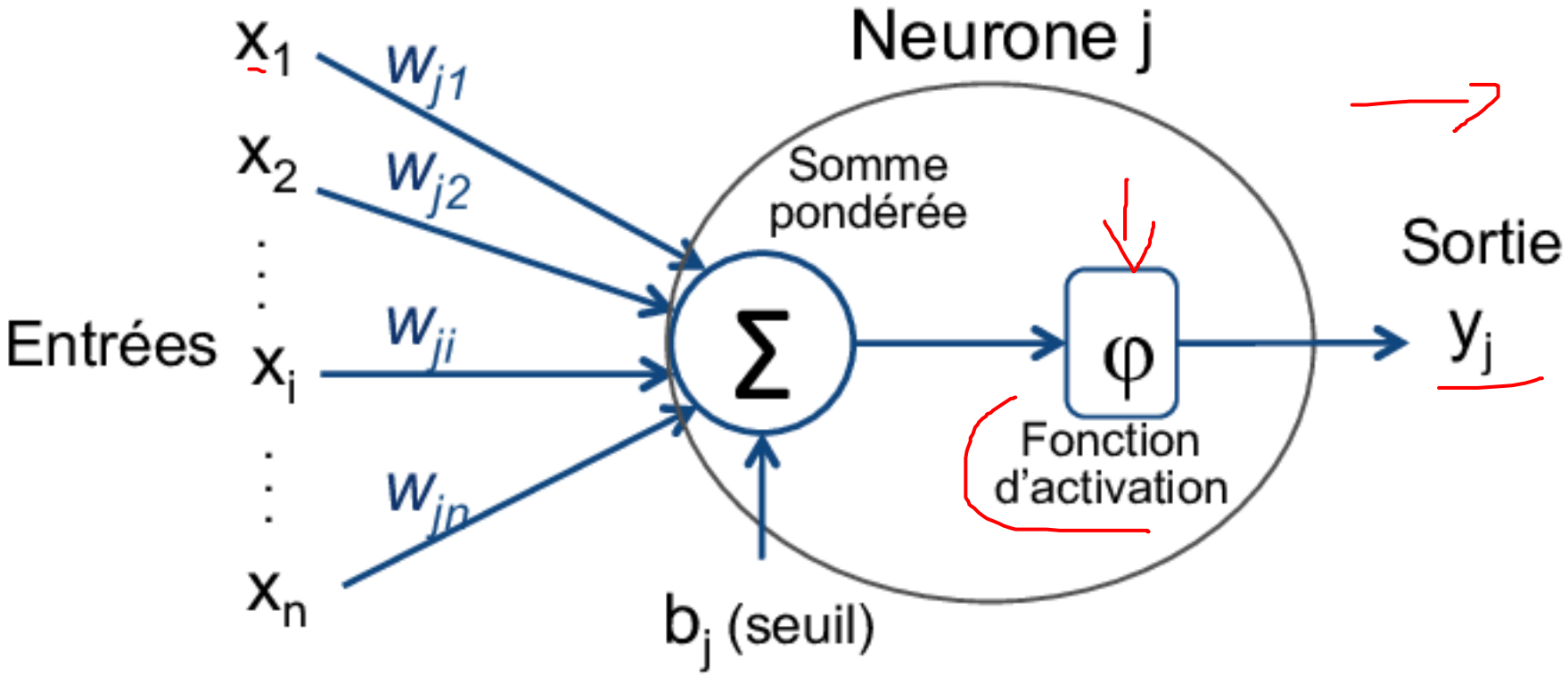
- Stacking multiple stages of
  - [Normalization → filter bank → non-linearity → pooling].
- Normalization: variations on whitening
  - Subtractive: average removal, high pass filtering
  - Divisive: local contrast normalization, variance normalization
- Filter bank: dimension expansion, projection on overcomplete basis
- Non-linearity: sparsification, saturation, lateral inhibition....
  - Rectification (relu), component-wise shrinkage, tanh, winner-takes-all
- Pooling: aggregation over space or feature type
  -

$$X_i; \quad L_p: \sqrt[p]{X_i^p}; \quad PROB: \frac{1}{b}\log\left(\sum_i e^{bX_i}\right)$$

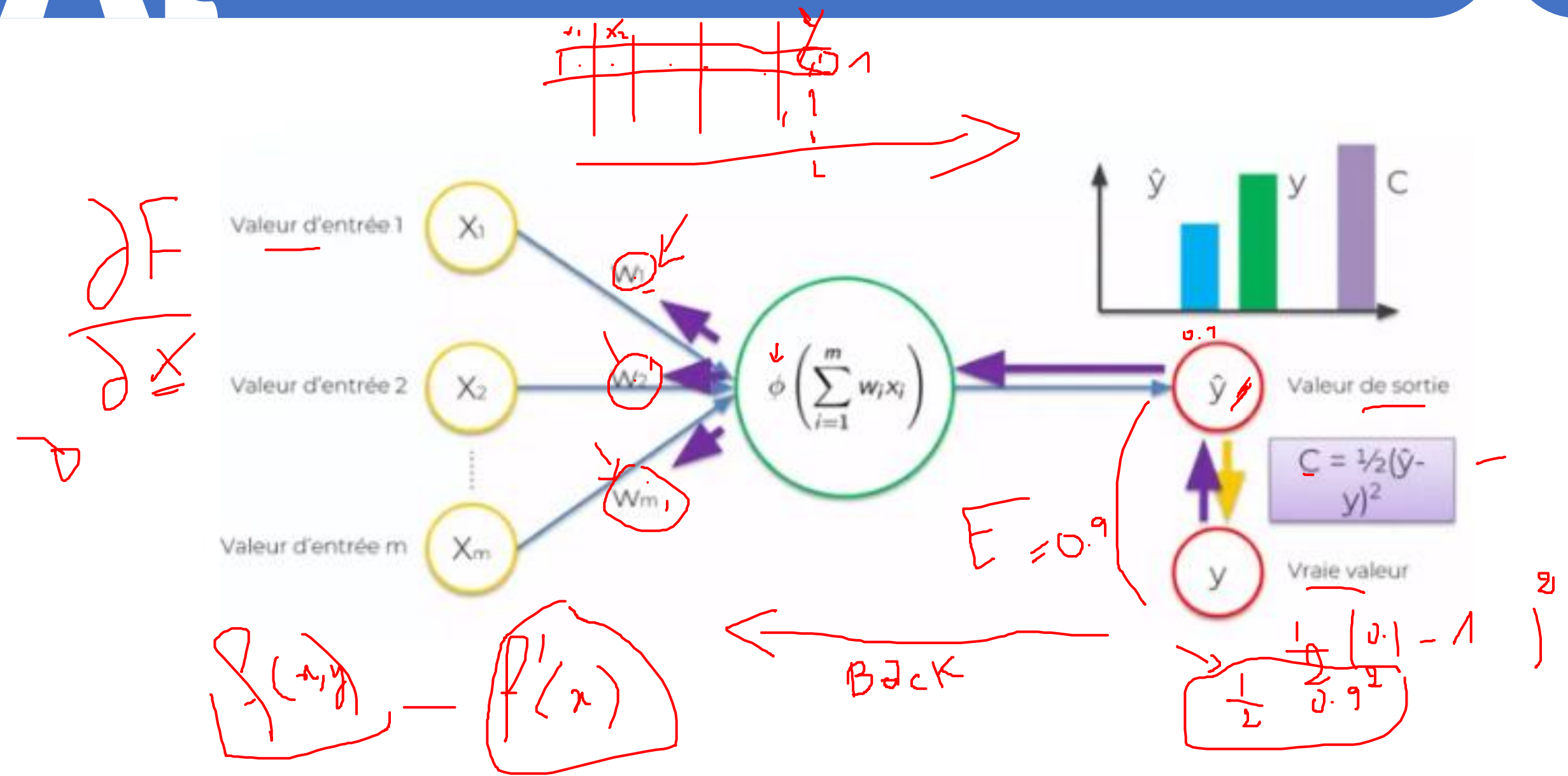
ANN: Les Principes



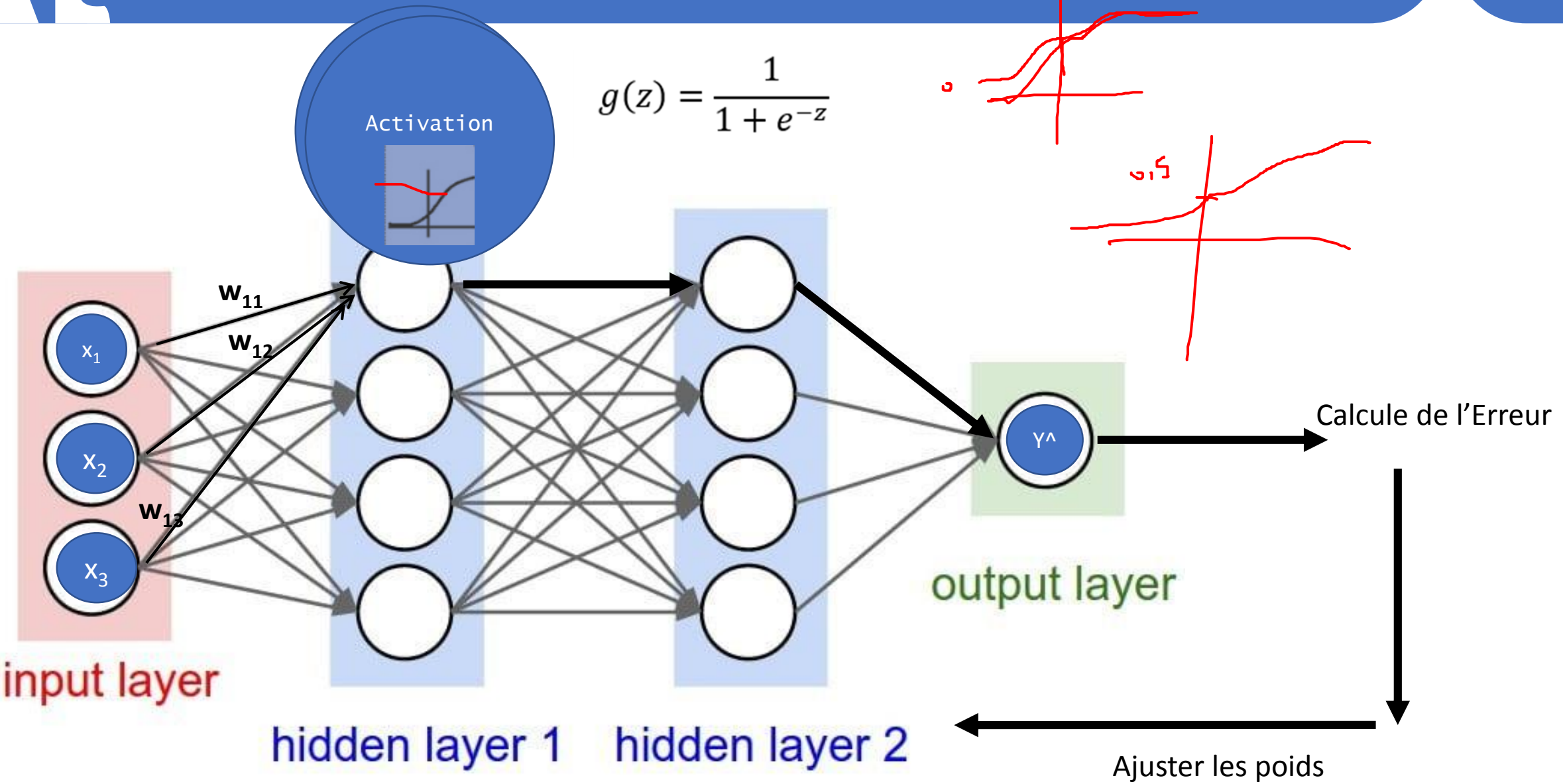
Perceptron


Neurone j
Somme pondérée
Entrées
x_1
x_2
x_i
x_n
w_{j1}
w_{j2}
w_{ji}
w_{jn}
Σ
φ
Fonction d'activation
b_j (seuil)
Sortie
y_j

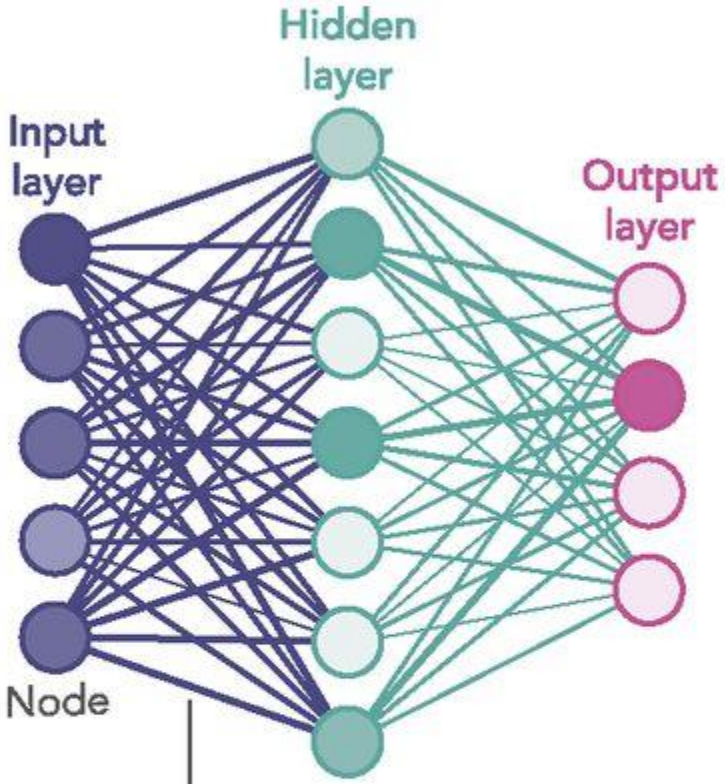
Perceptron


Valeur d'entrée 1
Valeur d'entrée 2
Valeur d'entrée m
Valeur de sortie
Vraie valeur
C = ½(ŷ-y)²
ŷ
y
C

Le perceptron multicouche

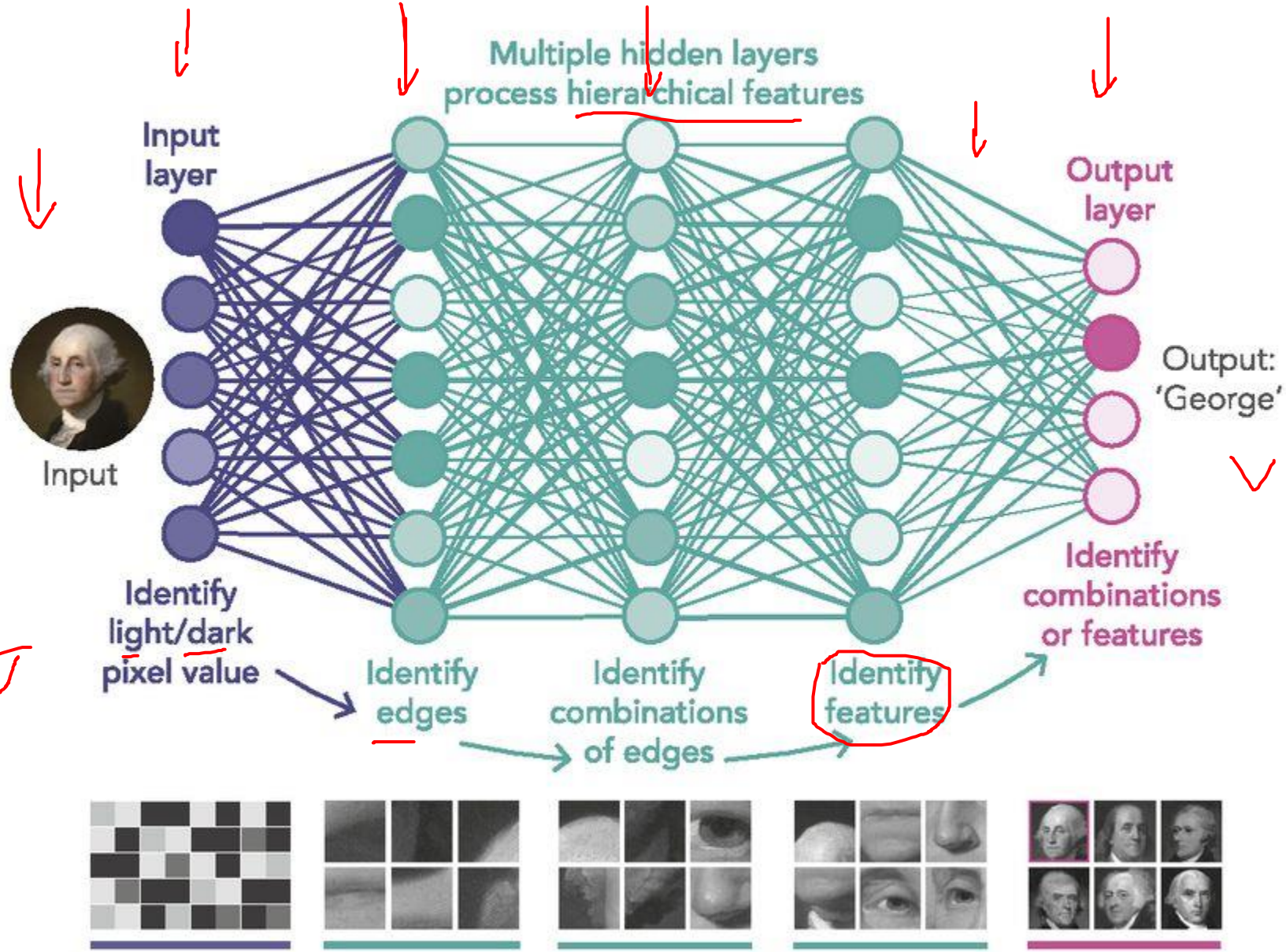


1980S-ERA NEURAL NETWORK



Links carry signals from one node to another, boosting or damping them according to each link's 'weight'.

DEEP LEARNING NEURAL NETWORK



Exemple

Warmup: Logistic regression is a pretty popular technique in machine learning to classify data into two categories. This technique builds over linear regression by using the same linear model but this is followed by the sigmoid function which converts the output of the linear model to a value between 0 and 1. This value can then be interpreted as a probability. This is usually represented as:

$$P(y = 1|x_{\text{in}}) = x_{\text{out}} = \sigma(x_{\text{in}}) = \frac{1}{1 + e^{-x_{\text{in}}}} \tag{1}$$

where x_{in} as the name would suggest is the input scalar (which is also the output of linear model) and x_{out} is the output scalar.

If the error backpropagated to x_{out} is $\frac{\partial E}{\partial x_{\text{out}}}$, write the expression for $\frac{\partial E}{\partial x_{\text{in}}}$ in terms of $\frac{\partial E}{\partial x_{\text{out}}}$.

Rappel

$(u+v)' = u' + v'$
$(ku)' = ku'$
$(u \times v)' = u'v + uv'$
$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$
$\left(u^2\right)' = 2u'u$
$\left(\frac{1}{u}\right)' = -\frac{u'}{u^2}$

$(g \circ f)' = (g' \circ f) \times f'.$

$$\sigma(x_{\text{in}}) = \frac{1}{1 + e^{-x_{\text{in}}}}$$

$$= \frac{(1 + e^{-x})(0) - (1)(-e^{-x})}{(1 + e^{-x})^2} = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1 - 1 + e^{-x}}{(1 + e^{-x})^2}$$

$$= \frac{1 + e^{-x}}{(1 + e^{-x})^2} - \frac{1}{(1 + e^{-x})^2} = \frac{1}{(1 + e^{-x})} - \frac{1}{(1 + e^{-x})^2} = \frac{1}{(1 + e^{-x})}\left(1 - \frac{1}{1 + e^{-x}}\right)$$

$$[1 - \sigma(x_{in})] \cdot \sigma(x_{in})$$

Exemple

Logistic regression:

$$P(y=1|x_{in}) = x_{out} = \sigma(x_{in}) = \frac{1}{1+e^{-x_{in}}}$$

If the error backpropogated to x_{out} is $\frac{\partial E}{\partial x_{out}}$, write the expression for $\frac{\partial E}{\partial x_{in}}$ in terms of $\frac{\partial E}{\partial x_{out}}$.

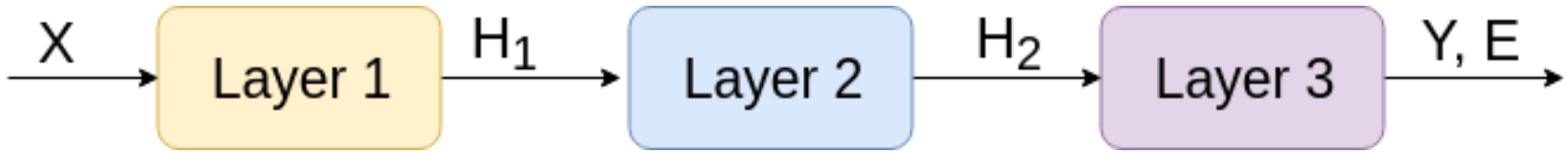
Answer:

$$\begin{aligned}
\frac{\partial E}{\partial x_{in}} &= \frac{\partial E}{\partial x_{out}} \cdot \frac{\partial x_{out}}{\partial x_{in}} \\
&= \frac{\partial E}{\partial x_{out}} \cdot \frac{\partial[\sigma(x_{in})]}{\partial x_{in}} \\
&= \frac{\partial E}{\partial x_{out}} \cdot [1-\sigma(x_{in})] \cdot \sigma(x_{in}) \\
&= \frac{\partial E}{\partial x_{out}} \cdot [1-x_{out}] \cdot x_{out}
\end{aligned}$$

Réseau se compose de:

- Des couches
- Deux opérations
  - Propagation en avant
  - Propagation en arrière
- Des couches d’activation
- Une couche full connected
- Fonction de perte
- Opération d’apprentissage
- Opération de prédiction/classification

Passe avant — Forward propagation

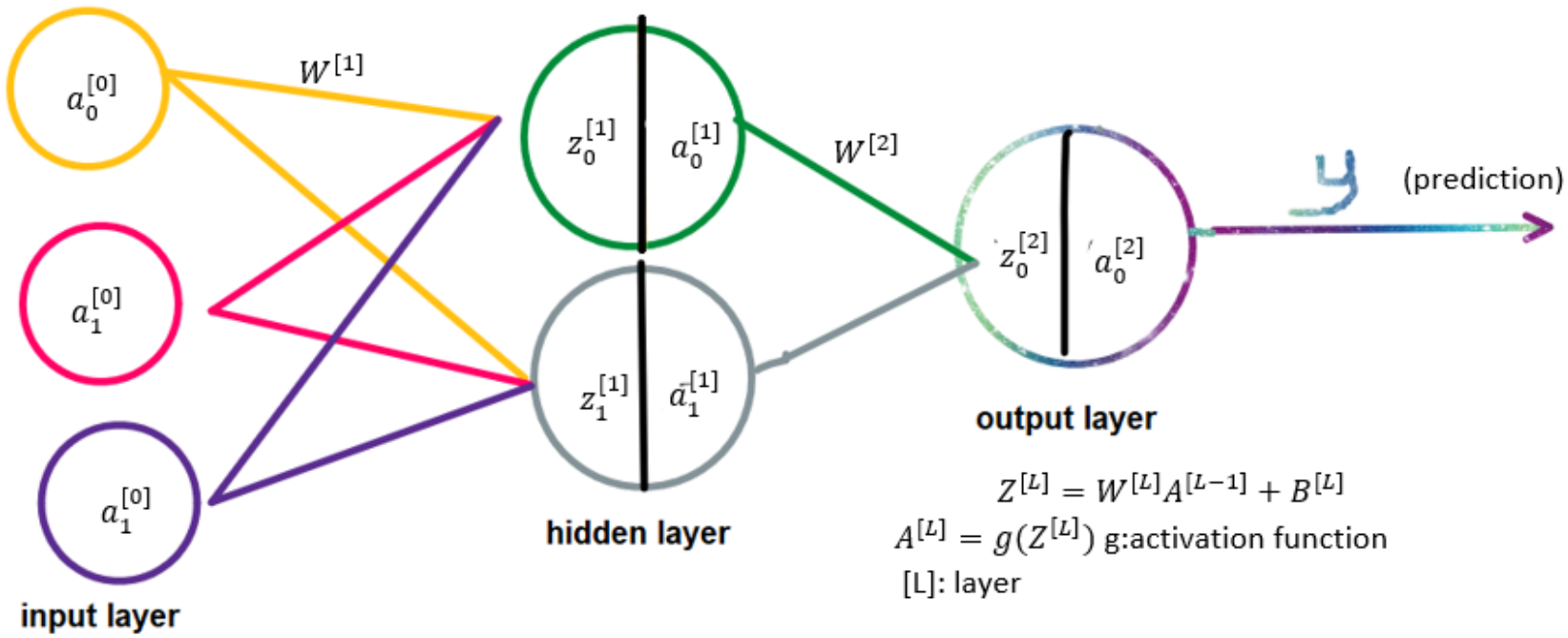


- On propage l'entrée **X** (image, son, texte, etc.) dans le réseau de neurones jusqu'à obtenir la sortie **Y**. Puis, on observe une erreur **E** qu'il faut diminuer.

$$y_j = b_j + \sum_i x_i w_{ij}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

Principe (propagation en avant)



Notation matricielle

$$X = \begin{bmatrix} x_1 & \dots & x_i \end{bmatrix} \quad W = \begin{bmatrix} w_{11} \dots w_{1j} \\ \vdots \quad \ddots \quad \vdots \\ w_{i1} \dots w_{ij} \end{bmatrix} \quad B = \begin{bmatrix} b_1 & \dots & b_j \end{bmatrix}$$

$$Y = X * W + B$$

Descente de Gradient

$$w \leftarrow w - \alpha \frac{\partial E}{\partial w}$$

Fonction de perte

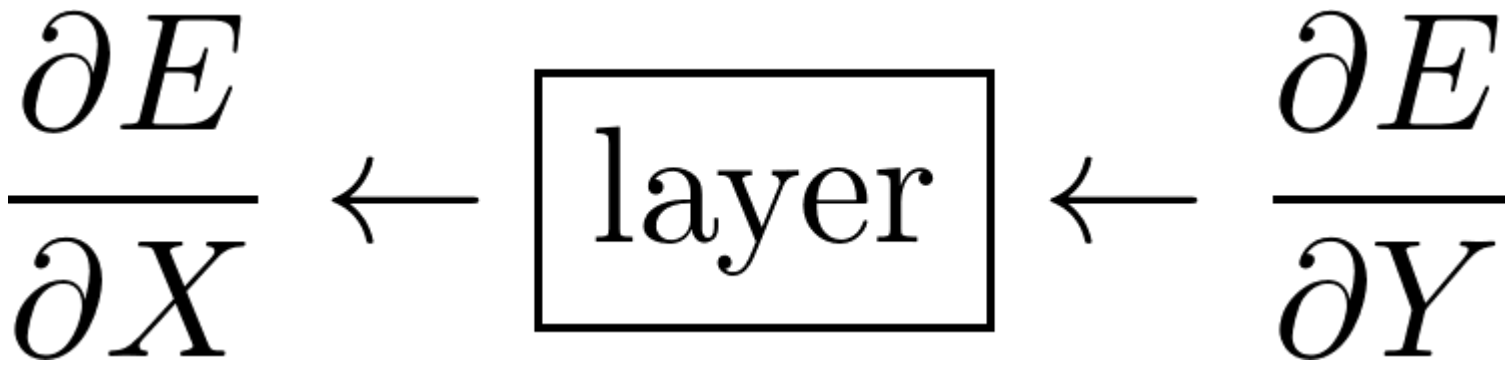
- L'erreur du réseau, qui mesure le degré de performance du modèle pour une entrée donnée, doit être défini. Il existe de nombreuses façons de définir l'erreur, et l'une des plus connues est appelée **MSE — Mean Squared Error.**

$$E = \frac{1}{n}\sum_{i}^{n}(y_i^* - y_i)^2$$

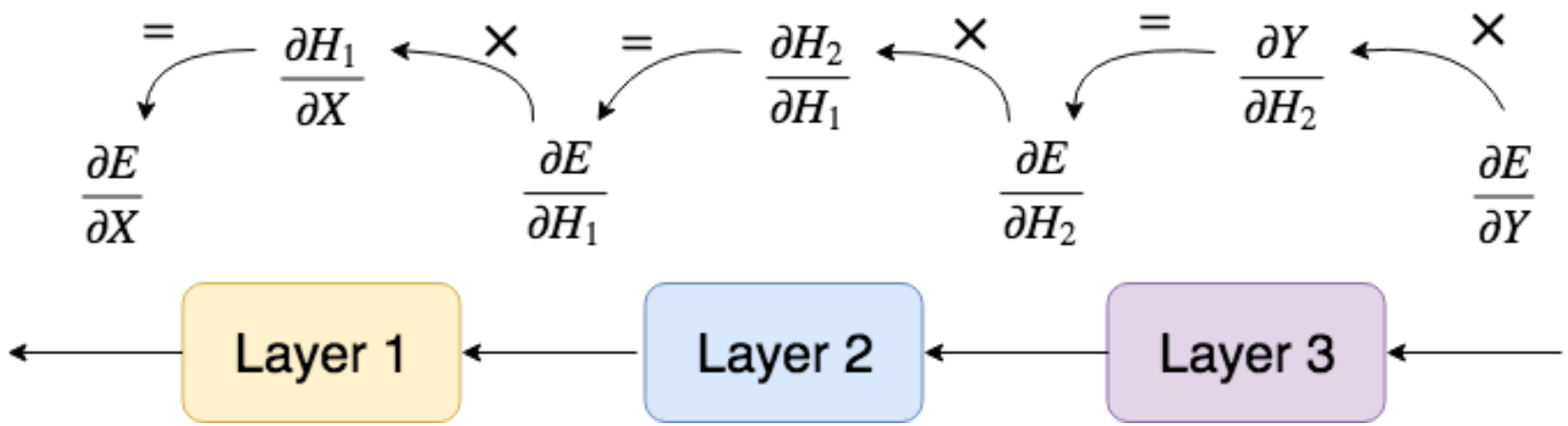
- Pour démunie la valeur de E, il faut ajuster les valeurs des poids Wi, pour cela il faut déterminer le degré de l'influence de Wi sur l'erreur.

Passe arrière — backward propagation

- Supposons que l'on donne à une couche la **dérivée de l'erreur** par rapport à **sa sortie** (∂E/∂Y), alors elle doit être capable de donner la **dérivée de l'erreur** par rapport à **son entrée** (∂E/∂X).



Passe arrière — Backward propagation



Explication

L’erreur E est un **scalaire** (un nombre), et X et Y sont des **matrices**.

$$\frac{\partial E}{\partial X} = \begin{bmatrix} \frac{\partial E}{\partial x_1} & \frac{\partial E}{\partial x_2} & \cdots & \frac{\partial E}{\partial x_i} \end{bmatrix}$$

$$\frac{\partial E}{\partial Y} = \begin{bmatrix} \frac{\partial E}{\partial y_1} & \frac{\partial E}{\partial y_2} & \cdots & \frac{\partial E}{\partial y_j} \end{bmatrix}$$

Explication

Si une couche a accès à ∂E/∂Y où Y est **sa propre sortie,** alors nous pouvons très facilement calculer la dérivée de l'erreur par rapport à ses paramètres ∂E/∂W (pour l'étape de l'ajustement), et cela, **indépendamment de l'architecture globale du réseau de neurones!** Il suffit d'utiliser la règle de dérivation des fonctions composées :

$$\frac{\partial E}{\partial w} = \sum_j \frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial w}$$

Explication

Pourquoi avons-nous besoin de ∂E/∂X ?
N'oubliez pas, la sortie d'une couche est l'entrée de la couche suivante. Donc **∂E/∂X pour une couche sera ∂E/∂Y pour la couche précédente!** Une fois munit de son propre ∂E/∂Y, la couche précédente pourra à son tour ajuster ses paramètres. Pour calculer ∂E/∂X on utilise encore une fois la règle de dérivation des fonctions composées :

$$\frac{\partial E}{\partial x_i} = \sum_j \frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial x_i}$$

Notation matricielle

$$\frac{\partial E}{\partial W} = \begin{bmatrix} \frac{\partial E}{\partial w_{11}} & \cdots & \frac{\partial E}{\partial w_{1j}} \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial w_{i1}} & \cdots & \frac{\partial E}{\partial w_{ij}} \end{bmatrix}$$

i est le nombre de neurones d'entrée et j le nombre de neurones de sortie.

Notation matricielle

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_1} * \frac{\partial y_1}{\partial w_{ij}} + \cdots + \frac{\partial E}{\partial y_j} * \frac{\partial y_j}{\partial w_{ij}}$$

$$= \frac{\partial E}{\partial y_j} * x_i$$

Notation matricielle

$$\frac{\partial E}{\partial W} = \begin{bmatrix} \frac{\partial E}{\partial y_1} * x_1 & \dots & \frac{\partial E}{\partial y_j} * x_1 \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial y_1} * x_i & \dots & \frac{\partial E}{\partial y_j} * x_i \end{bmatrix}$$

$$= \begin{bmatrix} x_1 \\ \vdots \\ x_i \end{bmatrix} * \begin{bmatrix} \frac{\partial E}{\partial y_1} & \dots & \frac{\partial E}{\partial y_j} \end{bmatrix}$$

$$= X^t * \frac{\partial E}{\partial Y}$$

Notation matricielle

- Première formule qui permet d'ajuster les poids ! Calculons à présent ∂E/∂B.

$$\frac{\partial E}{\partial B} = \left[\frac{\partial E}{\partial b_1} \quad \frac{\partial E}{\partial b_2} \quad \cdots \quad \frac{\partial E}{\partial b_j} \right]$$

Notation matricielle

- Encore une fois, ∂E/∂B doit-être de la même dimension que B, une dérivée par *bias*.

$$\frac{\partial E}{\partial b_j} = \frac{\partial E}{\partial y_1} * \frac{\partial y_1}{\partial b_j} + \cdots + \frac{\partial E}{\partial y_j} * \frac{\partial y_j}{\partial b_j}$$

$$= \frac{\partial E}{\partial y_j}$$

- D'où,

$$\frac{\partial E}{\partial B} = \begin{bmatrix} \frac{\partial E}{\partial y_1} & \frac{\partial E}{\partial y_2} & \cdots & \frac{\partial E}{\partial y_j} \end{bmatrix}$$

$$= \frac{\partial E}{\partial Y}$$

Notation matricielle

- Il nous reste simplement à calculer ∂E/∂X pour que la couche précédente puisse faire les mêmes calculs.

$$\frac{\partial E}{\partial X} = \left[\frac{\partial E}{\partial x_1} \quad \frac{\partial E}{\partial x_2} \quad \dots \quad \frac{\partial E}{\partial x_i}\right]$$

- Encore une fois, dérivation de fonctions composées :

$$\frac{\partial E}{\partial x_i} = \frac{\partial E}{\partial y_1} * \frac{\partial y_1}{\partial x_i} + \dots + \frac{\partial E}{\partial y_j} * \frac{\partial y_j}{\partial x_i}$$

$$= \frac{\partial E}{\partial y_1} * w_{i1} + \dots + \frac{\partial E}{\partial y_j} * w_{ij}$$

Notation matricielle

- Nous pouvons écrire la matrice entière :

$$\frac{\partial E}{\partial X} = \left[\left(\frac{\partial E}{\partial y_1} * w_{11} + \ldots + \frac{\partial E}{\partial y_j} * w_{1j}\right) \quad \ldots \quad \left(\frac{\partial E}{\partial y_1} * w_{i1} + \ldots + \frac{\partial E}{\partial y_j} * w_{ij}\right) \right]$$

$$= \left[\frac{\partial E}{\partial y_1} \quad \ldots \quad \frac{\partial E}{\partial y_j} \right] * \begin{bmatrix} w_{11} & \ldots & w_{i1} \\ \vdots & \ddots & \vdots \\ w_{1j} & \ldots & w_{ij} \end{bmatrix}$$

$$= \frac{\partial E}{\partial Y} * W^t$$

La couche FC

$$w \leftarrow w - \alpha \frac{\partial E}{\partial w}$$

- Nous avons obtenu ces trois formules fondamentale pour la couche FC !

$$\frac{\partial E}{\partial X} = \frac{\partial E}{\partial Y} * W^t$$

$$\frac{\partial E}{\partial W} = X^t * \frac{\partial E}{\partial Y}$$

$$\frac{\partial E}{\partial B} = \frac{\partial E}{\partial Y}$$

Fonction de perte

- L’erreur du réseau, qui mesure le degré de performance du modèle pour une entrée donnée, doit être défini. Il existe de nombreuses façons de définir l’erreur, et l’une des plus connues est appelée **MSE — Mean Squared Error.**

$$E = \frac{1}{n}\sum_{i}^{n}(y_i^* - y_i)^2$$

Sa dérivée

$$\frac{\partial E}{\partial Y} = \left[\frac{\partial E}{\partial y_1} \quad \dots \quad \frac{\partial E}{\partial y_i}\right]$$

$$= \frac{2}{n}\left[y_1 - y_1^* \quad \dots \quad y_i - y_i^*\right]$$

$$= \frac{2}{n}(Y - Y^*)$$

Il suffira de donner cette valeur à la **dernière** couche lors de la passe arrière, ce qui lui permettra d'ajuster ses paramètres, puis elle calculera **∂E/∂X** qu'elle passera à la couche d'avant, qui fera le même procédé à son tour, etc.

Implémentation avec python

Voir ateliers

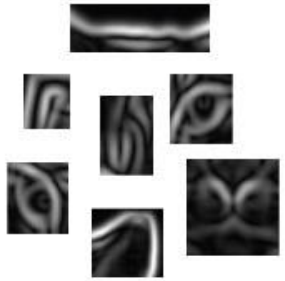
ML vs DL



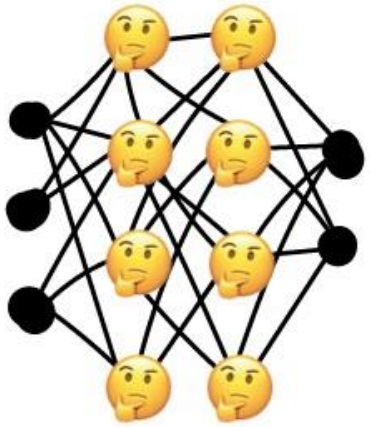
Original image



Preliminary processing



Hand-crafted features

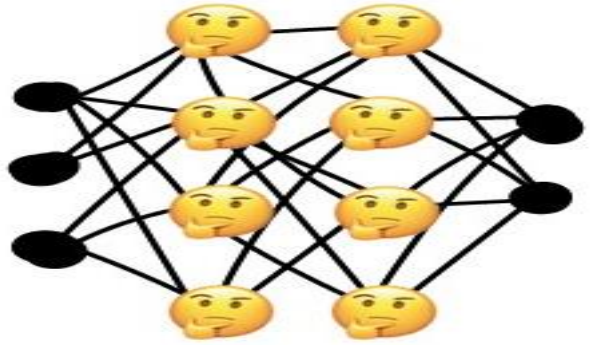


Neural Network

«cat»

Result





Neural Network

«cat»

Result

Images

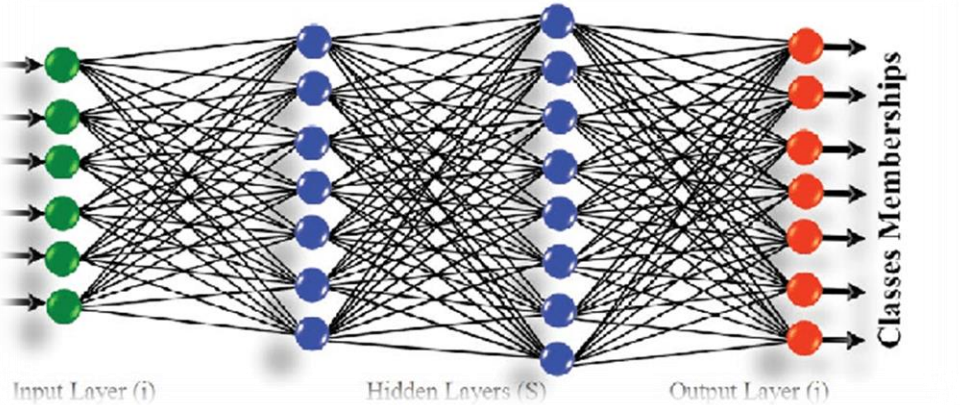
100

100

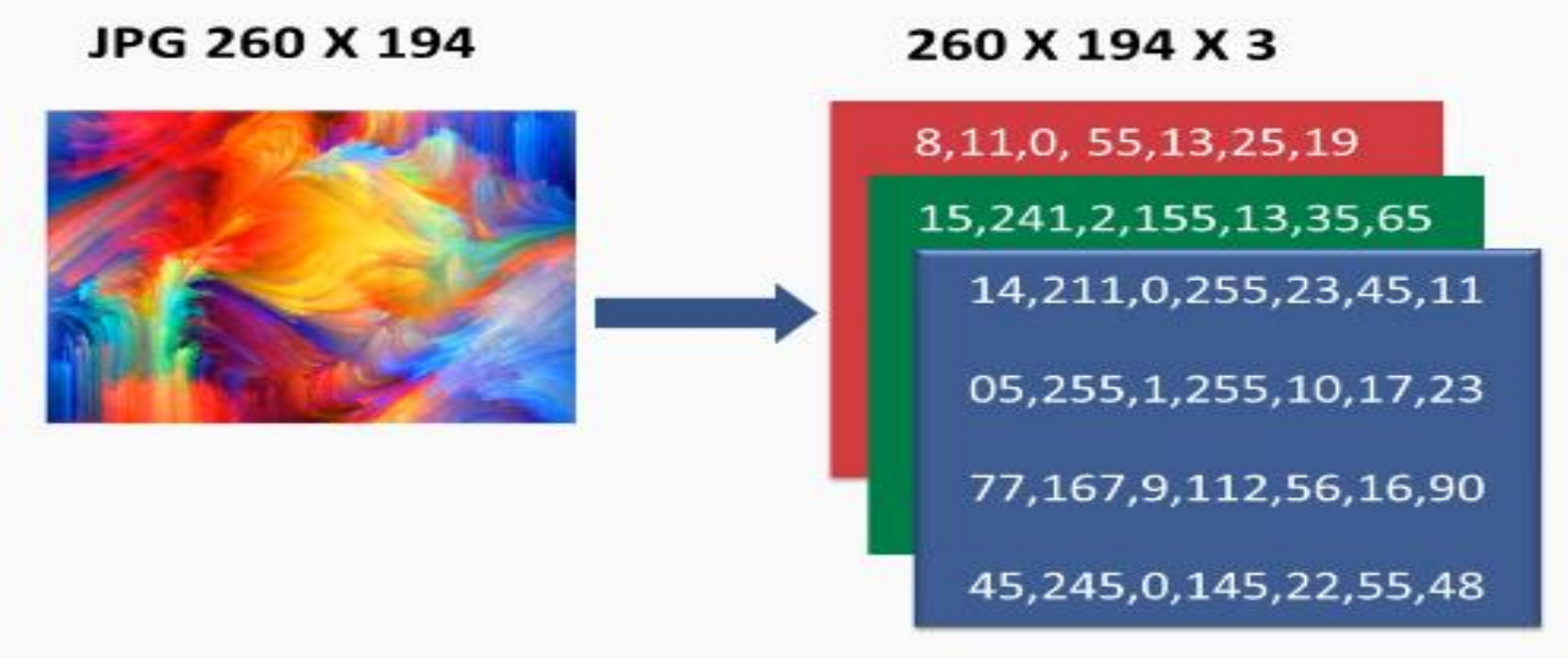


45	150	….	0
255	45	…	147
…	…	…	…
10	147	…	48

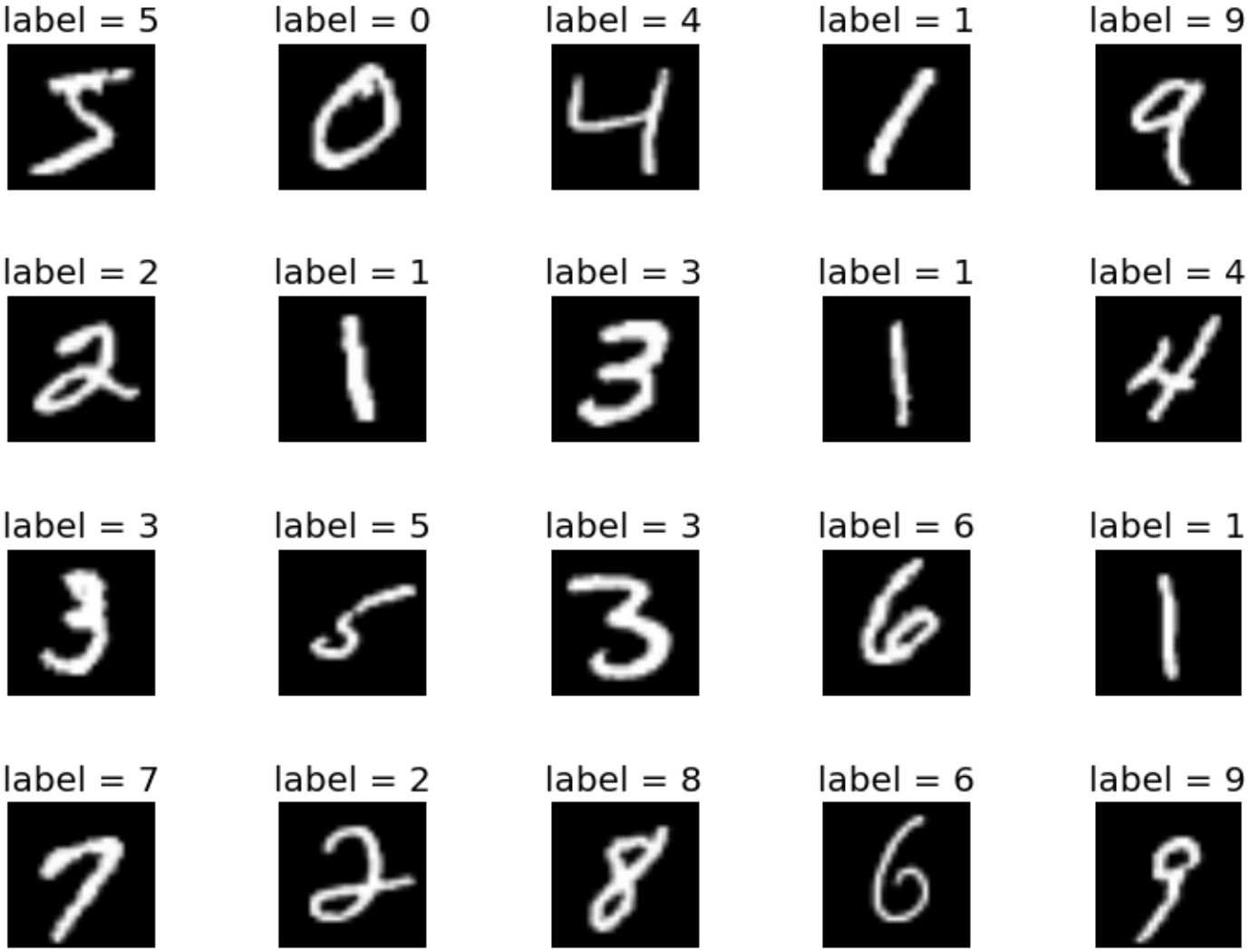
45
150
…
0
255
45
…
…
…
48



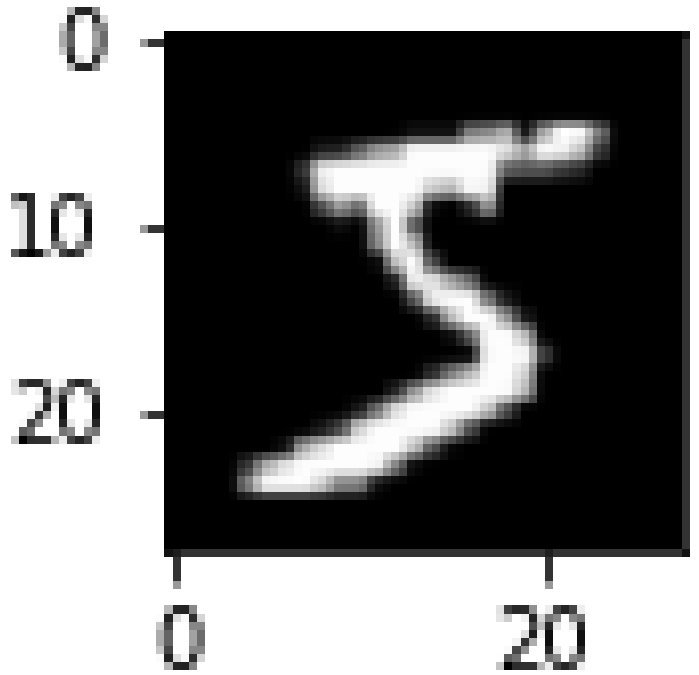
Rgb


JPG 260 X 194
260 X 194 X 3
8,11,0, 55,13,25,19
15,241,2,155,13,35,65
14,211,0,255,23,45,11
05,255,1,255,10,17,23
77,167,9,112,56,16,90
45,245,0,145,22,55,48

MNIST dataset



```
[[[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  3  18  18  18 126  136 175  26 166 255 247 127   0  0  0  0]
 [ 0  0  0  0  0  0  0  0 30  36  94 154 170 253 253 253 253  253 225 172 253 242 195  64   0  0  0  0]
 [ 0  0  0  0  0  0  0 49 238 253 253 253 253 253 253 253 253  251  93  82  82  56  39   0  0  0  0  0]
 [ 0  0  0  0  0  0  0 18 219 253 253 253 253 253 198 182 247  241   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  80 156 107 253 253 205  11   0  43  154   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0  14   1 154 253  90   0  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0 139 253 190   2  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0  11 190 253  70  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0  35 241 225 160 108    1  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0  81 240 253 253   119  25  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0  45 186 253   253 150  27  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0  16  93   252 253 187  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0  249 253 249  64  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0   0  46 130 183   253 253 207   2  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0  39 148 229 253 253   253 250 182   0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0  24 114 221 253 253 253 253   201  78   0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0  23  66 213 253 253 253 253 198  81    2   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  18 171 219 253 253 253 253 195  80   9   0   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  55 172 226 253 253 253 253 244 133  11   0  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 136 253 253 253 212 135 132  16   0  0  0  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0  0  0  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0  0  0  0  0    0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0  0  0  0  0    0  0  0  0  0  0  0  0  0  0  0]]]
```



– [[[0. 0. 0.

Exemple 2

les données du dossier médical des patients et indique s'ils ont eu un début de diabète dans les cinq ans. il s'agit d'un problème de classification binaire (apparition du diabète 1 ou non 0).
Toutes les variables d'entrée qui décrivent chaque patient sont numériques.

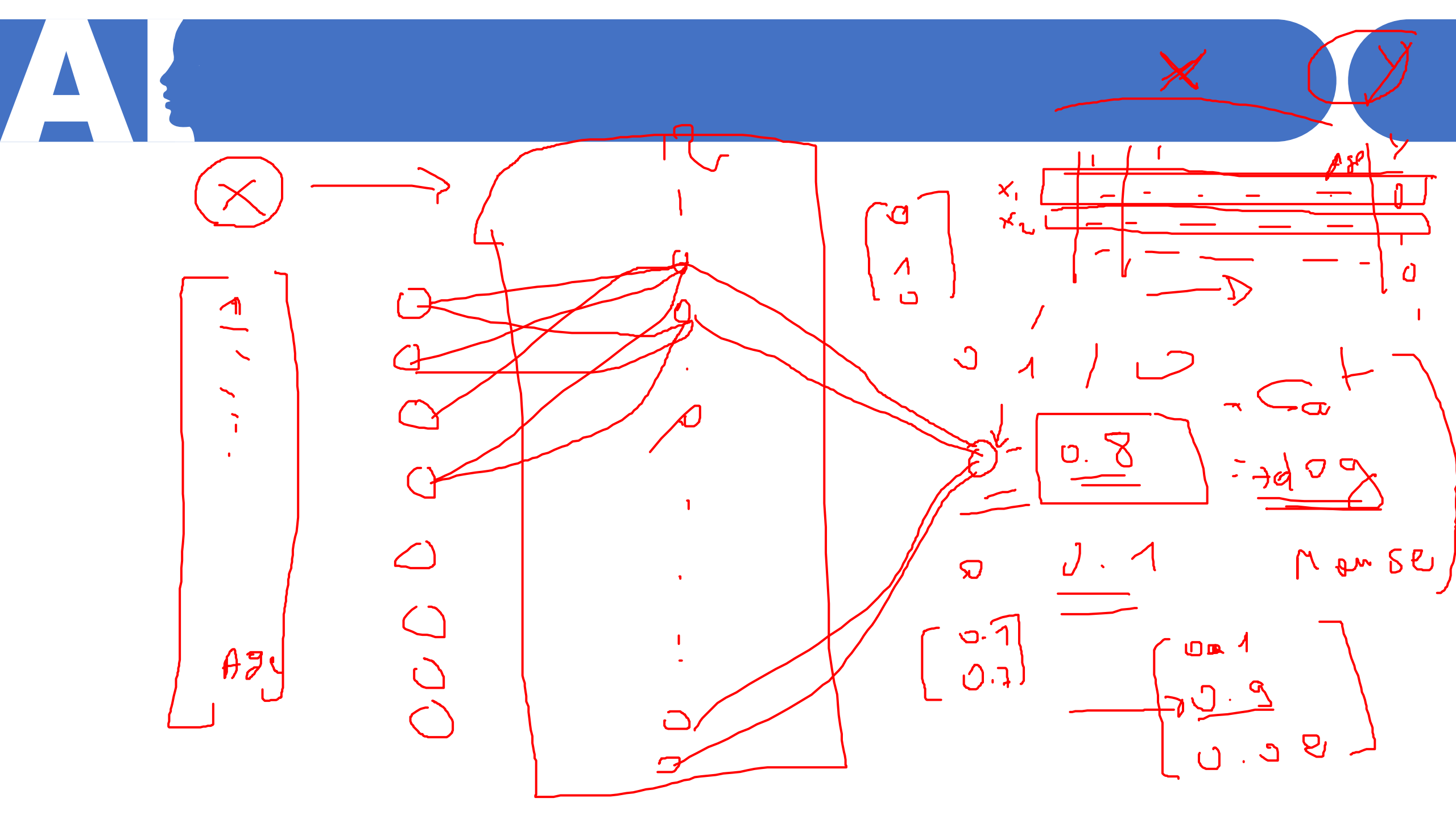
Variables d'entrée (X):

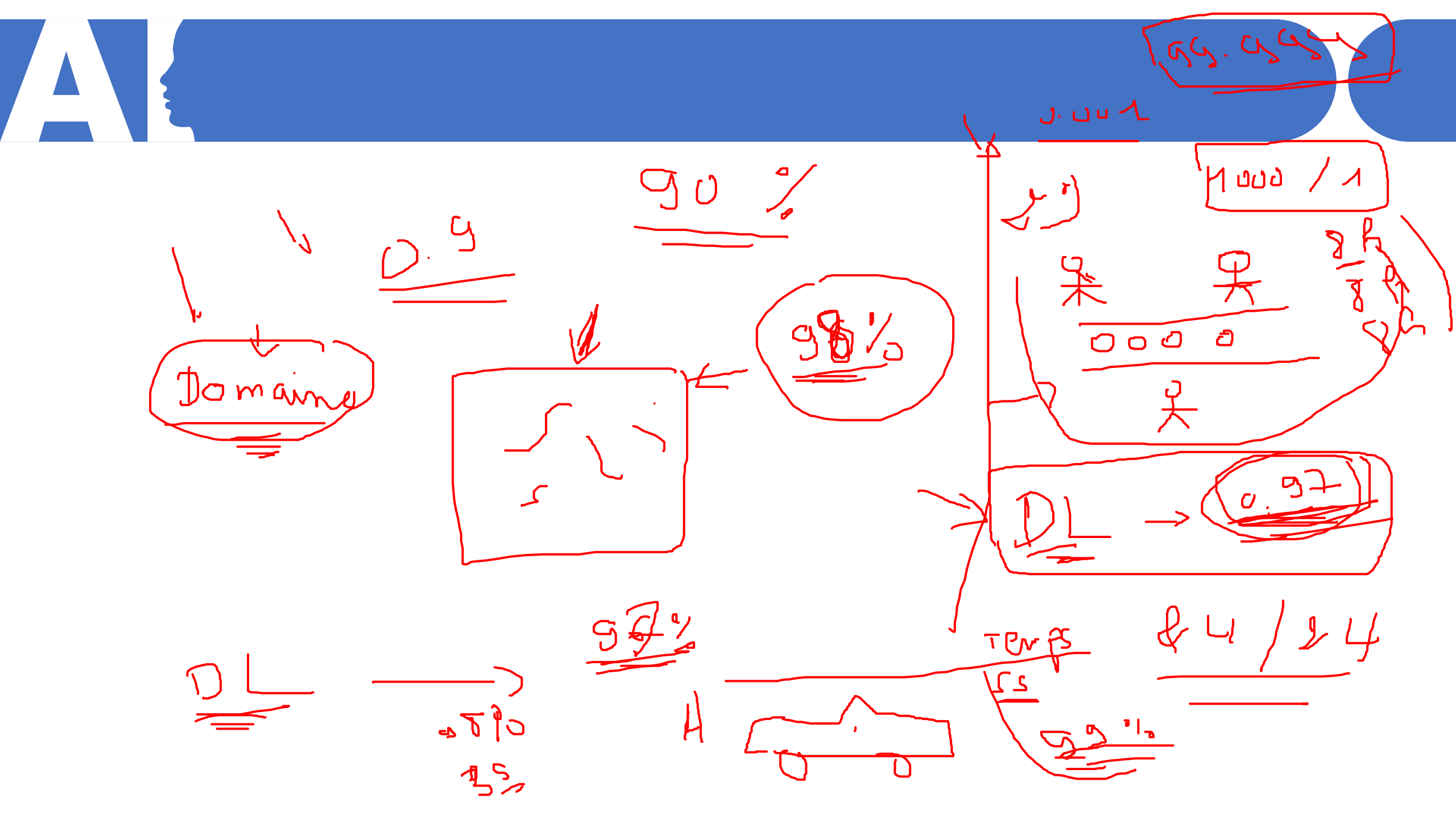
1. Number of times pregnant
2. Plasma glucose concentration a 2 hours in an oral glucose tolerance test
3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index (weight in kg/(height in m)^2)
7. Diabetes pedigree function
8. Age (years)

Variables de sortie (y):

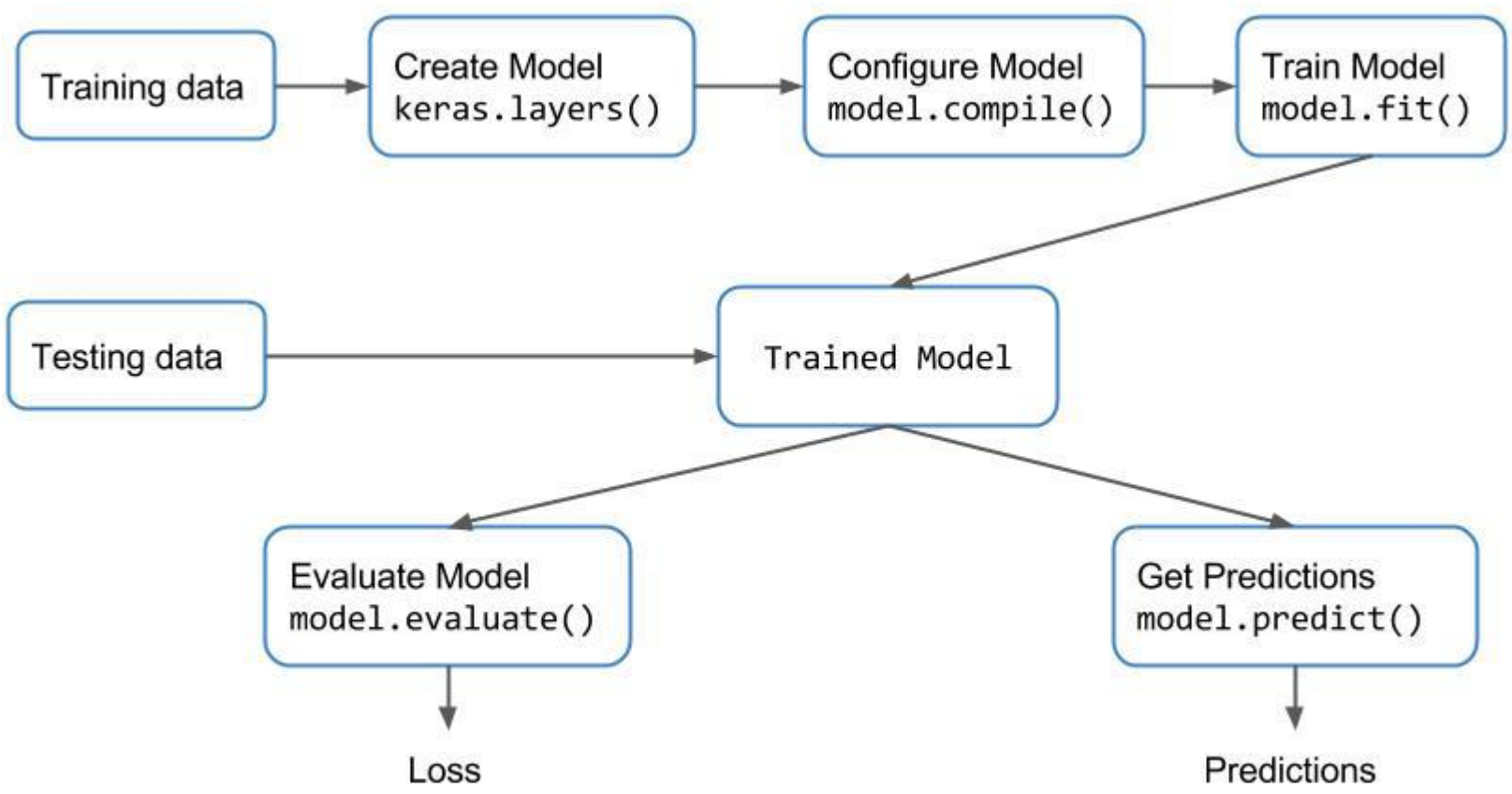
1. Variable de classe (0 ou 1)

https://www.kaggle.com/uciml/pima-indians-diabetes-database




99.99
1.00 1
90 %
0.9
1000 / 1
Domaine
98%
0.97
DL
DL
97%
TEMPS
64 / 64
H

Keras Workflow for training the network



Rappel

Fonction de perte

La fonction de perte est une méthode pour évaluer «dans quelle mesure votre algorithme modélise votre ensemble de données».

Quelle fonction de perte à choisir?

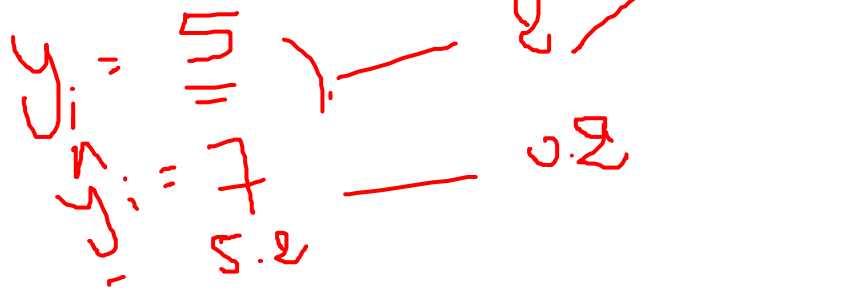
- Il existe divers facteurs impliqués dans le choix d'une fonction de perte pour un problème spécifique, tels que
  - le type d'algorithme d'apprentissage automatique choisi,
  - la facilité de calcul des dérivées
  - et, dans une certaine mesure, le pourcentage de valeurs aberrantes dans l'ensemble de données.

- les fonctions de perte peuvent être classées en deux grandes catégories selon le type de tâche d'apprentissage que nous traitons
  - les pertes de régression
  - les pertes de classification

Erreur quadratique moyenne / perte quadratique / perte L2

$$MSE = \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}$$

- l' erreur quadratique moyenne est mesurée comme la moyenne de la différence quadratique entre les prévisions et les observations réelles.
- Il ne s'agit que de l'ampleur moyenne des erreurs, quelle que soit leur direction.
- Cependant, en raison de la mise au carré, les prévisions qui sont loin des valeurs réelles sont fortement pénalisées par rapport aux prévisions moins déviées.
- De plus, MSE a de belles propriétés mathématiques qui facilitent le calcul des dérivées.



Erreur absolue moyenne / perte L1

$$MAE = \frac{\sum_{i=1}^{n} | y_i - \hat{y}_i |}{n}$$

- L'erreur absolue moyenne , est mesurée comme la moyenne de la somme des différences absolues entre les prévisions et les observations réelles.
- Comme MSE, cela mesure également l'ampleur de l'erreur sans tenir compte de leur direction.
- Contrairement à MSE, MAE a besoin d'outils plus compliqués tels que la programmation linéaire pour calculer les gradients.
- De plus, MAE est plus robuste aux valeurs aberrantes car il n'utilise pas de carré.

Perte d'entropie croisée / probabilité log négative

$$CrossEntropyLoss = -(y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i))$$

- Il s'agit du paramètre le plus courant pour les problèmes de classification.
- La perte d'entropie croisée augmente lorsque la probabilité prédite diverge de l'étiquette réelle.
  - Notez que lorsque l'étiquette réelle est 1 (y(i) = 1), la seconde moitié de la fonction disparaît alors que dans le cas où l'étiquette réelle est 0 (y(i) = 0) la première moitié est supprimée.
- Un aspect important de ceci est que la perte d'entropie croisée pénalise fortement les prédictions qui sont sûres mais erronées .

Entropie croisée

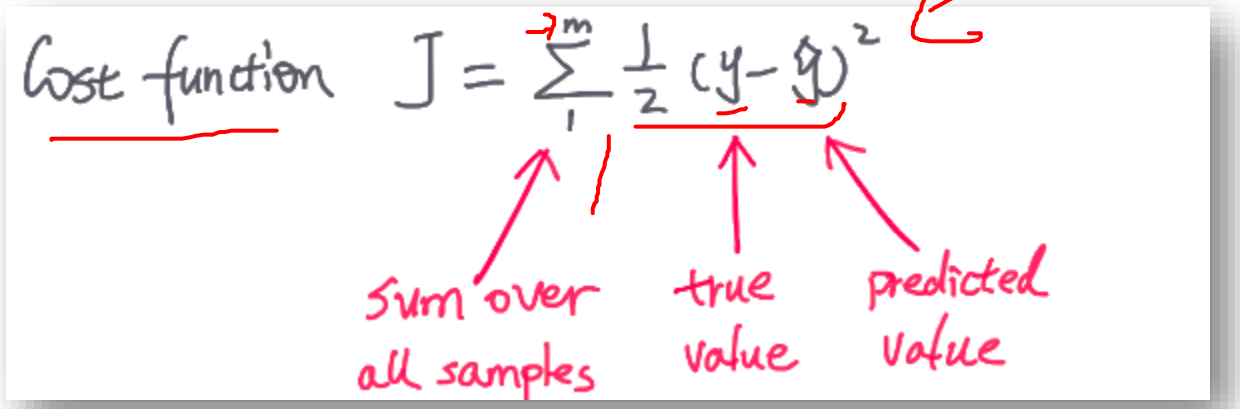
$$\text{Cross entropy} = -\sum_{i}^{M} y_i \log(\hat{y}_i)$$

Where $\hat{y}$ is the predicted value, y is the true value and M is the number of classes

- L' entropie croisée quantifie la différence entre deux distributions de probabilité.
- Si notre modèle prédit une distribution de modèle de {p1, p2, p3} (où p1 + p2 + p3 = 1). Nous utilisons l'entropie croisée pour comparer cela avec la vraie distribution {y1, y2, y3}

FONCTION DE COÛT

- L'erreur de perte est calculée pour un seul exemple d'apprentissage.
- Si nous avons un nombre «m» d'exemples, la moyenne de la fonction de perte de l'ensemble d'apprentissage entier est appelée «fonction de coût».



- C'est une fonction qui mesure les performances d'un modèle d'apprentissage automatique.

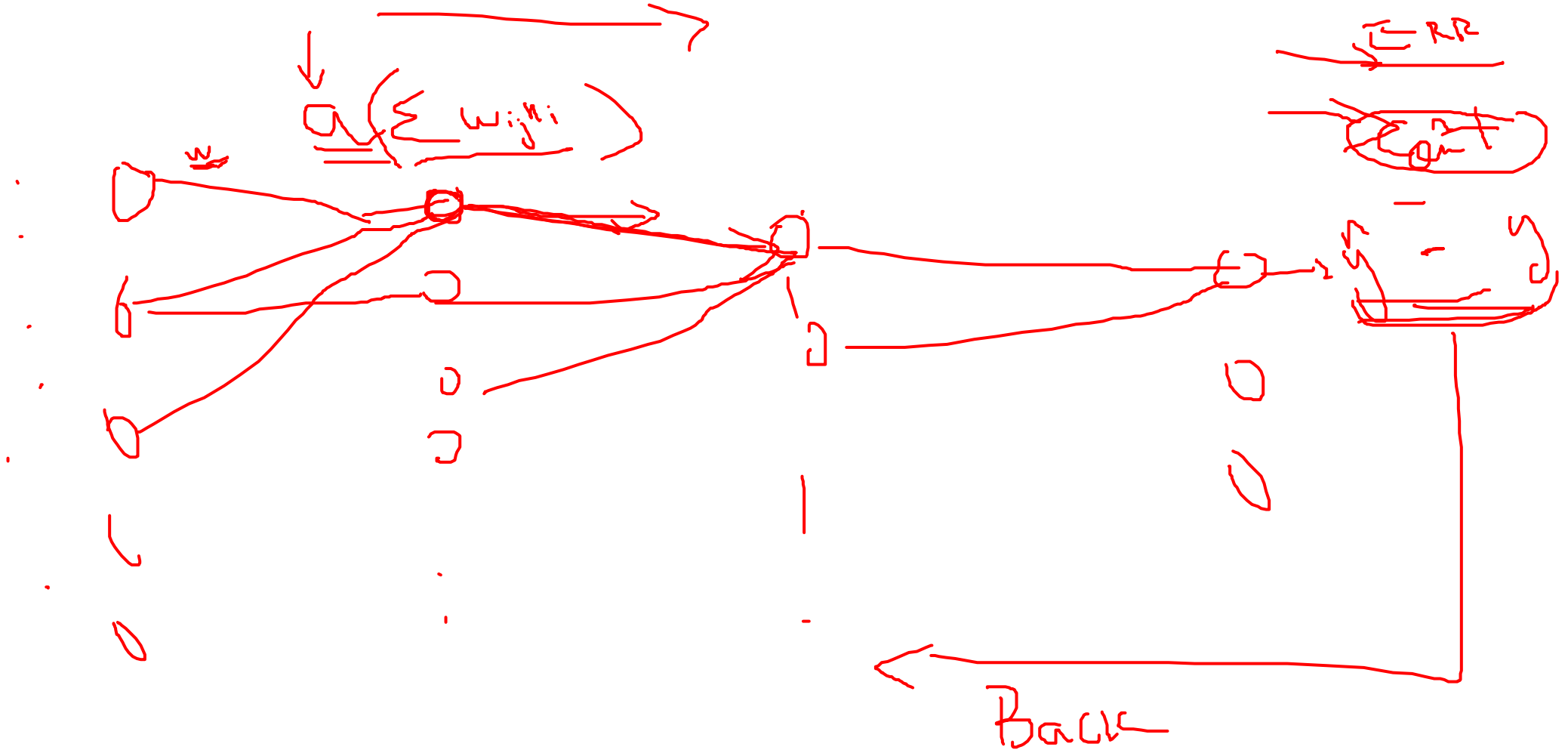
FONCTION DE COÛT

- La fonction de coût quantifie l'erreur entre les valeurs prédites et les valeurs attendues et la présente sous la forme d'un seul nombre réel .
- Selon le problème, la fonction de coût peut être formée de nombreuses manières différentes. L'objectif de la fonction de coût est soit:
  - Minimisé - la valeur renvoyée est généralement appelée coût , perte ou erreur . Le but est de trouver les valeurs des paramètres du modèle pour lesquels la fonction de coût renvoie le plus petit nombre possible.
  - Maximisé - alors la valeur qu'il rapporte est appelée récompense . Le but est de trouver des valeurs de paramètres de modèle pour lesquels le nombre renvoyé est aussi grand que possible.

Autres fonctions

- Exponentional cost
- Hellinger distance
- Kullback–Leibler divergence
- Generalized Kullback–Leibler divergence
- Itakura–Saito distance

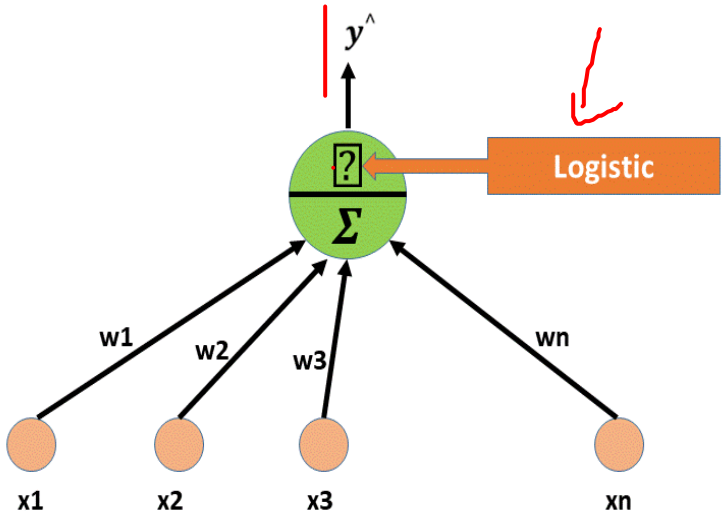
$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \sum_j \left(t(\mathbf{x}_j) - \sum_i w_i h_i(\mathbf{x}_j) \right)^2 + \lambda \sum_{i=1}^{k} |w_i|$$


Wifi
Back

Fonction d'activation

- Une fonction d'activation est une fonction non linéaire appliquée par un neurone pour introduire des propriétés non linéaires dans le réseau.

- Une fonction d'activation est une caractéristique très importante d'un réseau de neurones artificiels, ils décident fondamentalement si le neurone doit être activé ou non.
- Dans les réseaux de neurones artificiels , la fonction d'activation définit la sortie de ce nœud en fonction d'une entrée ou d'un ensemble d'entrées.



Common Activation Functions

1. **The sigmoid function** has a smooth gradient and outputs values between zero and one. For very high or low values of the input parameters, the network can be very slow to reach a prediction, called the *vanishing gradient* problem.
2. **The TanH function** is zero-centered making it easier to model inputs that are strongly negative strongly positive or neutral.
3. **The ReLu function** is highly computationally efficient but is not able to process inputs that approach zero or negative.
4. **The Leaky ReLu** function has a small positive slope in its negative area, enabling it to process zero or negative values.
5. **The Parametric ReLu** function allows the negative slope to be learned, performing backpropagation to learn the most effective slope for zero and negative input values.
6. **Softmax** is a special activation function use for output neurons. It normalizes outputs for each class between 0 and 1, and returns the probability that the input belongs to a specific class.
7. **Swish** is a new activation function discovered by Google researchers. It performs better than ReLu with a similar level of computational efficiency.

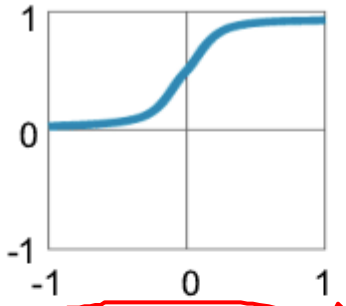
Types de fonction d'activation

- Types de fonction d'activation:
  - Fonction linéaire
  - Fonction pas binaire
  - Fonction non linéaire

- Propriétés que la fonction d'activation doit posséder?
- Dérivée ou différentielle: changement de l'axe y par rapport au changement de l'axe x, également connu sous le nom de pente. (Back prop)
- Fonction monotone: une fonction qui est soit entièrement non croissante, soit non décroissante.

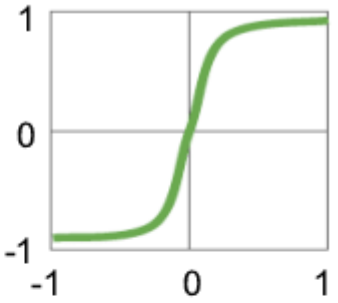
Traditional Non-Linear Activation Functions

Sigmoid



$y=1/(1+e^{-x})$

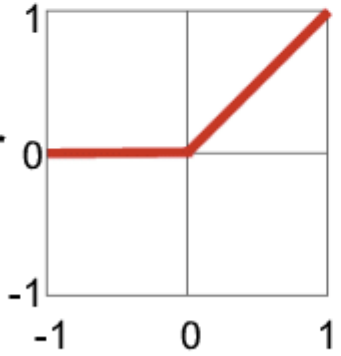
Hyperbolic Tangent



$y=(e^{x}-e^{-x})/(e^{x}+e^{-x})$

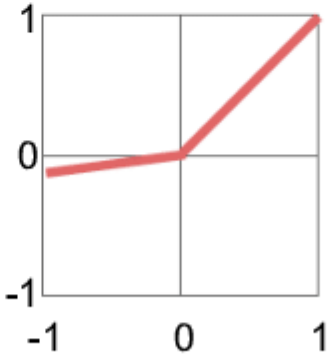
Modern Non-Linear Activation Functions

Rectified Linear Unit (ReLU)



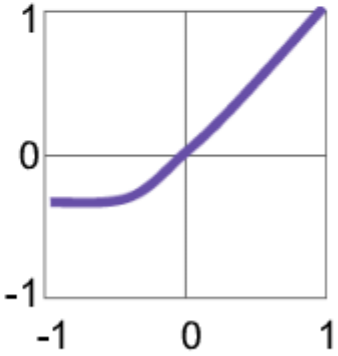
$y=\max(0,x)$

Leaky ReLU



$y=\max(\alpha x,x)$

Exponential LU



$y=\begin{cases} x, & x\geq 0 \\ \alpha(e^{x}-1), & x<0 \end{cases}$

α = small const. (e.g. 0.1)

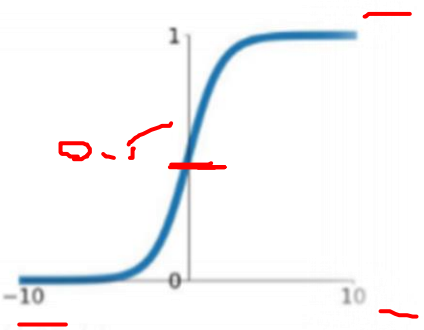
Fonction d'activation

- La plupart des réseaux de neurones modernes utilisent la fonction non linéaire comme fonction d'activation pour déclencher le neurone.
- La raison en est qu'ils permettent au modèle de créer des mappages complexes entre les entrées et les sorties du réseau, qui sont essentielles pour l'apprentissage et la modélisation de données complexes,
  - telles que des images, de la vidéo, de l'audio et des ensembles de données qui sont non linéaires ou ont une dimensionnalité élevée.
- Les différentiels sont possibles dans toutes les fonctions non linéaires.
- L'empilement du réseau est possible, ce qui nous aide à créer les réseaux neuronaux profonds.
- Les fonctions d'activation non linéaires sont principalement divisées en fonction de leur plage ou de leurs courbes.

Sigmoïde / Logistique

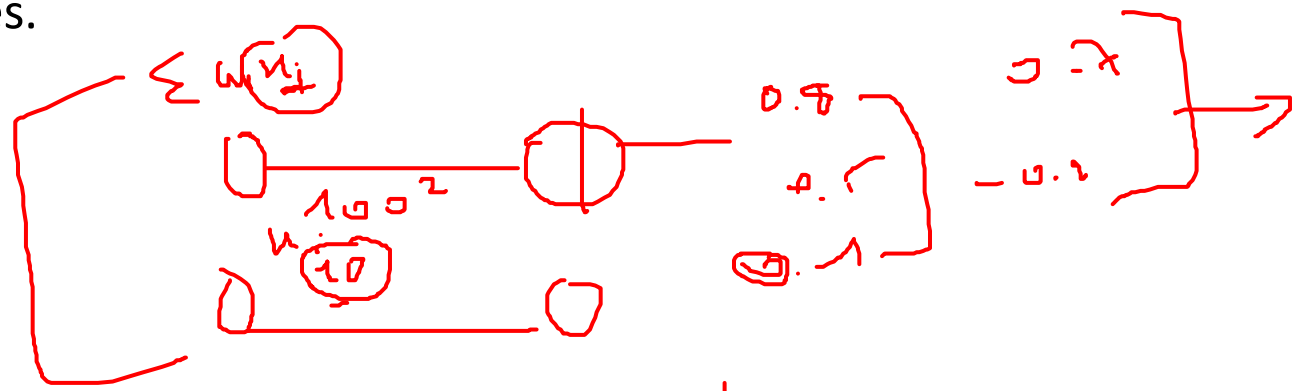
- Avantages
  - Gradient lisse , empêchant les «sauts» dans les valeurs de sortie.
  - Les valeurs de sortie sont liées entre 0 et 1, normalisant la sortie de chaque neurone.
  - Prédictions claires - Pour X supérieur à 2 ou inférieur à -2, tend à amener la valeur Y (la prédiction) au bord de la courbe, très proche de 1 ou 0. Cela permet des prédictions claires.
- Désavantages
  - Gradient de fuite - pour des valeurs de X très élevées ou très faibles, il n'y a pratiquement pas de changement dans la prédiction, ce qui entraîne un problème de gradient de fuite. Cela peut conduire le réseau à refuser d'apprendre davantage ou à être trop lent pour atteindre une prédiction précise.
  - Sorties non centrées sur zéro .
  - Calcul coûteux

Function	Equation	Range	Derivative
Sigmoid (Logistic)	$f(x) = \frac{1}{1+e^{-x}}$	0,1	$f'(x) = f(x)(1-f(x))$

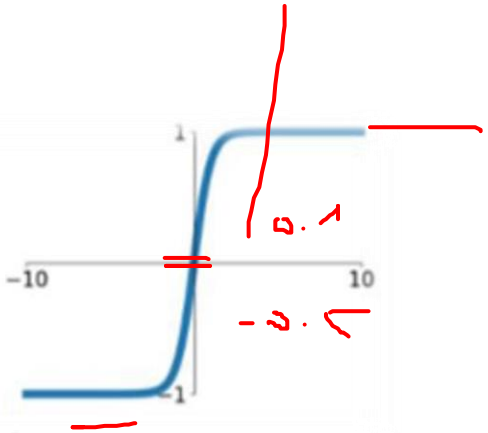


Tanh

- Avantages
  - Centré sur zéro - ce qui facilite la modélisation des entrées qui ont des valeurs fortement négatives, neutres et fortement positives.
  - Sinon, comme la fonction Sigmoid.
- Désavantages
  - Comme la fonction sigmoïde



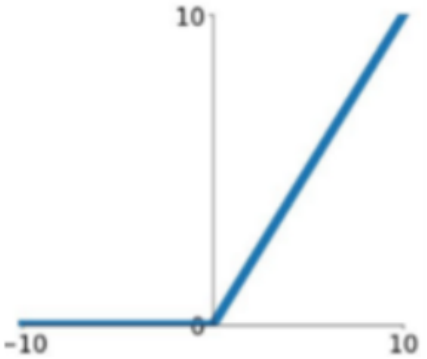
Function	Equation	Range	Derivative
Tanh (Hyperbolic tangent)	$f(x) = \frac{2}{1+e^{-2x}} - 1$	-1,1	$f'(x) = 1 - f(x)^2$



ReLU (unité linéaire rectifiée)

- Avantages
  - Efficace en termes de calcul - permet au réseau de converger très rapidement
  - Non linéaire - bien que cela ressemble à une fonction linéaire, ReLU a une fonction dérivée et permet une rétropropagation
- Désavantages
  - Le problème Dying ReLU - lorsque les entrées s'approchent de zéro, ou sont négatives, le gradient de la fonction devient nul, le réseau ne peut pas effectuer de rétropropagation et ne peut pas apprendre

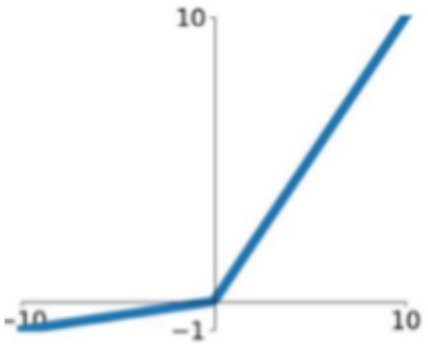
Function	Equation	Range	Derivative
ReLu (Rectified Linear Unit)	$f(x) = \begin{cases} 0 \ for \ x < 0 \\ x \ for \ x \geq 0 \end{cases}$	$0, +\infty$	$f'(x) = \begin{cases} 0 \ for \ x < 0 \\ 1 \ for \ x \geq 0 \end{cases}$



Leaky ReLU

- Advantages
  - **Empêche**le probléme de dying ReLU — cette variation de ReLU a une petite pente positive dans la zone négative, elle permet donc la rétropropagation, même pour des valeurs d'entrée négatives
  - Otherwise like ReLU
- Disadvantages
  - **Résultats non cohérents**— leaky ReLU ne fournit pas de prévisions cohérentes pour les valeurs d'entrée négatives.

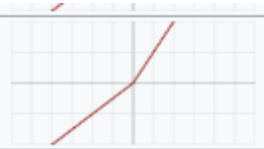
Function	Equation	Range	Derivative
Leaky ReLu	$f(x) = \begin{cases} \alpha x \ for \ x < 0 \\ x \ \ for \ x \geq 0 \end{cases}$	$-\infty, +\infty$	$f'(x) = \begin{cases} \alpha \ for \ x < 0 \\ 1 \ for \ x \geq 0 \end{cases}$



α =0.01

ReLU paramétrique

- Avantages
  - Permet d'apprendre la pente négative - contrairement à Laky ReLU, cette fonction fournit la pente de la partie négative de la fonction comme argument. Il est donc possible d'effectuer une rétropropagation et d'apprendre la valeur la plus appropriée de α.
  - Sinon comme ReLU
- Désavantages
  - Peut fonctionner différemment pour différents problèmes.

Parameteric rectified linear unit (PReLU)[20]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$[2]

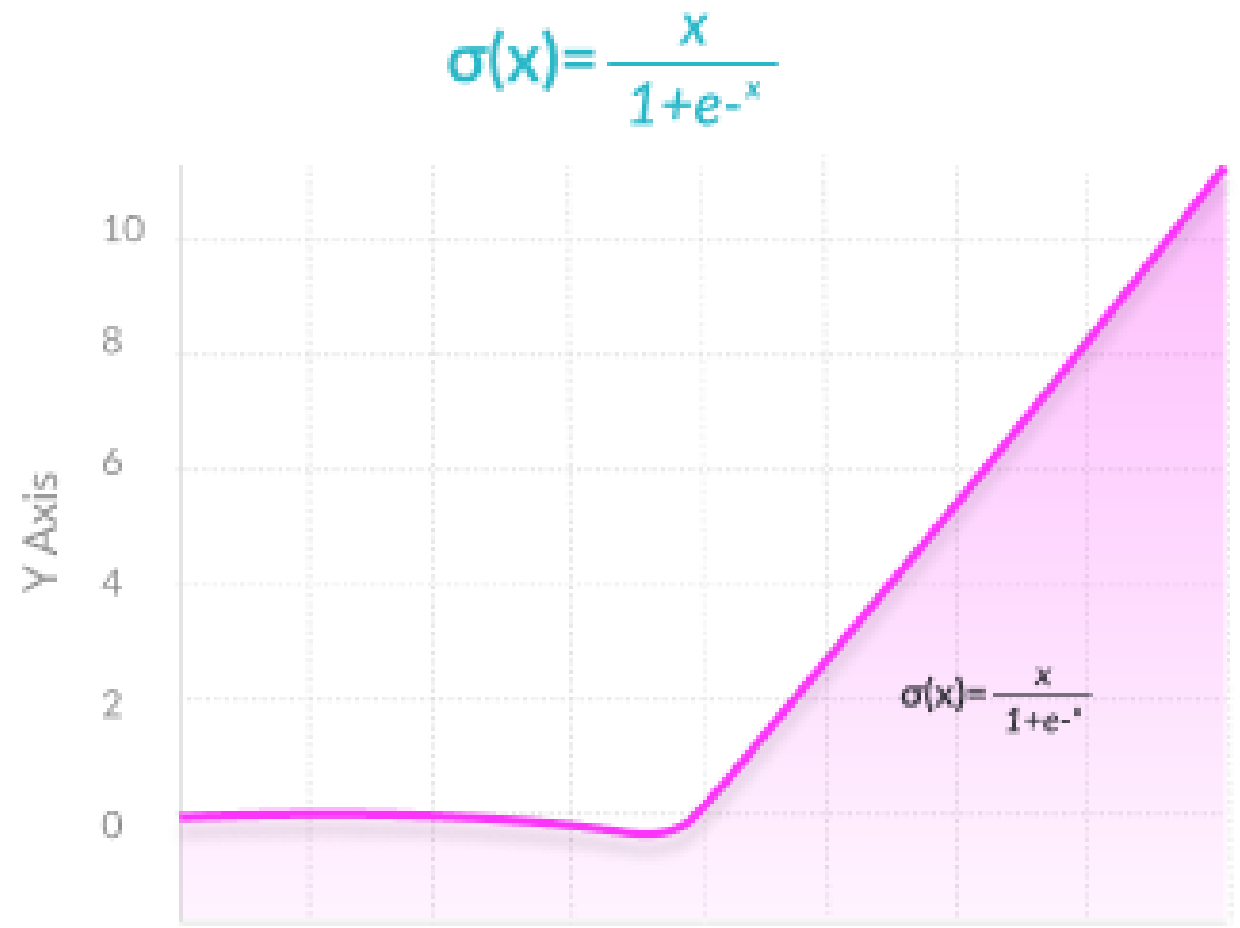
Softmax

- Avantages
  - Capable de gérer plusieurs classes une seule classe dans d'autres fonctions d'activation - normalise les sorties de chaque classe entre 0 et 1, et divise par leur somme, donnant la probabilité que la valeur d'entrée soit dans une classe spécifique.
  - Utile pour les neurones de sortie - généralement Softmax est utilisé uniquement pour la couche de sortie, pour les réseaux de neurones qui doivent classer les entrées en plusieurs catégories.

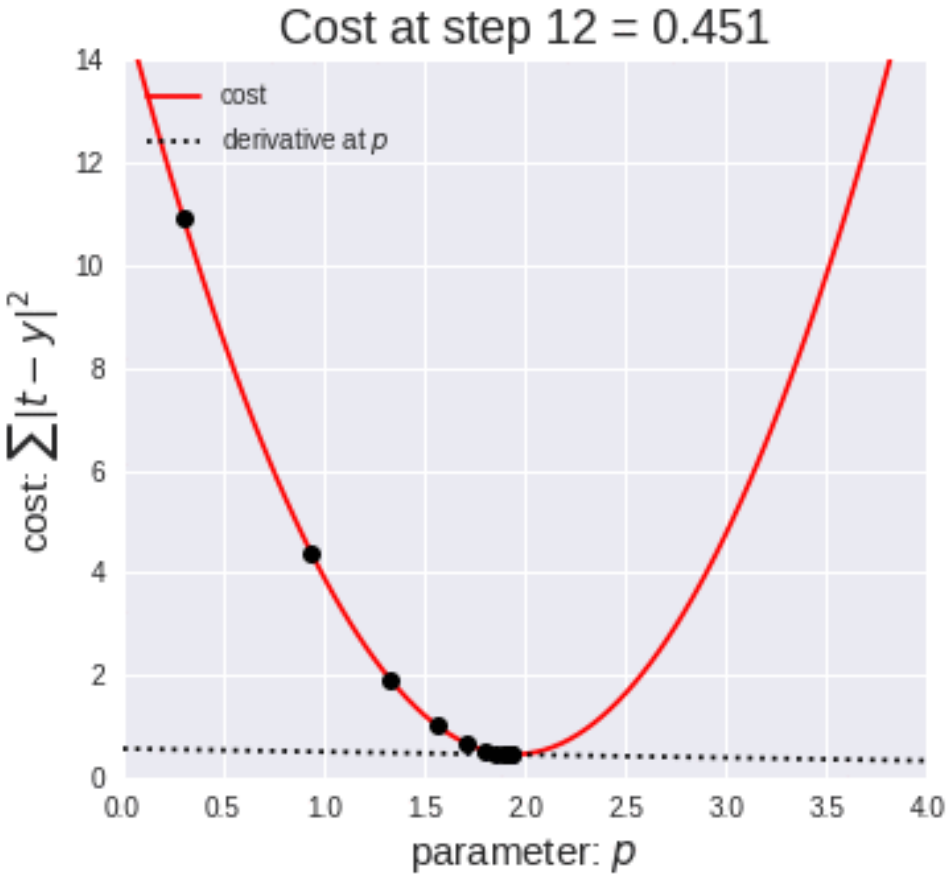
Name	Equation	Derivatives	Range
Softmax	$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^{J} e^{x_j}}$ for $i = 1, \ldots, J$	$\frac{\partial f_i(\vec{x})}{\partial x_j} = f_i(\vec{x})(\delta_{ij} - f_j(\vec{x}))$[5]	$(0, 1)$

Fonction d'activation Swish (A Self-Gated)

https://arxiv.org/pdf/1710.05941v1.pdf



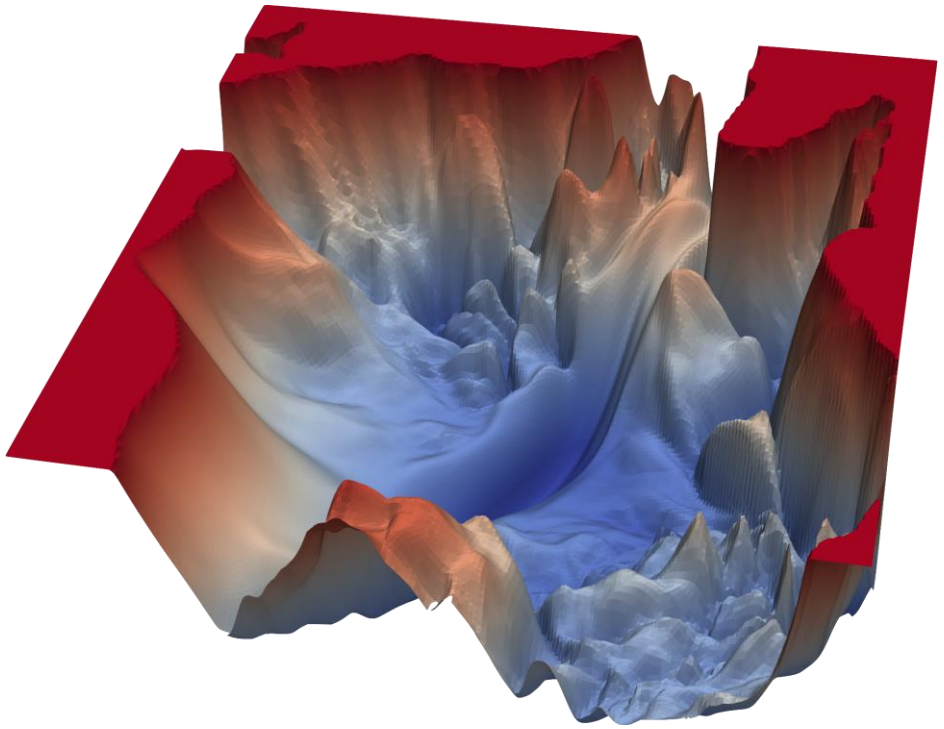
Gradient descent



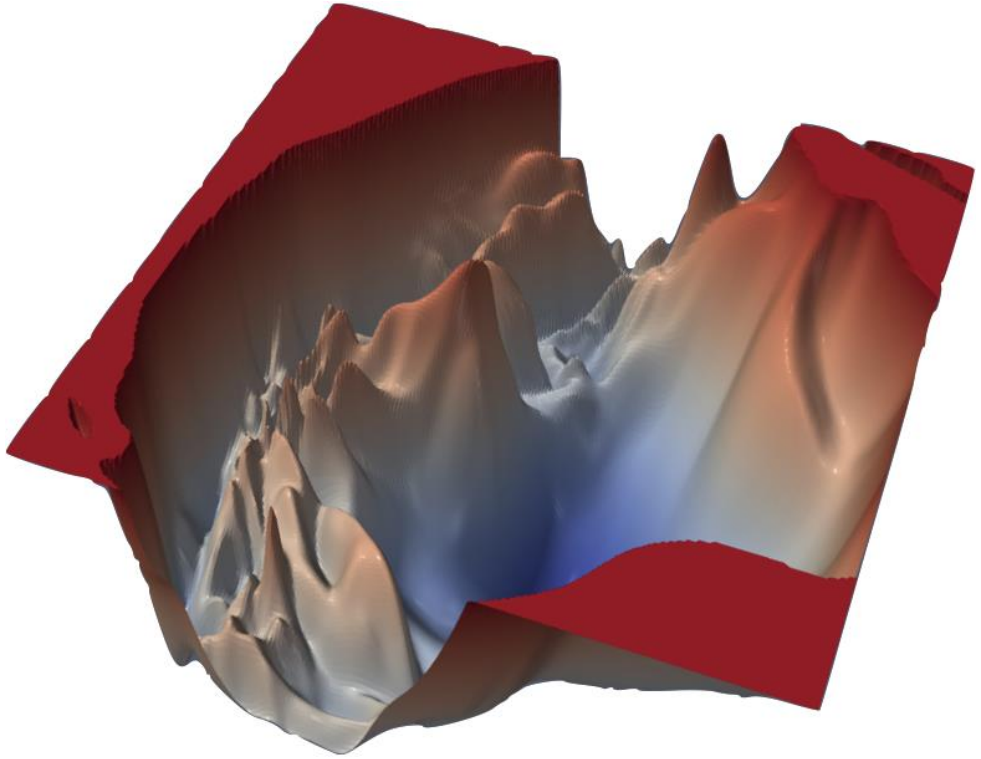
$$w \leftarrow w - \alpha \frac{\partial E}{\partial w}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\partial C}{\partial \mathbf{w}}$$

Exemple avec DL



VGG-56



VGG-110

https://www.cs.umd.edu/~tomg/projects/landscapes/

Matrice de confusion

		Classe **prédite** +	Classe **prédite** -
Classe **vraie**	+	**TP** True Positives	**FN** False Negatives Type II error
	-	**FP** False Positives Type I error	**TN** True Negatives

Indicateurs principaux

Indicateur	Formule	Interprétation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Performance globale du modèle
Précision	$\frac{TP}{TP + FP}$	À quel point les prédictions positives sont précises
Rappel Sensibilité	$\frac{TP}{TP + FN}$	Couverture des observations vraiment positives
Spécificité	$\frac{TN}{TN + FP}$	Couverture des observations vraiment négatives
F-mesure	$\frac{2TP}{2TP + FP + FN}$	Indicateur hybride utilisé pour les classes non-balancées

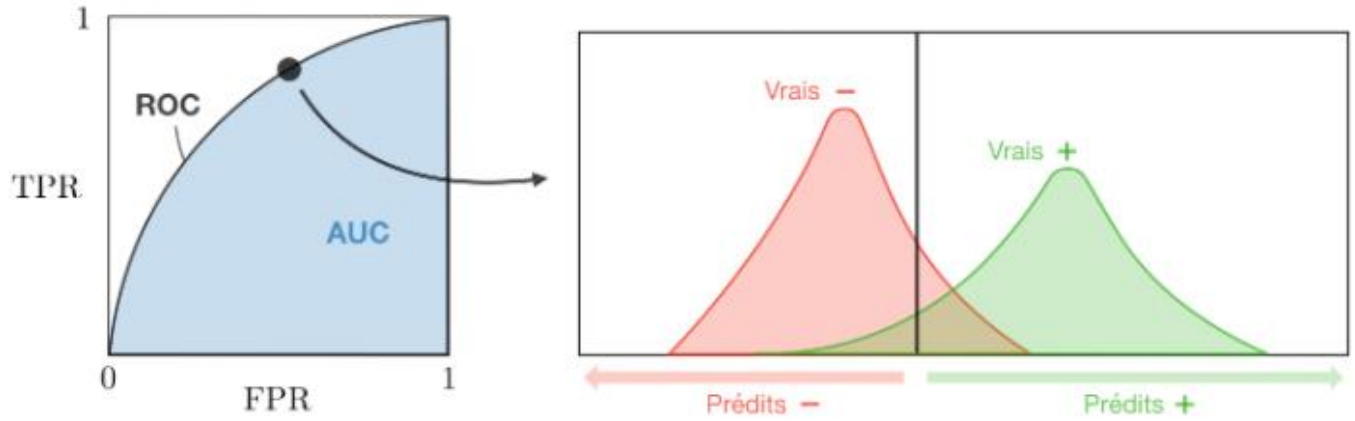
Courbe ROC

La fonction d'efficacité du récepteur, plus fréquemment appelée courbe ROC (de l'anglais *Receiver Operating Curve*), est une courbe représentant le taux de *True Positives* en fonction de taux de *False Positives* et obtenue en faisant varier le seuil. Ces indicateurs sont résumés dans le tableau suivant :

Indicateur	Formule	Equivalent
True Positive Rate TPR	$\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$	Recall, sensitivity
False Positive Rate FPR	$\frac{\mathrm{FP}}{\mathrm{TN}+\mathrm{FP}}$	1-specificity

AUC

L'aire sous la courbe ROC, aussi notée AUC (de l'anglais *Area Under the Curve*) ou AUROC (de l'anglais *Area Under the ROC*), est l'aire sous la courbe ROC comme le montre la figure suivante :



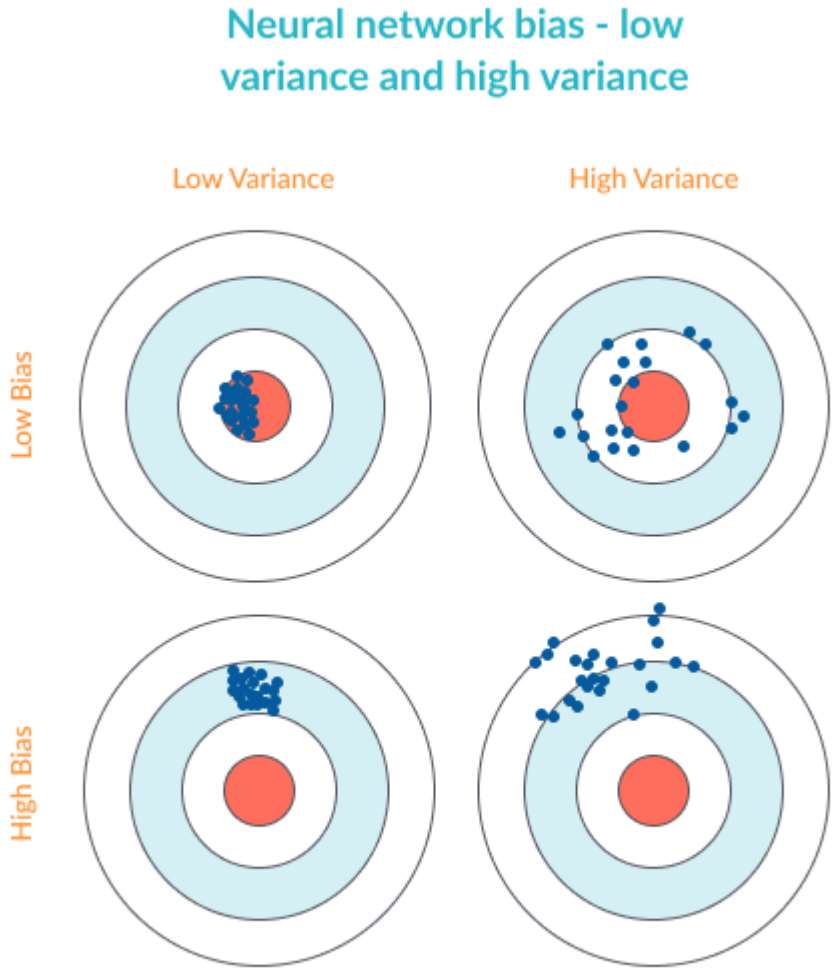
L'AUC fournit une mesure agrégée des performances pour tous les seuils de classification possibles. On peut interpréter l'AUC comme une mesure de la probabilité pour que le modèle classe un exemple positif aléatoire au-dessus d'un exemple négatif aléatoire.

- L'AUC est **invariante d'échelle**. Elle mesure la qualité du classement des prédictions, plutôt que leurs valeurs absolues.
- L'AUC est **indépendante des seuils de classification**. Elle mesure la qualité des précisions du modèle quel que soit le seuil de classification sélectionné.

Bias / Variance

- Bias: expected difference between model's prediction and truth
- Variance: how much the model differs among training sets

- Model Scenarios
  - High Bias: Model makes inaccurate predictions on training data
  - High Variance: Model does not generalize to new datasets
  - Low Bias: Model makes accurate predictions on training data
  - Low Variance: Model generalizes to new datasets



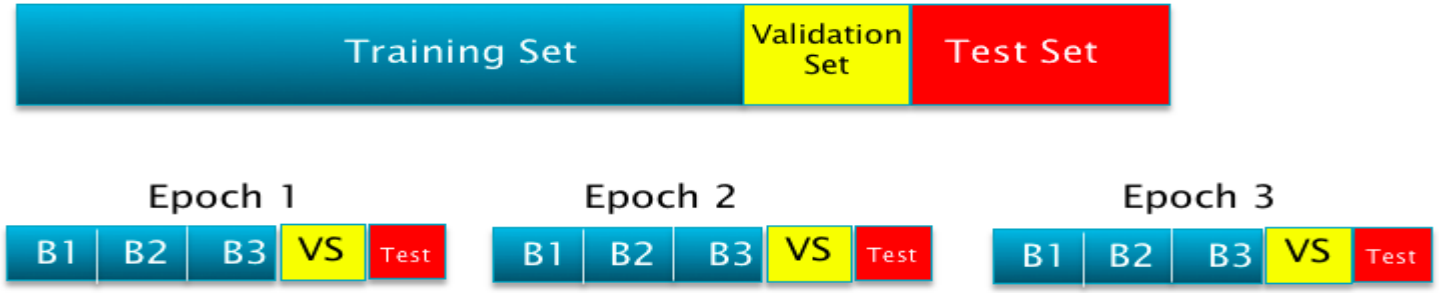
Bias

- In artificial neural networks, the word bias has two meanings:
  - It can mean a *bias neuron*, which is part of the structure of the neural network
  - It can mean *bias* as a statistical concept, which reflects how well the network is able to generate predictions based on the training samples you provide.

Bias

- When training neural networks, like in other machine learning techniques, we try to balance between bias and variance.
- Bias measures how well the model fits the training set—able to correctly predict the known outputs of the training examples.
- Variance measures how well the model works with unknown inputs that were not available during training.
- Another meaning of bias is a "bias neuron" which is used in every layer of the neural network. The bias neuron holds the number 1, and makes it possible to move the activation function up, down, left and right on the number graph.

Validation set



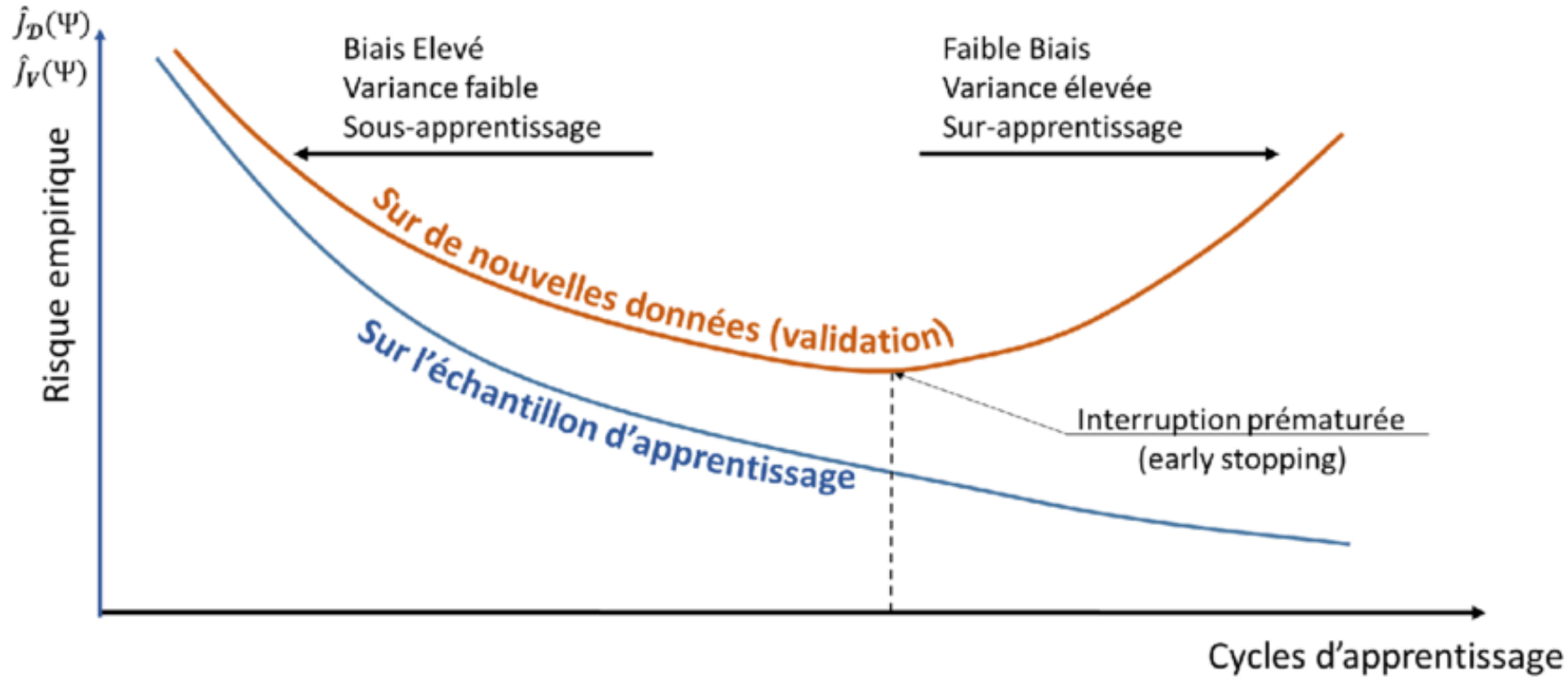
suivre la variation du taux d'erreur ou de la précision de la classification dans ces expériences à l'aide de l'ensemble de données de test, mais ce n'est cependant pas une bonne idée.

La raison en est que cela entraîne une fuite d'informations sur les données de test dans le processus d'apprentissage. Par conséquent, nous pouvons finir par choisir des hyper-paramètres qui conviennent bien à un choix particulier de données de test, mais qui ne fonctionneront pas bien pour d'autres.

L'utilisation de l'ensemble de données de validation garantit que cette fuite ne se produira pas.

Validation set : environ 10% des données disponible. Ce jeu sera appelé une seule fois, à la fin de chaque itération d'entrainement. Il va permettre d'équilibrer le système. C'est la phase d'ajustage.

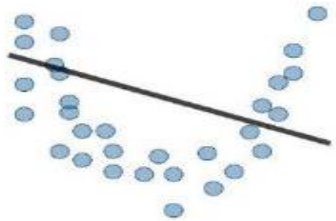
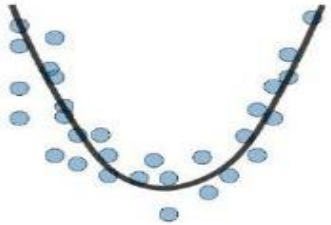
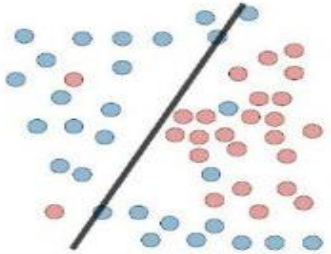
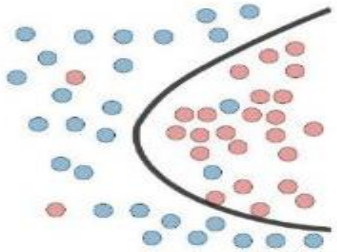
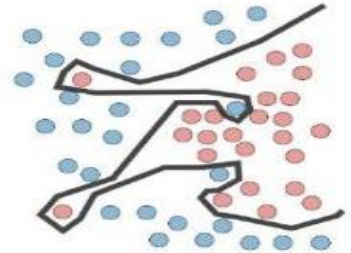
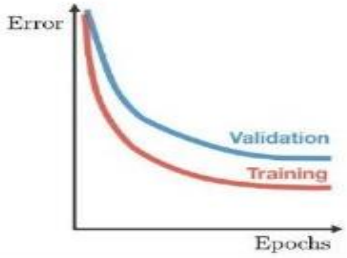
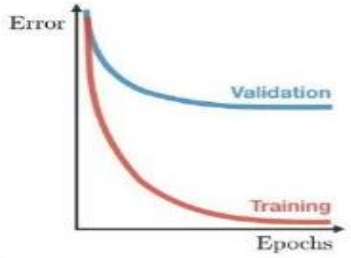
Overfitting & Underfitting



Overfitting & Underfitting

Compromis biais/variance

Plus le modèle est simple, plus le biais est grand
et plus le modèle est complexe, plus la variance est grande.

	Underfitting	Just right	Overfitting
Symptoms	- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
Regression			
Classification			
Deep learning			
Remedies	- Complexify model - Add more features - Train longer		- Regularize - Get more data

https://www.kaggle.com/getting-started/183376

Sur-ajustement

Les facteurs suivants déterminent la capacité d'un modèle à se généraliser du jeu de données d'entraînement au jeu de données de test:

- La capacité du modèle et sa relation avec la complexité des données: en général, si la capacité du modèle est inférieure à la complexité des données, cela conduit à un sous-ajustement, tandis que si l'inverse est vrai, cela peut conduire à un sur-ajustement. Par conséquent, nous devrions essayer de choisir un modèle dont la capacité correspond à la complexité des ensembles de données d'entraînement et de test.

- Même si la capacité du modèle et la complexité des données correspondent bien, nous pouvons toujours rencontrer le problème de surajustement. Cela est dû à une quantité insuffisante de données d'entraînement.

Sous ajustement

Afin de remédier le sous-ajustement , le concepteur peut augmenter la capacité du modèle en augmentant le nombre de couches cachées, en ajoutant plus de nœuds par couche cachée, en modifiant les paramètres de régularisation ou le taux d'apprentissage.
Si ces étapes ne permettent pas de résoudre le problème, cela indique que les données d'entraînement sont de mauvaise qualité.

La régularisation

- Il existe une grande variété de techniques utilisées pour la régularisation et, en général, la seule caractéristique qui les unit est que ces techniques réduisent la capacité effective d'un modèle, c'est-à-dire la capacité du modèle à gérer des tâches de classification plus complexes.
- Cela a du sens puisque la cause fondamentale du sur-ajustement est que la capacité du modèle dépasse les exigences du problème.

- Lors de l'utilisation de modèles DLN, il est judicieux de commencer avec un modèle plus complexe que le problème peut justifier, puis d'ajouter des techniques de régularisation si un sur-ajustement est détecté.

Régularisation

Certaines techniques de régularisation couramment utilisées comprennent:

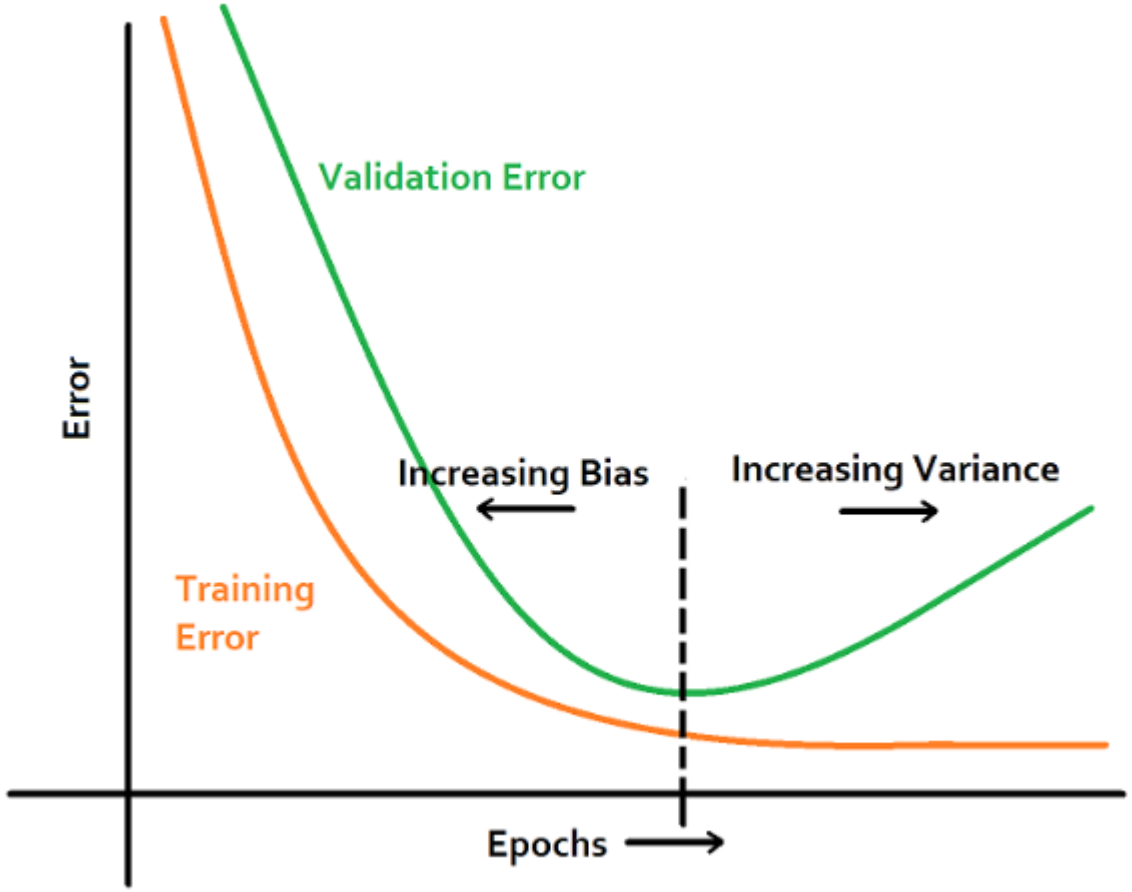
- ❑ Early Stopping
- ❑ L1 Regularization
- ❑ L2 Regularization
- ❑ Dropout Regularization
- ❑ Training Data Augmentation
- ❑ Batch Normalization

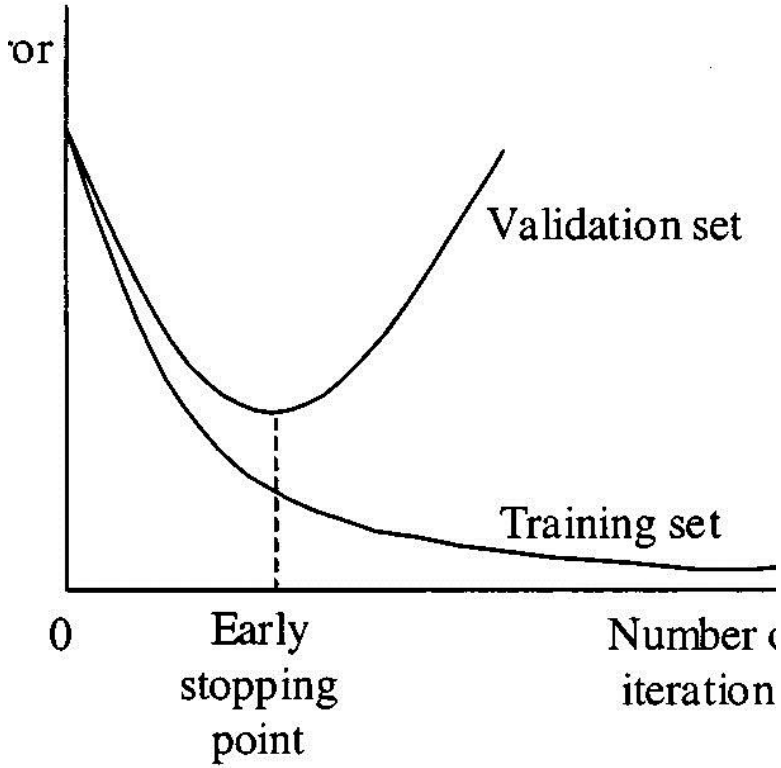
Les trois premières techniques sont bien connues depuis les jours du Machine Learning et continuent d'être utilisées pour les modèles DLN. Les trois dernières techniques, en revanche, ont été spécialement conçues pour les DLN et ont été découvertes ces dernières années.

Early Stopping

- L'arrêt précoce est l'une des techniques les plus populaires et les plus efficaces pour éviter le surajustement. L'idée de base est simple: Utilisez l'ensemble de données de validation pour calculer la fonction de perte à la fin de chaque période d'apprentissage, et une fois que la perte cesse de diminuer, arrêtez la formation et utilisez les données de test pour calculer la précision de classification finale.
- En pratique, il est plus robuste d'attendre que la perte de validation cesse de diminuer pendant quatre ou cinq époques successives avant de s'arrêter.

Early Stopping





Régularisation L1 et Régularisation L2

L1 Regularization

$$\text{Cost} = \sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^{M} W_j^2}_{\text{Regularization Term}}$$

Régularisation L2

La régularisation L2 est une technique couramment utilisée dans les systèmes ML et est également parfois appelée «perte de poids».

Cela fonctionne en ajoutant un terme quadratique à la fonction de perte C , appelé le terme de régularisation, qui se traduit par une nouvelle fonction de perte donné par:

$$\text{Cost} = \sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2 + \lambda \sum_{j=0}^{M} W_j^2$$

Le terme de régularisation se compose de la somme des carrés de tous les poids de lien dans le modèle, multipliée par un paramètre λ appelé le paramètre de régularisation.

La régularisation L1

La régularisation L1 utilise une fonction de régularisation qui est la somme de la valeur absolue de tous les poids dans le modèle, ce qui donne la fonction de perte suivante:

$$\text{Cost} = \sum_{i=0}^{N}(y_i - \sum_{j=0}^{M} x_{ij}W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

$$\begin{aligned} w_{\text{new}} &= w - \eta \frac{\partial L_1}{\partial w} \\ &= w - \eta \cdot \left[2x(wx + b - y) + \lambda \frac{d|w|}{dw}\right] \\ &= \begin{cases} w - \eta \cdot \left[2x(wx + b - y) + \lambda\right] & w > 0 \\ w - \eta \cdot \left[2x(wx + b - y) - \lambda\right] & w < 0 \end{cases} \end{aligned}$$

Weight Regularization API in Keras

Three different regularizer instances are provided; they are:

- L1: Sum of the absolute weights.
- L2: Sum of the squared weights.
- L1L2: Sum of the absolute and the squared weights.

```
keras.regularizers.l1(0.01)
keras.regularizers.l2(0.01)
keras.regularizers.l1_l2(l1=0.01, l2=0.01)
```

```
# example of l2 on a dense layer
from keras.layers import Dense
from keras.regularizers import l2
...
model.add(Dense(32, kernel_regularizer=l2(0.01),
bias_regularizer=l2(0.01)))
...
```

```
# example of l2 on a convolutional layer
from keras.layers import Conv2D
from keras.regularizers import l2
...
model.add(Conv2D(32, (3,3), kernel_regularizer=l2(0.01),
bias_regularizer=l2(0.01)))
...
```

Tutos

Dropout

- La co-adaptation est un problème majeur dans l'apprentissage des grands réseaux. Dans un tel réseau, si tous les poids sont appris ensemble, il est courant que certaines des connexions aient plus de capacité prédictive que les autres.

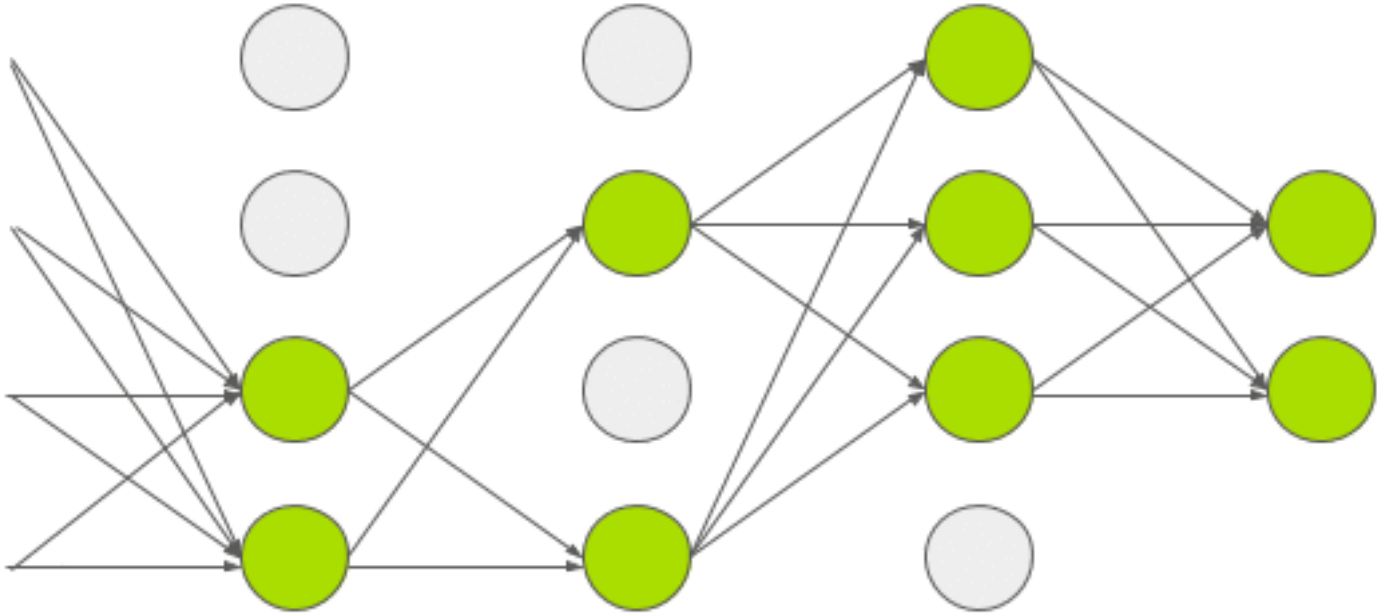
- Dans un tel scénario, au fur et à mesure que le réseau est formé de manière itérative, ces connexions puissantes sont apprises davantage tandis que les plus faibles sont ignorées. Sur de nombreuses itérations, seule une fraction des connexions de nœuds est entraînée. Et les autres arrêtent de participer.

- Ce phénomène est appelé co-adaptation. Cela ne pouvait pas être évité avec la régularisation traditionnelle, comme les L1 et L2. La raison en est qu'ils régularisent également en fonction de la capacité prédictive des connexions.

Dropout

- Le décrochage est l'une des techniques de régularisation les plus efficaces à avoir émergé ces dernières années,
- Forme de **régularisation** utile dans l'apprentissage des **réseaux de neurones**. La régularisation par abandon consiste à supprimer de manière aléatoire un nombre fixe d'unités dans une couche du réseau pour un pas de gradient unique.
- Plus le nombre d'unités abandonnées est élevé, plus la régularisation est solide. Cette méthode est analogue à l'entraînement du modèle pour émuler un groupe exponentiellement large de réseaux plus petits. Pour plus d'informations, consultez l'article Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Dropout



Formalisme

- L'idée de base derrière Dropout est d'exécuter chaque itération de l'algorithme Backprop sur des versions modifiées aléatoirement du DLN d'origine. Les modifications aléatoires sont effectuées sur la topologie du DLN selon les règles suivantes:
  - Attribuer des valeurs de probabilité p(r), 0 ≤ r ≤ R , qui est définie comme la probabilité qu'un nœud soit présent dans le modèle, et utilisez-les pour générer { 0 , 1 } -Valeurs variables aléatoires de Bernoulli e(r)j :
    - Assign probability values $p^{(r)}, 0 \leq r \leq R$, which is defined as the probability that a node is present in the model, and use these to generate $\{0,1\}$-valued Bernoulli random variables $e_j^{(r)}$:

$$e_j^{(r)} \sim Bernoulli(p^{(r)}), \quad 0 \leq r \leq R, \ 1 \leq j \leq P^r$$

    - Modify the input vector as follows:

$$\hat{x}_j = e_j^{(0)} x_j, \quad 1 \leq j \leq N \tag{8.7}$$

    - Modify the activations $z_j^{(r)}$ of the hidden layer r as follows:

$$\hat{z}_j^{(r)} = e_j^{(r)} z_j^{(r)}, \quad 1 \leq r \leq R, \ 1 \leq j \leq P^r \tag{8.8}$$

Augmentation des données d'entraînement



(a) Horizontal Flips



(b) Random Crops and Scales



(c) Randomize Contrast and Brightness

- Translation: Moving the image by a few pixels in various directions
- Rotation
- Reflection
- Skewing
- Scaling: Changing the size of the image while preserving its shape
- Changing contrast or brightness

Batch Normalization

Ioffe and Szegedy, 2015

Batch Normalization

- Le comportement des algorithmes d'apprentissage automatique peut changer lorsque la distribution d'entrée change: si le train et les ensembles de test proviennent de sources entièrement différentes, les distributions seraient différentes.

Batch Normalization

- Dans le contexte du deep learning, nous sommes particulièrement concernés par le changement de la répartition des entrées vers les nœuds internes d'un réseau.
- Un réseau de neurones modifie les poids de chaque couche au cours de la formation. Cela signifie que les activations de chaque couche changent également .
- Puisque les activations d'une couche précédente sont les entrées de la couche suivante, chaque couche du réseau neuronal est confrontée à une situation où la distribution d'entrée change à chaque étape .
- Ceci est problématique car cela oblige chaque couche intermédiaire à s'adapter en permanence à ses entrées changeantes.

Distribution of input

- Having a fixed input distribution is known to help training of linear classifiers
  - Normalize inputs for SVM
  - Normalize inputs for Deep Networks

Distribution of input at each layer

- la distribution des entrées de chaque couche change pendant l'entraînement, à mesure que les paramètres des couches précédentes changent.

- Chaque couche bénéficierait si son entrée avait une distribution constante

Normalize the input to each layer!

- Pour chaque fonctionnalité, la normalisation par lots calcule la moyenne et la variance de cette fonctionnalité dans le mini-lot. Il soustrait ensuite la moyenne et divise l'entité par son écart type du mini-lot. Résultat de la transformation des entrées en moyenne 0 et variance unitaire.

- C’est une technique pour améliorer la vitesse, les performances et la stabilité des réseaux de neurones artificiels. Il est utilisé pour normaliser la couche d'entrée en ajustant et en mettant à l'échelle les activations.

- La normalisation par lots nous permet d'utiliser des taux d'apprentissage beaucoup plus élevés et d'être moins prudents lors de l'initialisation. Il agit également comme un régularisateur, éliminant dans certains cas le besoin d'abandonner.

Normalize the input to each layer!

$$\mu_x = \frac{1}{m}\sum_{i=0}^{m} x_i$$

$$\sigma_x^2 = \frac{1}{m}\sum_{i=0}^{m}(x_i - \mu_x)^2$$

$$\hat{x_i} = \frac{x_i - \mu_x}{\sigma_x}$$

Is this normalization a good idea?

- Considérez les entrées d'une couche sigmoïde
- S'il est normalisé, le sigmoïde peut ne jamais «saturer»

Modify normalization …

- Accommodate identity transform

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

$$\mu_x = \frac{1}{m}\sum_{i=0}^{m} x_i$$

$$\sigma_x^2 = \frac{1}{m}\sum_{i=0}^{m}(x_i - \mu_x)^2$$

$$\hat{x}_i = \frac{x_i - \mu_x}{\sigma_x}$$

$$y_i = \gamma\hat{x}_i + \beta$$

γ, β Learned parameters

Batch Normalization Results

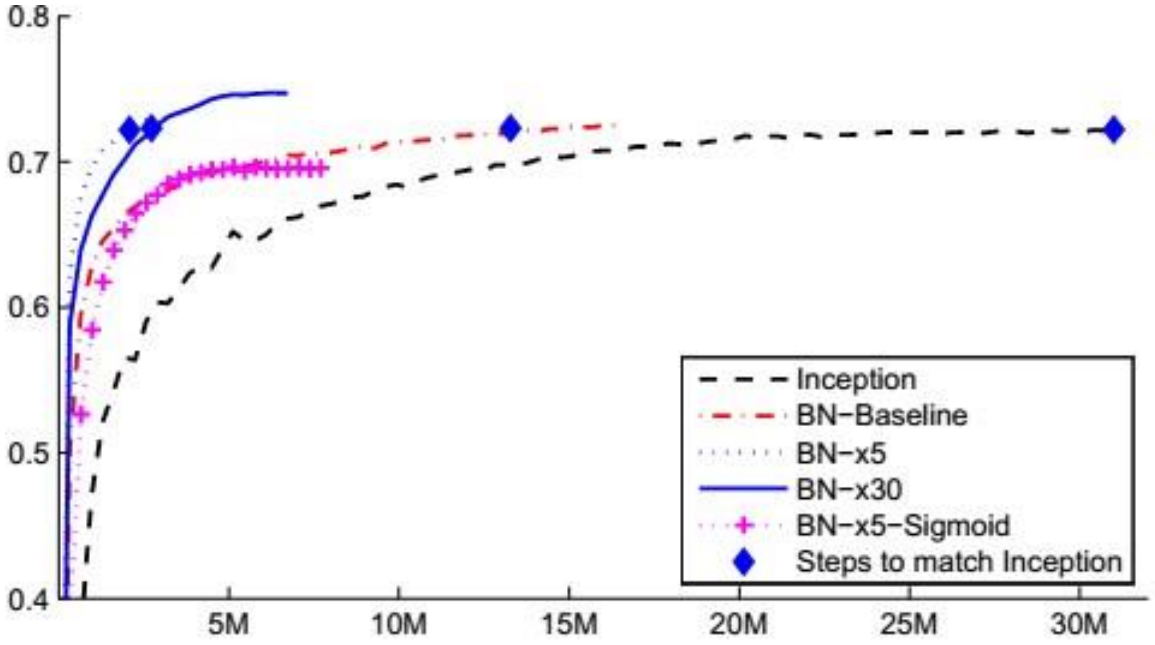


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

Batch Normalization for ensembles

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	**4.9%***

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. *BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*

Methods to Avoid Overfitting

- Here are a few common methods to avoid overfitting in neural networks:
  - **Retraining neural networks**—running the same model on the same training set but with different initial weights, and selecting the network with the best performance.
  - **Multiple neural networks**—training several neural network models in parallel, with the same structure but different weights, and averaging their outputs.
  - **Early stopping**—training the network, monitoring the error on the validation set after each iteration, and stopping training when the network starts to overfit the data.
  - **Regularization**—adding a term to the error function equation, intended to decrease the weights and biases, smooth outputs and make the network less likely to overfit.
  - **Tuning performance ratio**—similar to regularization, but using a parameter that defines by how much the network should be regularized.

Methods to Avoid Underfitting

- Here are a few common methods to avoid underfitting in a neural network:
  - **Adding neuron layers or inputs**—adding neuron layers, or increasing the number of inputs and neurons in each layer, can generate more complex predictions and improve the fit of the model.
  - **Adding more training samples or improving quality**—the more training samples you feed into the network, and the better they represent the variance in the real population, the better the network will perform.
  - **Dropout**—randomly "kill" a certain percentage of neurons in every training iteration. This ensures some information learned is randomly removed, reducing the risk of overfitting.
  - **Decreasing regularization parameter**—regularization can be overdone. By using a regularization performance parameter, you can learn the optimal degree of regularization, which can help the model better fit the data.

Hyperparamètres

- Les **hyperparamètres** sont les variables pouvant être ajustées par les programmeurs dans les algorithmes de Machine Learning.
- La plupart des programmeurs spécialisés dans le Machine Learning consacrent une bonne partie de leur temps à ajuster le taux d'apprentissage. Si vous sélectionnez un taux d'apprentissage trop bas, le temps requis pour l'apprentissage sera trop long.

Exemple

- L’utilisateur doit donc déterminer
  1. 1. les variables d’entrée et la variable de sortie ; leur faire subir comme pour toutes méthodes statistiques, d’éventuelles transformations, normalisations.
  2. 2. L’architecture du réseau : le nombre de couches cachées qui correspond à une aptitude à traiter des problèmes de non-linéarité, le nombre de neurones par couche cachée. Ces deux choix conditionnent directement le nombre de paramètres (de poids) à estimer et donc la complexité du modèle. Ils participent à la recherche d’un bon compromis biais/variance c’est-à-dire à l’équilibre entre qualité d’apprentissage et qualité de prévision.
  3. 3. Trois autres paramètres interviennent également sur ce compromis : le nombre maximum d’itérations, l’erreur maximum tolérée et un terme éventuel de régularisation ridge (decay).
  4. 4. Le taux d’apprentissage ainsi qu’une éventuelle stratégie d’évolution de celui-ci.
  5. 5. la taille des ensembles ou batchs d’observations considérés à chaque itération

Hyperparamètres

Hyperparameters related to neural network structure	**Hyperparameters related to the training algorithm**
•Number of hidden layers •Dropout •Activation function •Weights initialization	•Learning rate •Epoch, iterations and batch size •Optimizer algorithm •Momentum

Méthodes de réglage hyperparamètre

- Dans une expérience de réseau neuronal, vous allez généralement essayer plusieurs valeurs d'hyperparamètres et voir ce qui fonctionne le mieux.
- Pour évaluer le succès de différentes valeurs, reconformez le réseau en utilisant chaque jeu d'hyperparamètres et testez-le par rapport à votre jeu de validation.
- Si votre ensemble de formation est petit, vous pouvez utiliser la validation croisée - diviser le jeu de formation en plusieurs groupes, former le modèle sur chacun des groupes puis le valider sur les autres groupes. Voici les méthodes courantes utilisées pour accorder les hyperparamètres:

Méthodes de réglage hyperparamètre

- **Manual hyperparameter tuning**—an experienced operator can guess parameter values that will achieve very high accuracy. This requires trial and error.
- **Grid search**—this involves systematically testing multiple values of each hyperparameter and retraining the model for each combination.
- **Random search**—a research study by Bergstra and Bengio showed that using random hyperparameter values is actually more effective than manual search or grid search.
- **Bayesian optimization—**a method proposed by Shahriari, et al, which trains the model with different hyperparameter values over and over again, and tries to observe the shape of the function generated by different parameter values. It then extends this function to predict the best possible values. This method provides higher accuracy than random search.